

The Islamic University of Gaza
Deanship of Research and Graduate Studies
Faculty of Engineering
PhD in Computer Engineering



الجامعة الإسلامية بغزة
عمادة البحث العلمي والدراسات العليا
كلية الهندسة
دكتوراه هندسة الحاسوب

Proof-of-Collaboration: A Novel Blockchain Consensus Algorithm with Highly Efficient Power Consumption

**إثبات التعاون: خوارزمية توافق بلوك تشين جديدة ذات
كفاءة عالية في استهلاك الطاقة**

By

Raed S. Rasheed

Supervised by

Prof. Aiman AbuSamra

**A thesis submitted in partial fulfillment
of the requirements for the degree of
PhD in Computer Engineering**

08/2026

Abstract

Blockchain systems secured by Proof-of-Work (PoW) provide open participation and strong security, but their uncoordinated, competitive block-header hashing process can consume substantial energy. This thesis proposes Proof-of-Collaboration (PoCol), a novel consensus algorithm that preserves the openness of PoW, retains its hash-based puzzle and longest-chain structure, and is designed to eliminate most of the duplicate candidate-header inputs that arise in the evaluated common-template baseline. In PoCol, all miners participating in a mining round use the same immutable canonical block template and the same nonce-free header template, during that round, a miner instantiates candidate headers only by varying the nonce field. A deterministic, seed-based procedure partitions the permitted nonce domain among the active miners into disjoint ranges bound to that template. Each miner evaluates only the candidate headers generated from its assigned nonce range, and the resulting block and its pre-committed reward-distribution transaction are verified by all nodes on the chain. PoCol thereby eliminates duplicate serialized candidate-header inputs among honest miners within a mining round by requiring a common immutable template and assigning disjoint nonce ranges. The protocol specification includes mechanisms for miner registration, explicit and silent leaving, inactivity handling, and a proposed collaborative reward-distribution mechanism designed to improve fairness, together with a preliminary security analysis covering double spending, selfish mining, Sybil attacks, and deviations from assigned nonce ranges.

The thesis specifies the complete common-template PoCol protocol in Chapter 5. For evaluation, an idealized discrete-event abstraction of PoCol’s collaborative search and disjoint nonce allocation was implemented in an extended version of the BlockSim simulator with instrumented energy and carbon accounting, and compared against a PoW baseline across networks of 100 to 500 miners under identical, controlled simulation parameters. The full distributed common-template agreement protocol — including TemplateID enforcement, mempool reconciliation, and pre-committed reward enforcement — is specified but not yet executable in the prototype and remains a required implementation extension. Under these controlled simulation

assumptions, the results show that the simulated PoCol abstraction increases average throughput at scale (for example, 11.33 versus 2.87 transactions per second at 300 miners), shortens the approximate time to obtain six main-chain confirmations at larger network sizes, and reduces total energy consumption by approximately 98–99% (for example, 0.454 kWh versus 33.078 kWh at 400 miners), with a corresponding reduction in carbon footprint. These figures constitute proof-of-concept evidence for the collaborative search abstraction rather than measurements of a complete distributed implementation. The main identified limitation is an elevated stale-block rate at larger miner populations, this, together with the unmeasured communication and synchronization overhead of distributed template agreement in real networks, is set out as the principal direction for future work.

The thesis contributes a structured comparative literature review of blockchain consensus mechanisms, the complete PoCol protocol design, a sustainability-aware evaluation framework built on BlockSim, and simulation-based proof-of-concept evidence that collaborative mining can make PoW-class consensus substantially more energy-efficient under the evaluated assumptions while maintaining open participation.

Keywords: Blockchain, Consensus Algorithms, Proof of Work, Proof of Collaboration, Energy Efficiency, Sustainability, BlockSim.

الملخص

توفر أنظمة البلوك تشين القائمة على إثبات العمل (PoW) مشاركة مفتوحة ومستوى قويا من الأمان، إلا أن عملية التجزئة التنافسية غير المنسقة لرؤوس الكتل المرشحة قد تستهلك قدرا كبيرا من الطاقة. وتقتصر هذه الأطروحة خوارزمية توافق جديدة تسمى إثبات التعاون (PoCol)، تحافظ على انفتاح إثبات العمل وبنيتها القائمة على أحجية التجزئة وقاعدة السلسلة الأطول، وتهدف إلى الحد من تكرار مدخلات رؤوس الكتل المرشحة في خط الأساس ذي القالب الموحد المستخدم في التقييم. وفي كل جولة تعدين في PoCol، يستخدم جميع المعدنين قالب كتلة قانونيا موحدا وغير قابل للتغيير، وقالب رأس خاليا من قيمة النونس؛ وخلال الجولة لا ينشئ المعدن رؤوسا مرشحة إلا بإسناد قيم مختلفة إلى حفل النونس. ويقسم إجراء حتمي يعتمد على بذرة الجولة مجال قيم النونس المسموح بها إلى نطاقات منفصلة مرتبطة بالقالب، بحيث يقيم كل معدن رؤوس الكتل المرشحة الناتجة من نطاق النونس المخصص له فقط. وبذلك لا يحسب معدنان أمينان المدخل المتسلسل نفسه لدالة التجزئة خلال الجولة. ثم تتحقق جميع العقد من الكتلة الناتجة ومن معاملة توزيع المكافآت الملتزم بها مسبقا. كما يتضمن توصيف البروتوكول آليات لتسجيل المعدنين، والمغادرة الصريحة والصامتة، ومعالجة الخمول، وآلية تعاونية مقترحة لتوزيع المكافآت تهدف إلى تحسين العدالة، إضافة إلى تحليل أمني أولي يغطي الإنفاق المزدوج، والتعدين الأثافي، وهجمات سبيل، والانحراف عن نطاقات النونس المخصصة.

تحدد الأطروحة في الفصل الخامس بروتوكول PoCol الكامل القائم على القالب الموحد. ولأغراض التقييم، نفذ في نسخة موسعة من محاكي BlockSim تجريد مثالي قائم على الأحداث المتقطعة للبحث التعاوني وتوزيع نطاقات النونس المنفصلة، مع تسجيل استهلاك الطاقة والانبعثات الكربونية، ثم قورن بخط أساس لإثبات العمل في شبكات تضم 100 إلى 500 معدن تحت معلمات محاكاة مضبوطة ومتماثلة. أما بروتوكول الاتفاق الموزع الكامل على القالب الموحد، بما في ذلك فرض معرف القالب (TemplateID)، والمواءمة بين تجمعات المعاملات المعلقة، وفرض معاملة المكافأة الملتزم بها مسبقا، فهو محدد في الأطروحة لكنه غير منفذ بعد في النموذج الأولي، ويظل امتدادا تنفيذيا مطلوباً. وفي ظل افتراضات المحاكاة المعتمدة، تشير النتائج إلى أن تجريد PoCol المحاكى يزيد متوسط الإنتاجية عند التوسع، على سبيل المثال 11.33 مقابل 2.87 معاملة في الثانية عند استخدام 300 معدن، ويقصر الزمن التقريبي للحصول على ستة تأكيدات على السلسلة الرئيسية في الشبكات الأكبر، ويخفض إجمالي استهلاك الطاقة بنحو 98 إلى 99 بالمئة، على سبيل المثال 0.454 كيلوواط ساعة مقابل 33.078 كيلوواط ساعة عند استخدام 400 معدن، مع انخفاض مماثل في البصمة الكربونية. وتمثل هذه الأرقام دليلا على صحة المفهوم لتجريد البحث التعاوني في ظل افتراضات المحاكاة، وليست قياسات لتنفيذ موزع كامل. ويتمثل القيد الرئيس في ارتفاع معدل الكتل المتقادمة عند زيادة عدد المعدنين، إضافة إلى أن أعباء الاتصال والمزامنة المرتبطة بالاتفاق الموزع على القالب الموحد في الشبكات الفعلية لم يجر قياسها بعد، وتتطلب تقييما مستقبليا.

وتسهم الأطروحة بمراجعة أدبية مقارنة منظمة لآليات التوافق في البلوك تشين، وبالتصميم الكامل لبروتوكول PoCol، وبإطار تقييم يراعي الاستدامة ومبني على BlockSim، وبأدلة قائمة على المحاكاة تثبت صحة المفهوم وتشير إلى أن التعدين التعاوني يمكن أن يجعل آليات التوافق من فئة إثبات العمل أكثر كفاءة في استهلاك الطاقة بدرجة كبيرة، في ظل افتراضات المحاكاة المعتمدة، مع الحفاظ على المشاركة المفتوحة.

Acknowledgment

First and foremost, all praise and thanks are due to Allah, the Most Gracious and Most Merciful, for granting me the strength, patience, and perseverance to complete this work.

I would like to express my deepest gratitude to my supervisor, Prof. Aiman AbuSamra, for his continuous guidance, insightful feedback, and generous support throughout every stage of this research. His experience and encouragement were essential in shaping this thesis.

I am also grateful to the Faculty of Engineering and the Deanship of Research and Graduate Studies at the Islamic University of Gaza for providing the academic environment and resources that made this research possible, and to the members of the examination committee for their time and valuable remarks.

Finally, I owe special thanks to my family for their patience, unconditional support, and constant encouragement, without which this thesis could not have been completed.

Table of Contents

Abstract	I
المخلص	III
Acknowledgment.....	IV
List of Tables.....	XIII
List of Figures	XIV
List of Abbreviations.....	XV
Chapter 1 Introduction.....	2
1.1. Background.....	2
1.2. What the Problem Is and Why It Exists.....	3
1.3. Questions and Goals for the Research	5
1.4. Summary of the Research Methodology	6
1.5. Important Contributions.....	8
1.6. The Structure of the Thesis.....	9
Chapter 2 Technical Background	12
2.1. Blockchain Fundamentals.....	12
2.1.1. Distributed ledger and blocks	12
2.1.2. Network model and nodes.....	13
2.1.3. Transactions and Smart Contracts.....	14
2.2. Consensus Mechanisms in Distributed Systems.....	14
2.2.1. Classical consensus (BFT).....	15
2.2.2. Nakamoto-style consensus (Proof-of-Work)	15
2.2.3. Proof-of-Stake and its variants (DPoS, etc.).....	16
2.3. Energy Consumption in Blockchain Systems.....	18

2.3.1. Energy and power basics (definitions, metrics)	18
2.3.2. Empirical measurements of PoW/PoS energy consumption.....	18
2.4. Summary	19
Chapter 3 Literature Review and Related Work	22
3.1. Introduction.....	22
3.2. Methodology	24
3.2.1. Review Design	24
3.2.2. Information Sources and Search Period.....	25
3.2.3. Search Strategy	25
3.2.4. Eligibility Criteria	26
3.2.5. Study Selection Procedure	26
3.2.6. Data Extraction	27
3.2.7. Selection of Consensus Algorithms	27
3.2.8. Qualitative Comparative Coding Framework	28
3.2.9. Methodological Scope and Limitations	29
3.2.10. Clarification of Comparative Interpretation.....	29
3.3. Background.....	31
3.3.1. Definition of Consensus Algorithms in Blockchain	31
3.3.2. Consensus Algorithms in Blockchain	31
3.3.3. Importance of Consensus Algorithms.....	32
3.3.4. Historical Development of Consensus Mechanisms.....	33
3.4. Security Aspects of Consensus Algorithms.....	34
3.4.1. Security Features and Vulnerabilities	37
3.5. Energy Consumption in Consensus Algorithms.....	38
3.6. Overview of the Selected Consensus Algorithms	41

3.6.1. Byzantine Fault Tolerance (BFT)	44
3.6.2. Practical Byzantine Fault Tolerance (PBFT)	44
3.6.3. Proof of Work (PoW).....	45
3.6.4. Proof of Stake (PoS)	45
3.6.5. Delegated Proof of Stake (DPoS)	45
3.6.6. Proof of Authority (PoA)	46
3.6.7. Proof of Elapsed Time (PoET).....	46
3.6.8. Proof of Burn (PoB).....	46
3.6.9. Proof of History (PoH).....	47
3.6.10. Proof of Space (PoSpace)	47
3.6.11. Proof of Reputation (PoR)	47
3.6.12. Proof of Contribution (PoC)	47
3.7. Collaborative PoW Approaches	48
3.7.1. Parallel and Partitioned Mining	48
3.7.2. Collaborative and Team-Based Proof-of-Work.....	48
3.7.3. Energy-Aware Mining Policies.....	49
3.7.4. Auditable Mining Contribution.....	51
3.7.5. Comparison with PoCol and Identified Research Gap	51
3.8. Comparative Analysis and Research Question Synthesis	53
3.8.1. Comparative Analysis	53
3.8.2. Application-Oriented Selection of Consensus Algorithms.....	57
3.8.3. Answers to the Research Questions	59
3.9. Conclusion	62
Chapter 4 System Model and Problem Formulation.....	65
4.1. Overview of the Proposed Setting	65

4.2. Network Model	66
4.3. Adversary and Threat Model	67
4.4. Performance and Energy Metrics	68
4.4.1. Throughput, latency, and finality	69
4.4.2. Energy Metrics and Trade-offs	70
4.5. Goals and Needs for Design	73
4.6. Problem Statement (Formal).....	74
4.7. Summary	76
Chapter 5 The Proof-of-Collaboration Consensus Algorithm	79
5.1. Introduction and Design Goals	79
5.2. System and Threat Model	80
5.2.1. Network and Node Assumptions	80
5.2.2. Miner Identities and Cryptographic Primitives.....	81
5.2.3. Adversary Capabilities	81
5.2.4. Security Properties	82
5.3. PoCol State Representation on the Blockchain	82
5.3.1. Blockchain Ledger and Global State	82
5.3.2. Registered Miner Set.....	83
5.3.3. Balance and Reward State.....	83
5.4. Miner Life-Cycle: Registration, Activity, and Leaving.....	83
5.4.1. Miner Registration Protocol.....	83
5.4.2. Explicit Miner Deregistration	84
5.4.3. Handling Silent Leaves and Inactive Miners	84
5.4.4. Optional Inactivity-Based Removal.....	85
5.5. Overview of the PoCol Mining Round.....	85

5.5.1. Round Structure and Inputs.....	85
5.5.2. High-Level Steps.....	86
5.6. Deterministic Nonce-Domain Distribution.....	86
5.6.1. Round Seed	87
5.6.2. Pseudo-Random Ordering of Miners	87
5.6.3. Partitioning the Permitted Nonce Domain	88
5.6.4. Fairness Across Rounds	89
5.6.5. Handling Unevaluated Nonce Ranges	89
5.6.6. Post-Range Operating Policies.....	90
5.6.6.1. Energy-Saving Idle Policy	91
5.6.6.2. Continuous Performance Policy.....	91
5.7. PoCol Genesis Round	92
5.7.1. Genesis Preconditions and First Miner Setup	92
5.7.2. Genesis Miner Registration Sequence	92
5.7.3. Genesis Round Nonce Distribution and Collaborative Search	93
5.7.4. Genesis Reward Distribution and Ledger Initialization.....	93
5.8. PoCol Normal Rounds ($r \geq 1$).....	96
5.8.1. Reading State at the Start of Round r	96
5.8.2. Canonical Block Template Construction	96
5.8.3. Nonce-Domain Distribution in Round r	98
5.8.4. Collaborative Proof-of-Work Search	99
5.8.5. Block Proposal, Verification, and Commitment.....	99
5.9. Reward Distribution Mechanism in PoCol.....	100
5.9.1. Reward Distribution Transaction	101
5.9.2. Verification Rules	102

5.9.3. Inactive Miners and Policy Options.....	102
5.9.4. Interaction with Miner Set Dynamics	103
5.10. Dynamic Miner Set Management in PoCol.....	104
5.10.1. Joining New Miners	105
5.10.2. Leaving Miners (Explicit and Silent).....	106
5.10.3. Long-Term Inactivity and Forced Removal.....	108
5.11. Security and Robustness Analysis of PoCol.....	109
5.11.1. Resistance to Double Spending and Chain Reorganizations	110
5.11.2. Nonce-Domain Violations Analysis	110
5.11.3. Effects of Inactive and Partially Active Miners.....	112
5.11.4. Sybil Attacks and Miner Identity Management	113
5.11.5. Comparison with Existing Consensus Mechanisms	113
5.12. Summary.....	115
Chapter 6 Implementation and Experimental Setup.....	118
6.1. Chapter Scope and Objectives	118
6.2. Simulator Platform and Baseline Model.....	118
6.3. PoCol-Specific Implementation in BlockSim	119
6.4. Experimental Design	121
6.5. Logged Outputs and Data Collection Pipeline	123
6.6. Derived Performance, Energy, and Sustainability Indicators.....	124
6.7. Scenario Matrix and Traceability of Executed Runs	124
6.8. Reproducibility Procedure	125
6.9. Threats to Validity and Limitations of the Setup.....	126
6.10. Summary.....	127
Chapter 7 Experimental Results and Evaluation	129

7.1. Introduction.....	129
7.2. Evaluation Objectives	129
7.3. Performance Results	130
7.3.1. Throughput under varying network sizes.....	130
7.3.2. Latency and approximate confirmation time	131
7.3.3. Scalability and fault-tolerance	132
7.4. Energy Consumption Results	134
7.4.1. Total energy per block / per transaction.....	134
7.4.2. Comparison with PoW, PoS, PBFT, etc.	136
7.4.3. Energy–performance trade-offs	137
7.5. Sensitivity Analysis	138
7.5.1. Impact of topology, delay, churn	138
7.5.2. Impact of collaboration parameters.....	138
7.6. Discussion of Results.....	139
7.6.1. Interpretation relative to research questions	139
7.6.2. Practical implications for real-world deployments	140
7.7. Summary.....	141
Chapter 8 Conclusion and Future Work.....	143
8.1. Summary of Research Work.....	143
8.2. Revisited Research Questions and How They Were Answered	144
8.3. Contributions Revisited	145
8.4. Recommendations for Practitioners.....	146
8.5. Future Research Directions.....	147
References	150

List of Tables

Table (3.1) Inclusion and exclusion criteria	25
Table (3.2) Data extraction framework used in this survey	28
Table (3.3) Qualitative coding rubric for the security comparison table	29
Table (3.4) Qualitative coding rubric for the comparative summary table	31
Table (3.5) Evolution of Blockchain Consensus	34
Table (3.6) Comparative summary of representative security exposures	38
Table (3.7) Comparative summary of representative security exposures	57
Table (3.8) Qualitative comparative summary of selected algorithms	59
Table (3.9) PoCol vs. Collaborative PoW Approaches	52
Table (6.1) Fixed experimental parameters and output structure	122
Table (6.2) Summary of executed PoW and PoCol scenarios	125
Table (7.1) Summary of the main PoCol and PoW evaluation metrics	130

List of Figures

Figure (3.1) Illustrative electricity consumption trend of PoW-based	40
Figure (3.2) Monthly and cumulative electricity with Bitcoin mining	41
Figure (5.1) PoCol Round with Inactive Miner	90
Figure (5.2) PoCol consensus genesis round	95
Figure (5.3) PoCol reward distribution and verification in round r	104
Figure (5.4) PoCol miner leaving sequence	108
Figure (7.1) Average throughput for PoCol and PoW	131
Figure (7.2) Approximate six-block confirmation time	132
Figure (7.3) Stale-block rate for PoCol and PoW	133
Figure (7.4) Total energy consumption across miner populations	135
Figure (7.5) Energy per accepted main-chain block	136
Figure (7.6) Energy per transaction	136
Figure (7.7) Cumulative energy over time	137
Figure (7.8) Total carbon footprint for PoCol and PoW	138

List of Abbreviations

AI	Artificial Intelligence
ASIC	Application-Specific Integrated Circuit
BFT	Byzantine Fault Tolerance
CBNSI	Cambridge Blockchain Network Sustainability Index
CO₂	Carbon Dioxide
DAG	Directed Acyclic Graph
DLBC	Deep Learning-Based Consensus
DPoS	Delegated Proof of Stake
EVM	Ethereum Virtual Machine
FL	Federated Learning
IoT	Internet of Things
PBFT	Practical Byzantine Fault Tolerance
P2P	Peer-to-Peer
PoA	Proof of Authority
PoB	Proof of Burn
PoC	Proof of Contribution
PoCol	Proof of Collaboration
PoET	Proof of Elapsed Time
PoFL	Proof of Federated Learning
PoH	Proof of History
PoR	Proof of Reputation
PoS	Proof of Stake
PoSpace	Proof of Space
PoW	Proof of Work
PPT	Probabilistic Polynomial Time
PUE	Power Usage Effectiveness
RQ	Research Question
SHA-256	Secure Hash Algorithm 256-bit
TEE	Trusted Execution Environment
TPS	Transactions per Second
UTXO	Unspent Transaction Output

VPP	Virtual Power Plant
VRF	Verifiable Random Function

Chapter 1

Introduction

Chapter 1

Introduction

1.1. Background

Initially designed by Nakamoto as a peer-to-peer system for making payments electronically, blockchain technology is now widely used in banking, supply chain management, e-government, health care, and education. These applications rely on blockchain's decentralized, tamper-resistant ledger [1], surveys and domain-specific studies have shown that blockchain enables transparent and auditable processes (e.g. Circular Supply Chain [2], e-voting [3], Intelligent Real Estate Contracts for Smart Cities [4], Integrating AI-based Services in Healthcare and Agriculture [5], [6], etc.). In addition, blockchain technology is increasingly used in energy systems and virtual power plants for coordination and trading purposes.

The consensus mechanism forms the foundation of all blockchain systems by allowing groups of nodes—all of which do not trust each other—to validate the current status of a distributed ledger without relying on a single central authority. The consensus mechanisms have been derived from classical types of fault tolerant distributed systems such as the Byzantine Generals Problem and their solutions based on the concepts of Byzantine Fault Tolerance (BFT) [7] and Practical Byzantine Fault Tolerance [8]. As the popularity of blockchain technology has grown, so too has the variety and complexity of consensus algorithms based on these ideas. Examples include: Proof-of-Work (PoW), Proof-of-Stake (PoS), Proof-of-Space (PoSpace), Proof-of-Burn (PoB), Delegated Proof-of-Stake (DPoS), Proof-of-Authority (PoA), Proof of Elapsed Time (PoET), Proof-of-History (PoH), Proof-of-Contribution (PoC), and Proof-of-Reputation (PoR). Recent surveys provide extensive categorizations of these alternatives and of their relative merits in terms of security, decentralization, and performance. Research on validator behaviour further clarifies the security and incentive mechanisms of these consensus protocols.

Consensus algorithms, initially developed for the cryptocurrency sector, are now being repurposed to address challenges in various fields, including smart cities, power markets, renewable energy distribution, green power traceability, and the integration of machine learning methodologies like LightGBM into blockchain applications for

electricity usage analysis. Numerous studies report that the substantial energy consumption of Proof of Work (PoW) blockchains affects their operational sustainability, notably, the annual energy usage of the Bitcoin network can be comparable to that of a mid-sized nation, and its greenhouse-gas emissions are substantial [9], [10]. Comparable findings have been reported for other large networks, such as Ethereum before its transition to Proof of Stake (PoS) [11], [12]. Researchers have therefore investigated both more efficient hardware and alternative consensus designs. Energy-efficient Application-Specific Integrated Circuits (ASICs) for double SHA-256 have been studied using optimized architectures and multi-threshold-voltage techniques [13], [14]. Energy-aware and useful-work alternatives include Green-PoW [15], Proof-of-Learning [16], deep-learning-based consensus (DLBC) [17], and Proof of Federated Learning (PoFL) [18]. Other studies compare the energy consumption of non-PoW consensus mechanisms [19], [20], examine fairness and decentralization in PoS-based protocols [12], [21], and investigate fair participation and reward allocation among heterogeneous miners [22]. Recent evaluations of greener blockchain architectures similarly advocate sustainable consensus design while preserving essential blockchain properties [20], [23]. This thesis introduces the proposed Proof-of-Collaboration (PoCol) consensus algorithm to shift PoW mining from entirely competitive block-header hashing toward a collaborative architecture. PoCol uses collaborative mining over a common immutable block template, distributed nonce-range allocation, and a proposed collaborative reward-distribution mechanism designed to improve fairness, to eliminate duplicate serialized candidate-header inputs within each mining round and reduce energy consumption while aiming to retain security, decentralization, and economic incentives.

1.2. What the Problem Is and Why It Exists

The rising energy usage of Proof of Work (PoW) creates a sustainability challenge despite the established security and open-participation properties of PoW-based blockchain technology. The security of Bitcoin's PoW has demonstrated that a truly open and decentralized system can operate securely in an adversarial environment by binding the creation of blocks to computational work [1]. In fact, research shows that Bitcoin's total annual electricity consumption has grown dramatically from tens of terawatt-hours per year to levels in the low hundreds of

terawatt-hours per year (TWh/year)—an amount exceeding that consumed by various countries classified as "developed" or "industrialized" [9], [10]. De Vries showed that earlier estimates of total energy consumption in the Bitcoin environment underestimated electricity use because market dynamics and miner behavior were not fully represented [24]. Measurements of Ethereum prior to its transition to Proof of Stake (PoS) likewise indicate that full nodes and validator clients consumed substantial amounts of energy [11], [12]. Increasingly, as global climate-change mitigation and sustainability policies intensify, these levels of consumption become increasingly difficult to reconcile with sustainability goals [19], [25].

To address these issues, alternative consensus methods such as Proof of Stake (PoS) and its derivatives were subsequently developed, they reduce energy consumption by replacing computational work with the economic stake of the participating parties [26], [27]. At the same time, however, new concerns regarding centralization and wealth concentration as well as the complexity of the economic incentives associated with these newer consensus models are emerging [27], [21]. DPoS-based systems (e.g. BitShares [28], [29]), and later improvements to validator election [30], are able to process a large volume of transactions rapidly with minimal energy consumption. However, they only allow a small set of delegates to make decisions, meaning that they lack several features of decentralization and are often more incentivized to collude due to less decentralization than PoW systems [28], [29]. Permissioned consensus mechanisms such as PoA [31], [32] are much more energy-efficient and help manage identities, however, they provide less decentralization than PoW systems and are limited when applied in open, competitive markets. Energy-efficient consensus models include PoSpace [33], Proof of Burn [34], Proof of Elapsed Time [35], Proof of History [36], Proof of Consensus [37], and Proof of Research [38]. Each model trades some aspects of security, hardware assumptions, complexity, or deployment model for energy-efficiency.

A growing body of research examines green consensus models and approaches that redirect mining computation toward useful work. Green-PoW reduces wasted computation through modified competition and reward rules [15]. Proof-of-Learning and DLBC integrate machine-learning tasks into block validation [16], [17], while PoFL redirects mining computation toward federated-learning tasks [18]. Other

studies compare the energy consumption of existing and alternative consensus methods [19], [20], examine fairness and miner participation [22], and develop more energy-efficient SHA-256 hardware architectures [13], [14]. Although these alternatives provide important energy-efficiency improvements, none fully resolves the combined challenges of energy efficiency, decentralization, fairness, and security in open blockchain networks.

Based on the studies discussed above, this thesis aims to develop an energy-efficient consensus algorithm for open blockchain networks that: (i) uses less than 50% of the energy consumed by the evaluated Proof-of-Work baseline, (ii) is designed to maintain levels of decentralization and security comparable to Proof of Work, an objective that requires formal and empirical validation, and (iii) provides incentives intended to support fair participation while limiting excessive concentration of mining power. The proposed Proof-of-Collaboration model addresses this design challenge by requiring all miners in a round to mine one immutable common block template, assigning disjoint nonce ranges, and distributing rewards through a mechanism designed to improve fairness.

1.3. Questions and Goals for the Research

This thesis centers on the design, analysis, and evaluation of the Proof-of-Collaboration consensus mechanism, informed by the preceding challenge. The primary objective is to demonstrate that distributed nonce allocation and collaborative mining can significantly conserve energy while aiming to retain safety and decentralization comparable to PoW-based systems — an objective that requires formal and empirical validation [1], [20].

The main questions for the research are:

1. RQ1: How can the concepts of collaboration and resource sharing be integrated into a Proof-of-Work consensus mechanism to reduce redundant computation and energy consumption while maintaining security and accessibility?
2. RQ2: Under the controlled simulation assumptions used in this thesis, to what extent does collaborative mining with distributed nonce allocation reduce block-production energy relative to classical PoW across miner populations of 100 to 500?

3. RQ3: What effects does the proposed PoCol algorithm have on decentralization and fairness, on resistance to attacks such as double spending and selfish mining, and on the tendency toward mining-pool centralization, compared with PoW systems [21], [22]?

The thesis seeks to accomplish the following specific objectives to address these inquiries:

1. O1: To systematically evaluate the energy consumption, security assumptions, decentralization characteristics, and operational performance of representative blockchain consensus mechanisms [39], [20].
2. O2: To propose a formal system model and design principles for Proof-of-Collaboration, encompassing collaborative mining, distributed nonce allocation, and a collaborative reward-distribution mechanism designed to support fair allocation while reducing redundant computation and discouraging excessive concentration of hash power [1], [15].
3. O3: To develop and use a blockchain simulation testbed for comparing the idealized PoCol abstraction with the PoW baseline across varying miner populations under fixed latency, workload, hardware-efficiency, and simulation-duration parameters, fairness and decentralization metrics are defined analytically, and their experimental evaluation remains future work [40], [41].

This thesis focuses on the consensus and block-production layers of public or permissionless blockchain systems, in which any node may join, propose blocks, and participate in consensus subject to the protocol rules. Smart-contract languages, application-layer protocols, and detailed token-economic design are outside its scope. Case studies and motivating examples may derive from areas including financial transactions [1], [25], supply chains [2], [42], e-voting [3], real estate management [4], education [43], and energy systems [44], [45], however, the PoCol design aims to be universal and application-agnostic.

1.4. Summary of the Research Methodology

The research methodology followed a multi-stage iteration of theoretical analysis, simulation, and implementation. It comprised a review of the literature and

comparison with related work, the development of formal models and algorithms, the construction of a prototype, the execution of experiments, and the measurement of performance metrics.

The first phase comprised a review of the literature on blockchain consensus processes, on energy-efficient variants of such processes, and on energy-centric applications, conducted through survey and comparative research [19], [39]. The literature review established initial energy-usage benchmarks and security and efficiency reference points that the subsequent evaluations used for comparison. This phase analysed consensus processes based on Proof of Work (PoW) and its variants [1], [15], Proof of Stake (PoS) and Delegated Proof of Stake (DPoS) [26], [28], and several consensus processes based on machine-learning models [16], [18].

The second phase fully specified the Proof of Collaboration (PoCol) algorithm, including the threat model and the energy-metric scheme, and specified how collaborative mining operates, how nonce ranges are allocated, how blocks are validated, and how incentives are provided. Security and liveness were examined through analytical models and a preliminary, informal analysis [46], [47].

The third phase implemented the prototype and simulation environment used to model mining and energy use, employing established programming languages and tools. The simulator supports direct performance comparison of consensus algorithms across networks with differing miner populations, transaction arrival rates, and network structures [40], [41]. The evaluation reported in Chapters 6–7 measures throughput, approximate confirmation latency, stale-block rate, and energy per block and per transaction, together with the carbon footprint, across miner populations of 100 to 500 under fixed latency and workload parameters, reward-distribution fairness and decentralization metrics (e.g., Gini coefficients applied to miner income distributions) are defined in the analytical framework but were not experimentally evaluated in this thesis and remain future work [22], [20].

The collected simulation data were analysed descriptively (totals, averages, and derived ratios across the executed scenario runs) to estimate the energy saved in comparison with PoW, formal statistical significance testing and quantitative fairness

or decentralization analysis were not performed and are identified as future work [24], [20].

The methodology was designed to combine practical applicability of the PoCol algorithm with strong theoretical foundations, deployment and measurement beyond simulation remain future work.

1.5. Important Contributions

This thesis contributes to current research on energy-efficient distributed systems, blockchain consensus methods, and the development of sustainable digital infrastructures. The main contributions of this thesis are summarized in the following five categories:

1. C1: To provide a structured comparative examination of blockchain consensus mechanisms and their implications for energy consumption, covering established mechanisms such as PoW, PoS, DPoS, PoA, PoET, PoH, Green-PoW, and Proof-of-Useful-Randomness, together with emerging approaches such as Proof-of-Learning, DLBC, and PoFL. These mechanisms illustrate the trade-offs inherent in consensus design, primarily among security, decentralization, performance, and sustainability.
2. C2: To design a collaborative mining mechanism in which miners share a common immutable block template with disjoint nonce ranges, eliminating duplicate hash inputs among honest miners within each mining round and reducing energy use. The PoCol algorithm aims to optimize PoW-style mining by encouraging miners to collaborate and combine their efforts to create new blocks while increasing overall efficiency. PoCol retains the hash-based puzzle and longest-chain structure of Nakamoto-style PoW, providing security and probabilistic finality comparable to existing PoW algorithms is a design objective supported by a preliminary analysis (Section 5.11), and formal equivalence of the corresponding guarantees remains to be established for the complete protocol.
3. C3: To develop an analytical framework for evaluating PoCol that integrates models for energy consumption and decentralization, enabling structured comparison of PoCol and PoW from a variety of operational perspectives, using

metrics derived from previous empirical and theoretical research related to blockchain energy consumption and consensus effectiveness.

4. C4: To implement a simulation testbed and prototype for PoCol and a PoW baseline, providing a simulation-based experimental evaluation. The executed evaluation focused on energy savings and performance across networks with varying miner populations under fixed latency and workload parameters, the testbed additionally supports fairness assessment, which remains future work. The evaluation is an idealized simulation of the collaborative search abstraction, no real-world deployment or measurement was performed.
5. C5: To establish a foundation for future research on collaborative and energy-efficient consensus, including extensions of PoCol toward useful-work tasks, machine-learning objectives, and application-specific goals, informed by recent developments in machine-learning and federated-learning consensus methods.

1.6. The Structure of the Thesis

The remainder of the thesis is organized as follows.

Chapter 2 introduces the technical foundations required for the study, including blockchain architecture, cryptographic primitives, network models, Byzantine fault tolerance, Nakamoto-style Proof of Work, alternative consensus mechanisms, and the distinction between power and energy metrics.

Chapter 3 presents a structured comparative literature review of blockchain consensus mechanisms, with particular attention to functionality, security, energy consumption, and sustainability. It also examines empirical measurements, analytical models, and policy-oriented studies relevant to PoW, PoS, DPoS, Green-PoW, PoUR, PoFL, and machine-learning-based consensus.

Chapter 4 defines the system model, adversary assumptions, performance and energy metrics, design goals, and formal problem formulation for Proof of Collaboration (PoCol).

Chapter 5 specifies the PoCol consensus protocol, including canonical block-template construction, deterministic nonce-domain allocation, collaborative mining, miner-set management, reward distribution, and preliminary security and robustness

analysis. It also compares PoCol conceptually with representative consensus mechanisms such as PoW.

Chapter 6 describes the BlockSim-based prototype, the idealized PoCol abstraction implemented for evaluation, the experimental design, parameter settings, data-collection pipeline, reproducibility procedure, and threats to validity.

Chapter 7 reports the experimental results for throughput, approximate confirmation time, stale-block rate, total energy consumption, energy per block, energy per transaction, and carbon footprint. It compares the simulated PoCol abstraction with the PoW baseline and discusses the results under the stated assumptions, fairness and decentralization remain analytical or future evaluation dimensions.

Chapter 8 summarizes the findings and contributions, discusses the extent to which the research questions were addressed, identifies practical limitations, and outlines directions for future work.

Chapter 2

Technical Background

Chapter 2

Technical Background

2.1. Blockchain Fundamentals

A blockchain is a distributed ledger in which participating nodes maintain and validate an ordered record of state transitions according to a consensus protocol [1], [39]. Rather than relying on a single database administrator, blockchain systems distribute validation authority among participating nodes, and accepted state changes are governed by protocol rules [48], [39]. Cryptographic linking and replicated validation make previously accepted records tamper-evident, although the degree of immutability depends on the consensus model and its security assumptions. Blockchain technology has been applied in financial services, supply-chain management, electronic voting, real estate, healthcare, energy systems, and education [48], [45].

This section introduces the blockchain concepts required for the subsequent analysis of consensus mechanisms and energy consumption.

2.1.1. Distributed ledger and blocks

Blockchain systems organize validated state transitions in an append-only or append-mostly ledger whose integrity is protected by cryptographic commitments and consensus rules. In conventional linear blockchains, transactions are grouped into blocks, and each block header commits to the preceding block and to consensus-relevant metadata, such as the previous-block hash, timestamp, transaction commitment, and difficulty-related fields [1], [7]. Modifying an accepted historical block changes its cryptographic digest and invalidates subsequent links unless the attacker can also replace the affected chain according to the protocol's consensus rule. Node responsibilities differ: archival nodes may retain the complete history, full validating nodes verify consensus rules and maintain the state required for validation, validators or miners participate in block production, and light clients verify selected claims using block headers and cryptographic proofs. Bitcoin represents ledger state through unspent transaction outputs (UTXOs), where transactions consume previously unspent outputs and create new outputs assigned to recipients [49], [50]. Permissioned

systems may instead authorize a known validator set and employ Byzantine-fault-tolerant protocols such as PBFT and its variants [8], [51], [52]. Many blockchain systems can be modelled as replicated state machines: correct replicas apply the same agreed sequence of valid transactions and consequently reach the same state [7], [8]. Ledger structures may be linear, DAG-based, sharded, or consortium-oriented, and these designs make different trade-offs in throughput, latency, fault tolerance, storage, and decentralization [49], [39].

2.1.2. Network model and nodes

Blockchain data are disseminated over a peer-to-peer overlay, commonly through gossip-based propagation of transactions and blocks [1], [39]. Full validating nodes verify consensus rules and maintain the state required to validate new transactions and blocks, mining or validator nodes additionally participate in block production, and light clients use block headers and cryptographic proofs without storing or validating the complete ledger. The network is commonly modelled as partially synchronous: after an unknown global stabilization time, message delays are bounded, although the bound itself need not be known in advance [7], [8]. Bitcoin is a permissionless system in which participation is open to nodes that follow the protocol and meet the resource requirements of block production [1], [19]. By contrast, permissioned systems such as Hyperledger Fabric and consortium networks restrict validation to approved organizations or identities [31], [32]. Other designs weight participation by stake, organizational authority, reputation, or hybrid combinations of these factors [31], [38], [53]–[55].

Related abstractions have also been applied to energy and cyber-physical systems. Xu et al. proposed a blockchain-based decentralized scheduling model in which distributed participants coordinate flexible loads and renewable generation through a shared ledger [56]. Li et al. examined blockchain-based management and trading of power-data assets [57]. Other proposals use distributed ledgers for renewable-energy traceability, peer-to-peer energy trading, and profit distribution in virtual power plants [58]. These applications typically rely on permissioned or consortium networks with explicit trust assumptions, bounded participant sets, and application-specific requirements for network latency, privacy, and governance.

2.1.3. Transactions and Smart Contracts

Transactions encode authorized state changes. In UTXO-based systems, a transaction consumes one or more unspent outputs and creates new outputs that may be spent by their designated owners. Account-based systems update balances or contract state associated with accounts. Before inclusion in a block, a transaction must satisfy syntactic validity rules, authorization requirements such as digital signatures, and any protocol-specific state-transition constraints.

Smart contracts are programs deployed on a blockchain platform and executed deterministically by validating nodes when transactions invoke their functions. Some platforms support Turing-complete languages, whereas others deliberately restrict expressiveness to simplify verification or reduce risk. Contract code and consensus-critical state are committed to the ledger, while application data may also be stored or referenced off-chain depending on the platform. Smart contracts can reduce reliance on centralized intermediaries for processes whose rules and inputs can be represented and validated on-chain, but they do not eliminate governance, legal, or operational dependencies. Prior studies apply smart contracts to circular supply chains, electronic voting, real-estate transactions, smart-city services, agriculture, healthcare, and digital credentials. In these applications, transactions invoke contract logic, and the resulting state updates are replicated across validating nodes.

2.2. Consensus Mechanisms in Distributed Systems

Distributed systems employ consensus protocols so that non-faulty nodes agree on the order of state transitions despite crashes, network faults, or adversarial behaviour [7], [8]. The manner of achieving consensus on the order of the transactions in the blockchain also determines how secure, performant, and energy efficient the systems are. This section briefly summarizes classical consensus methods, Nakamoto consensus based on Proof of Work (PoW), the principal Proof of Stake (PoS) methods, and recent variants proposed to reduce the energy expenditure associated with these methods.

2.2.1. Classical consensus (BFT)

The traditional model of consensus research is prior to blockchain and assumes a closed environment with verified participants. By creating the Byzantine Generals Problem, Lamport described the difficulty in determining consensus among nodes that have failed or are acting maliciously (Byzantine) [7].

The Practical Byzantine Fault Tolerance (PBFT) method, created by Castro and Liskov, enables consensus among honest nodes provided that fewer than one-third of the nodes are Byzantine. In the normal case, PBFT proceeds through pre-prepare, prepare, and commit message phases over a partially synchronous network, with a view-change protocol to replace a faulty leader, safety relies on the one-third fault bound, while liveness additionally depends on the partial-synchrony assumption [8]. This work has been followed by many successful enhancements, such as the addition of proactive recovery [59], auditable logs for permissioned ledgers [51], performance enhancements to PBFT, and a PBFT variant called DPNPBFT that uses multiple primary nodes to provide increased scalability [52].

Many works also adapt BFT-style consensus approaches for domain-specific or untrusted environments. This includes extensions to support voting by groups [60], energy-limited IoT implementations [61], PBFT-based parallel consensus mechanisms used for Internet of Vehicle scenarios [62], and the use of PBFT in zero trust-based architectures used to exchange clinical data [63]. Other studies evaluate how well consortium chains will scale using PBFT [55] and what consensus methods are being used in networks like Hyperledger Fabric [64]. Generally speaking, though, while classical methods tend to offer rapid finality and consistency, they also require multiple rounds of communication over authenticated channels, the all-to-all message exchange of PBFT-style protocols entails $O(n^2)$ message complexity in the number of participants, which increases coordination cost and energy consumption as the validator set grows.

2.2.2. Nakamoto-style consensus (Proof-of-Work)

Nakamoto's protocol for Bitcoin represented a significant departure from traditional consensus models in a permissionless environment [1]. Miners repeatedly hash serialized candidate block headers and seek a header whose hash, interpreted as

an integer, is less than or equal to the target threshold. The 32-bit nonce field is one value miners vary, but a successful proof of work is a property of the complete header, not of the nonce in isolation. When a miner produces a proof-of-work-valid header and the corresponding block satisfies the remaining consensus rules, the block is propagated through the network. Honest nodes follow the valid chain with the greatest cumulative proof of work, the probability that a confirmed transaction is later reversed decreases as additional blocks extend that chain. The core security argument assumes that no attacker controls a majority of the network's total hash power and that honest nodes are sufficiently synchronized to converge on the same chain [1], [7], below this threshold, attacks such as selfish mining and double spending against transactions with few confirmations remain possible with lower success probability.

The primary benefit of using PoW as a method of validating transactions is its openness and conceptual simplicity: any person or entity capable of performing the necessary computations to create a valid block can participate, with no restrictions based on pre-existing identity or stake holdings. However, it is important to note that PoW is an extremely resource-intensive method of validating transactions due to the continuous computations done by the miners regardless of whether their blocks are found to be valid. A number of empirical studies that have examined the cost of PoW hardware and the use of double SHA-256 implementations for PoW operations demonstrate both the relatively high cost of the hardware and the energy intensity of ASIC-based PoW mining [13], [14]. Models that have been developed by various researchers to assess the total energy consumption of the Bitcoin network using market dynamics, hash rate data, and estimates of hardware efficiency indicate that naive extrapolations of energy use from these models will greatly underestimate the actual total energy consumption [24], [19]. Similarly, research has shown that the pre-merge Ethereum PoW chain was a significant consumer of energy for both mining and for maintaining full nodes on the PoW chain [11], [12].

2.2.3. Proof-of-Stake and its variants (DPoS, etc.)

Proof of Stake (PoS) replaces hash-rate-based Sybil resistance with economic stake. Early designs such as PPCoin selected validators probabilistically according to coin holdings and, in some variants, coin age. Later PoS protocols introduced random

beacons, validator committees, slashing conditions, finality mechanisms, and staking pools. Their security and liveness properties depend on the specific protocol, network model, validator-selection process, and economic assumptions, they therefore cannot be treated as universally equivalent to PoW, although their operational energy use is generally much lower. Comparative studies classify PoS protocols by validator-selection rules, chain structure, and finality mechanisms, while other proposals address stake concentration and long-range attacks. In Delegated Proof of Stake (DPoS), token holders elect a limited set of block producers, separating governance participation from block production. BitShares popularized this model, and subsequent work has refined delegate-election and multi-layer designs to balance throughput, governance, and fairness.

Other consensus mechanisms assign block-production authority according to identity, organizational role, reputation, or combinations of stake and reputation. Proof of Authority (PoA) relies on a fixed or governed validator set, typically established through consortium or institutional processes. Proof of Reputation weights participation using a score derived from prior behaviour or external evidence. Proof of Elapsed Time (PoET) uses trusted execution environments to generate randomized waiting periods for leader election, although this approach reduces continuous computation, research has identified vulnerabilities such as multi-certificate attacks in some PoET-style deployments.

Resource-based alternatives include Proof of Space, Proof of Replication, and Proof of Space-Time. Proof of Replication demonstrates that a prover stores a unique encoded replica of data, whereas Proof of Space-Time demonstrates that specified data remain stored over a defined interval. Proof of Burn assigns virtual mining weight by irreversibly sending assets to an unspendable address. Other mechanisms reward useful or application-specific contributions. Comparative reviews emphasize that Proof-of-X mechanisms make different trade-offs among decentralization, security assumptions, performance, governance, and resource cost, no single mechanism is optimal for every deployment context.

2.3. Energy Consumption in Blockchain Systems

The way blockchain consumes energy, particularly in proof-of-work (PoW) public networks [65], [66], has become a central concern in debates over the environmental impact and operating costs of blockchain technology. Comparing consensus mechanisms therefore requires precisely defined power and energy metrics, an understanding of how these metrics are measured empirically, and an assessment of how the environmental and economic effects of blockchain deployment arise.

2.3.1. Energy and power basics (definitions, metrics)

Power is the rate of energy conversion, expressed in watts (W) or megawatts (MW), energy is power integrated over time, expressed in joules (J), watt-hours (Wh), kilowatt-hours (kWh), or, at network scale, terawatt-hours (TWh). The blockchain literature reports energy consumption using several metrics: network power demand (MW), energy per block and energy per transaction (kWh), energy per hash (J/hash) in Proof of Work systems, and energy intensity relative to economic output (kWh per USD of transaction volume) [9], [24]. Meaningful comparison of consensus algorithms additionally requires documenting the hardware efficiency of the mining or validation equipment, the workload characteristics (transactions per second, block size, and smart-contract complexity), and the adversarial or security assumptions (e.g., the assumed fraction of adversarial hash power or stake).

De Vries and related studies estimate network power demand using bottom-up models based on observed hash rate, mining-hardware efficiency, and data-centre power usage effectiveness (PUE) [24], [25]. Cambridge indices report estimates of electricity consumption and associated emissions for major blockchain networks [10], [67]. Zhang et al. modelled climate-related effects of cryptocurrency energy use under alternative regulatory and technological scenarios [9]. Other studies examined Ethereum's energy profile before and after the transition from Proof of Work to Proof of Stake at the Merge [11], [68].

2.3.2. Empirical measurements of PoW/PoS energy consumption

Empirical estimates indicate that global PoW mining can consume electricity on the scale of national power systems, although the result depends on network activity,

hardware efficiency, electricity sourcing, and modelling assumptions [10], [69]. De Vries argued that fixed power-per-hash estimates may understate consumption when they omit the effects of market price, capital investment, hardware turnover, and expansion of mining capacity [24]. Sapra et al. reviewed how application workload, network topology, and hardware selection affect the energy consumption of PoW-based blockchain systems [19]. Truby analysed legal and policy options for reducing Bitcoin's environmental impact while noting possible tensions with innovation and financial inclusion [25].

Ethereum's transition from PoW to PoS provides a prominent comparison of consensus-related energy demand. Studies of pre-Merge Ethereum primarily attribute its high energy consumption to competitive PoW mining, while full-node operation accounts for a much smaller component [11]. Measurements of validators and full nodes in PoS-based Ethereum indicate substantially lower operational energy use than PoW mining, although PoS introduces different communication, hardware, and validator-management requirements [12], [68]. Bada et al. compared the energy characteristics of PoW, PoS, PBFT, and other mechanisms, noting that permissioned BFT protocols can have low energy demand at moderate validator-set sizes but face scalability limits as communication overhead grows [20].

Alternative consensus mechanisms often seek to reduce or repurpose the energy used by PoW. Green-PoW modifies competition and reward policies to reduce wasted computation under energy-aware conditions [16], [70]. Proof of Useful Randomness and Proof of Learning redirect required computation toward tasks intended to produce useful outputs [16], [17]. Proof of Federated Learning integrates collaborative model training into the block-production or validation process, although the exact role of training and producer selection depends on the protocol design [60]. Storage-based mechanisms replace much of the continuous hash computation of PoW with storage-capacity and proof-generation requirements, however, storage manufacture, operation, maintenance, and proof computation still impose energy and resource costs [33], [71].

2.4. Summary

This chapter has provided the technical context required to assess the impact of energy usage on blockchain consensus mechanisms. It first reviewed the fundamentals

of blockchain technology — the distributed ledger, peer-to-peer (P2P) networking, and the role of transactions and smart contracts across application areas [1], [45]. It then examined the consensus mechanisms used by distributed systems, including classical Byzantine Fault Tolerance (BFT) protocols, Nakamoto-style Proof of Work (PoW), and a range of Proof of Stake and other Proof-of-X (PoX) protocols, each based on different security assumptions and resource requirements for maintaining the ledger. Finally, it summarized empirical and theoretical work on the energy consumption of blockchains, including the associated environmental and economic impacts, at both the protocol level and in specific energy-system applications worldwide [19], [20].

Chapter 3

Literature Review and Related Work

Chapter 3

Literature Review and Related Work

3.1. Introduction

Historically, blockchain technology was mostly deployed in the cryptocurrency marketplace. Recently, many applications have been developed that take advantage of blockchain technology, for example supply chain management [72], healthcare, the Internet of Things, digital identity, finance, governance, and smart cities. In all blockchain systems, the consensus algorithm allows the distributed nodes to agree on the validity and ordering of transactions without a central controlling authority. The consensus mechanism therefore determines how secure, performant, decentralized, fault-tolerant, and sustainable a blockchain network can be.

Numerous consensus algorithms have been developed to address limitations identified in earlier protocols. Proof of Work, for example, has demonstrated strong robustness in open-ended adversarial environments, however, it requires a substantial amount of energy to operate, which has raised concerns regarding its sustainability and the operating cost of PoW mining hardware. To reduce these energy requirements, and to address at least in part the environmental impact of PoW, several alternative consensus mechanisms have emerged, including Proof of Stake, Delegated Proof of Stake, Practical Byzantine Fault Tolerance, Proof of Authority, and various hybrid and application-specific mechanisms intended to improve transaction throughput, reduce latency, reduce energy costs, and improve usability in permissioned networks and consortia.

The development of numerous consensus algorithms has enriched the consensus-design space, however, it has also made systematic comparison more difficult. Although several important surveys detail blockchain architecture, security, and consensus mechanisms, a considerable gap remains in the comparative understanding of these mechanisms. Many prior surveys have examined consensus mechanisms through one or a limited number of isolated lenses (e.g., taxonomy, architecture, protocol design, or application orientation), thereby failing to provide an overarching comparative framework in which functionality, security and energy consumption can be integrated to evaluate these mechanisms [73], [74]. The authors of these surveys have described the mechanisms using broad labels (e.g., secure, scalable, decentralized, or energy efficient), and have not established sufficiently consistent criteria for the rigorous cross-family comparison of these mechanisms. Most of the prior reviews remain predominantly descriptive, providing useful classification but limited decision-oriented synthesis for determining when a given consensus mechanism is most suitable for a specific deployment context and what makes it so [75], [76]. This limitation is increasingly important given the increased use of blockchain platforms for high-volume services, enterprise systems, low-energy and

resource-constrained IoT or fog-computing environments [77], [78], and sustainability-sensitive applications [73], [74].

In response to this gap, this chapter presents a structured comparative literature review of blockchain consensus algorithms across three analytical dimensions: functionality, security, and energy consumption. Rather than classifying mechanisms only by family or procedural logic, the review evaluates how they achieve agreement, the security guarantees and vulnerabilities associated with each mechanism, and their energy implications under different deployment conditions. This approach links theoretical properties to practical system-design decisions.

The review makes two principal contributions. First, it consolidates representative consensus mechanisms used in public, permissioned, consortium, and hybrid environments. Second, it introduces a comparative framework that evaluates each mechanism across three dimensions: (1) functionality, including validation model, finality, throughput, and scalability, (2) security, including resistance to intentional attacks and robustness against non-adversarial faults, and (3) energy consumption and sustainability, including operational efficiency and environmental implications.

The structured comparative literature review addresses the following research questions:

RQ3.1: Which principal functional characteristics serve to differentiate blockchain consensus algorithms with respect to their validation model, finality, fault tolerance, throughput, and scalability? [76]

RQ3.2: In what ways do prominent consensus algorithms diverge in their resilience to security threats, including Byzantine faults, majority attacks, Sybil attacks, collusion, and pressures toward centralization? [75], [76]

RQ3.3: How does energy consumption differ across the various consensus families, and which design decisions exert the greatest influence on the sustainability profile of blockchain systems? [73], [74]

RQ3.4: What trade-offs emerge among functionality, security, and energy efficiency in the selection of a consensus mechanism for a given blockchain application? [75], [76]

RQ3.5: Which consensus mechanisms are most appropriate for distinct deployment contexts, such as public blockchain platforms, consortium networks, enterprise environments, and resource-constrained systems? [76], [74]

Recent work continues to extend blockchain consensus design in new directions. Ladder, for example, is a convergence-based structured-DAG design that facilitates parallel processing of blocks, achieving increased throughput and lower confirmation

latency [79]. Similarly, Orthrus accelerates Multi-BFT consensus through concurrent partial ordering of transactions, reducing latency while maintaining a global ordering of all transactions [80]. Finally, Crackle is a fast sector-based BFT consensus protocol with sublinear communication complexity, improving scalability relative to communication-heavy BFT methods [81]. While these works do not replace the consensus families evaluated in the current survey, they are representative of the fact that a significant amount of current work on this topic focuses on addressing long-standing problems with throughput, latency, and/or communication overhead, particularly in high-performance and BFT-based applications [79], [81].

The principal contribution of this structured comparative literature review is not a new consensus mechanism or an exhaustive quantitative evaluation across all blockchain settings. Instead, it provides an analytical and decision-oriented framework that integrates functionality, security, and energy consumption and relates these dimensions to practical deployment contexts [73], [75].

3.2. Methodology

This chapter employs a structured comparative literature-review methodology informed by relevant PRISMA reporting principles. Because the review does not report a complete PRISMA flow diagram, database-level record counts, or a formal study-quality assessment, it is not presented as a full systematic review. The comparative framework evaluates each mechanism across three dimensions: functionality, security, and energy consumption, with the objective of providing a structured evidence-based synthesis rather than a quantitative meta-analysis.

3.2.1. Review Design

The review process was conducted in five stages:

1. formulation of the review objectives and research questions,
2. identification of relevant studies through database searching,
3. title and abstract screening,
4. full-text eligibility assessment, and
5. structured data extraction and qualitative synthesis.

This design was adopted to reduce selection bias, improve traceability, and ensure that the comparison of consensus algorithms was grounded in an explicit review process rather than ad hoc literature selection [82], [83]

3.2.2. Information Sources and Search Period

Utilizing the major academic publisher platforms and digital Libraries that provide Open Access to peer-reviewed Research on Blockchain and Distributed Systems, such as IEEE Xplore, ScienceDirect, and the ACM Digital Library, a literature search looking for blockchains was conducted over the years of 2008 to January/2025. This resulted in capturing both previously published foundational Consensus Research Papers for Blockchain technology and Latest Studies regarding Security, Scalability and Energy-Aware Protocol Development. In addition, backwards reference checking of the most relevant survey papers or technical research articles identified other possible sources to include which were not included in the original database search [84], [85] (see Table 3.1).

Table (3.1): Inclusion and exclusion criteria

Criterion Type	Criterion
Inclusion	Study proposes, analyzes, or compares a blockchain consensus mechanism
Inclusion	Study discusses functionality, security, scalability, or energy/energy consumption of consensus algorithms
Inclusion	Study is foundational, highly cited, or directly relevant to the historical evolution of consensus mechanisms
Inclusion	Study provides recent advances relevant to one or more selected consensus families
Exclusion	Study focuses on blockchain applications without substantial treatment of consensus design
Exclusion	Study lacks sufficient technical detail for comparative analysis
Exclusion	Study is duplicated or superseded by a more complete version
Exclusion	Study is non-scholarly, promotional, or lacks reliable technical grounding
Exclusion	Study addresses unrelated distributed-systems topics without direct blockchain-consensus relevance

3.2.3. Search Strategy

The search strings combined general blockchain-consensus terms with mechanism-specific and evaluation-oriented keywords. Representative search expressions included:

- “blockchain consensus” AND survey
- “consensus algorithm” AND blockchain
- “Proof of Work” OR PoW OR “Proof of Stake” OR PoS
- PBFT OR BFT OR DPoS OR PoA OR PoET OR PoH OR “Proof of Space”

- blockchain AND security AND consensus
- blockchain AND energy consumption AND consensus
- blockchain AND scalability AND consensus

The search syntax was adapted as required by each platform. Priority was given to peer-reviewed journal articles, conference papers, and highly relevant technical studies directly addressing blockchain consensus mechanisms and their comparative properties [84], [85].

3.2.4. Eligibility Criteria

Studies were included if they satisfied at least one of the following criteria:

1. they proposed, analyzed, or compared a blockchain consensus mechanism,
2. they discussed consensus from the perspective of functionality, security, scalability, or energy consumption,
3. they provided foundational or widely cited technical insights relevant to the evolution of blockchain consensus mechanisms, or
4. they introduced recent advances directly relevant to the selected consensus families considered in this survey.

Studies were excluded if they:

1. focused on blockchain applications without meaningful treatment of consensus design,
2. lacked sufficient technical detail for comparative analysis,
3. were duplicates or shorter versions of more complete studies,
4. were non-scholarly opinion pieces or promotional material, or
5. addressed unrelated distributed-systems topics without direct relevance to blockchain consensus [84], [85].

3.2.5. Study Selection Procedure

The procedure for obtaining the publications included removing duplicates and then implementing a systematic process for evaluating the titles and abstracts of each article before reviewing them in full-text form. Studies that met the requirements of the research objectives were fully reviewed, and only those that satisfied the

requirements specified for inclusion were included in the final published results of this work.

The selection process for the articles in this review was informed by PRISMA principles, including identifying articles to be included, undertaking the review of titles and abstracts, determining the articles' eligibility for inclusion, and finally selecting the articles for inclusion in the study. Unlike the previous systematic reviews which were statistical, this study presents the findings of the review process as a comparative study of existing literature guided by the evidence gathered. This publication makes no claims for completeness and exclusively relies on the available findings from the blockchain consensus literature [82], [83].

3.2.6. Data Extraction

For each included study, the following information was extracted in a structured manner:

- consensus mechanism or protocol family,
- validation logic and participation model,
- trust assumptions and deployment setting,
- fault tolerance characteristics,
- scalability and performance implications,
- security properties and reported attack surfaces,
- energy or power-related observations,
- representative application contexts.

This extraction scheme was aligned directly with the analytical dimensions of the survey and supported cross-study comparison across heterogeneous consensus families [84], [85]. Table 3.2 presents the data extraction framework used in this survey.

3.2.7. Selection of Consensus Algorithms

This survey includes consensus algorithms that are representative of the primary blockchain consensus families that have been identified in the literature. Selected algorithms include foundational protocols like BFT, PBFT, PoW, and PoS, as well as other influential protocols including DPoS, PoA, PoET, PoB, PoH, PoSpace, PoR, and

PoC. The intent of this survey was not to produce a definitive list of every consensus variant that has been published, but rather to identify mechanisms that represent the major design directions, assumptions about trust levels, and levels of resources used by blockchain consensus researchers. By presenting these algorithms and their relationships to each other in a comparative manner, this survey provides a more valid basis for comparing public, permissioned, consortium, and application-specific implementations.

Table (3.2): Data extraction framework used in this survey

Extracted Item	Description
Consensus family	PoW, PoS, BFT, PBFT, DPoS, PoA, PoET, PoH, PoSpace, PoB, PoR, PoC
Validation logic	Computational work, economic stake, delegated validation, trusted authority, storage commitment, etc.
Participation model	Permissionless, permissioned, consortium, hybrid
Trust assumptions	Open adversarial, bounded-trust, authority-based, reputation-based, contribution-based
Fault tolerance	Degree and type of fault or adversarial tolerance reported in the literature
Security properties	Representative attack surface, collusion exposure, majority-control susceptibility, centralization pressure
Performance and scalability	Throughput tendency, communication overhead, validator-set growth constraints
Energy consumption	Relative computational and infrastructural intensity of the validation process
Application suitability	Public blockchain, enterprise, IoT, consortium, high-throughput DApps, etc.

3.2.8. Qualitative Comparative Coding Framework

It was not possible to create a comprehensive and quantitative standard across all consensus families for all the studies included in this research, due to the heterogeneity of the findings, assumptions, metrics, and implementation models used in the various studies. Rather than creating a single, fully standardized quantitative benchmark for all the consensus families, the comparison tables presented in this research paper were created using qualitative synthesis of the evidence available. The terms low, moderate, high, limited, good and very good, were assigned based on recurring trends identified in the literature, with the classification for each consensus family, based on the architectural characteristics of each consensus algorithm, as they relate to the security of each algorithm. This process does not establish a quantitative standard for all consensus families, but rather provides a clear way to compare and assess the differences between consensus families, through the use of a comprehensive

qualitative rubric for combining different evidence types [84], [85]. The coding rubric used in creating the comparison tables of the security of each consensus algorithm is presented in Table 3.3 and the qualitative coding rubric used to generate the summary of the comparison of the various consensus algorithms is presented in Table 3.4.

3.2.9. Methodological Scope and Limitations

This review has several limitations. It is a structured comparative literature review rather than a statistical meta-analysis. The second limitation is that the literature on blockchain consensus contains a diverse range of terminology, terms, assumptions, and evaluation metrics, and as such, it is difficult to undertake a direct one-to-one comparison between the consensus families of all blockchain platforms. The third limitation is that the qualitative labels provided in the comparative tables should be treated as evidence-based synthesis indicators rather than strict quantitative benchmarks. We acknowledge these limitations to avoid the overstatement of the precision of the study and to provide methodological transparency.

Table (3.3): Qualitative coding rubric for the security comparison table

Dimension	Coding Logic
Representative Security Exposures	Assigned from the most frequently reported attack surfaces in the reviewed literature for each mechanism, including Sybil attacks, majority-control attacks, collusion, long-range attacks, centralization pressure, network partitioning, insider threats, and validator compromise
Qualitative Risk Indication: Low	Limited architectural exposure under the mechanism’s normal operating assumptions, with strong resistance in the intended deployment setting
Qualitative Risk Indication: Medium	Noticeable exposure to one or more relevant attack classes, but with mitigating architectural or governance controls reported in the literature
Qualitative Risk Indication: Medium to High	Recurrent exposure to multiple attack surfaces or strong dependence on concentration-sensitive assumptions such as stake, delegates, authority, reputation, or hardware trust
Majority-Control Attack Susceptibility	Marked “Yes” only when the literature or protocol structure supports an equivalent majority-control pathway, whether through hash power, stake concentration, delegate collusion, validator compromise, or reputation manipulation

3.2.10. Clarification of Comparative Interpretation

This research does not provide a generic, quantitative gauge for measuring blockchain consensus algorithms globally. However, it does present an organized, quality-supported comparative framework, produced through consistent and repeatable findings identified across the many studies reviewed within this document.

Consequently, the terms of low, medium, high, good, and very good reflect comparative synthesis indicators instead of static numerical thresholds or measurements of repeatable performance. Distinguishing between indicators and performance measurements is critical, as many of the variables surrounding the state's consensus mechanism are dependent on very different assumptions (e.g., the size of the network, the composition of the validator set, type of governance model, profile of hardware, details of implementation, or the context in which the application is utilized, and many other factors) [82], [84].

As a consequence, this paper presents, does not present a single consistency factor across functionality, security, and energy consumption for the survey. The importance of each of these parameters varies significantly from use case to use case. For example, open, public blockchains are likely to be focused on adversarial-resilient designs and decentralized solutions while organizations or consortiums will be more focused on throughput, having limited trusts, governance control, and use of power-efficient and lightweight validation in a resource-constrained environment. Therefore, the comparative synthesis presented in this work should be used to provide different interpretations of the results depending upon the use of the surrounding interpretations rather than use results across all use cases [83], [85].

Table (3.4): Qualitative coding rubric for the comparative summary table

Dimension	Coding Logic
Deployment Suitability	Assigned according to the typical governance and participation model reported for the mechanism: permissionless, permissioned, consortium, enterprise-controlled, or hybrid
Fault Tolerance	Summarized from the mechanism's commonly reported tolerance to faulty or adversarial nodes, taking into account whether resilience depends on hash power, stake concentration, bounded Byzantine assumptions, authority trust, or delegated honesty
Energy consumption: Very Low	No continuous mining competition and low validation overhead, typically found in authority-based, permissioned, or lightweight delegated settings
Energy consumption: Low	Limited computational burden under normal operation, with validator selection based on stake, delegation, reputation, or contribution rather than persistent computation
Energy consumption: Low to Moderate	Lower than PoW, but still involving non-trivial storage, hardware, or infrastructure overhead
Energy consumption: Moderate	Reduced computational intensity compared with PoW, but involving resource sacrifice, storage burden, or persistent auxiliary overhead
Energy consumption: Very High	Continuous computation-intensive validation, typically associated with mining competition as in PoW

Scalability: Limited	High communication overhead, low throughput tendency, or significant degradation with validator-set growth
Scalability: Good	Better validator efficiency or throughput than classical mining- or communication-heavy protocols, but still constrained by trust, governance, or infrastructure assumptions
Scalability: Very Good	High throughput tendency and relatively low coordination overhead in the mechanism's intended deployment environment
Security Profile	Assigned as a qualitative synthesis of adversarial resilience, trust assumptions, concentration risk, and exposure to representative attack surfaces in the reviewed literature

3.3. Background

3.3.1. Definition of Consensus Algorithms in Blockchain

Blockchain's consensus algorithms help multiple users around the world come together to verify and accept transactions as valid or not which will result in a new block being added to the transaction history of the blockchain network, they also record all information that is in this current version of the ledger, including all changes to that data, therefore, it is important for any distributed system to have a consistent, reliable method to handle validation of transactions and maintain up-to-date versions of the ledger.

Various consensus algorithms have developed to achieve these goals, some algorithms like Proof-of-Work use difficulty of computation as part of their validation approaches, while Proof-of-Stake relies on economic investment instead of reliance upon energy-consuming methods for validators selection. Additional mechanisms and Hybrid approaches have emerged to improve upon these methods in order to decrease the amount of time needed for validating blocks, increase the ability to scale up or down and their capacity to support permissioned (private) environments. It's important to note, when talking about the different consensus algorithms available, that they are also seen as choices of design which involve certain trade-offs in performance, security, decentralization and energy efficiency.

3.3.2. Consensus Algorithms in Blockchain

Consensus methods help ensure that blockchain systems continue to work effectively and that their data remains accurate. Proof of Work is the earliest (and most widely used) consensus method but requires high levels of computing power and energy to mine new blocks. Due to these concerns related to sustainability, other

methods of reaching consensus have been developed that focus on maintaining the security of the blockchain while reducing energy demand.

All consensus methods provide varying degrees of both strength and weakness with respect to providing security for the blockchain. Historically, PoW has been considered to be a secure method of achieving consensus, however, PoW is still vulnerable to an attack in which a single party controls enough mining power to maintain majority control over the network. In addition, while PoS (and subsequent models) reduce energy needed for the validation process, there are questions concerning the level of concentration or economic centralization amongst validators. More recent approaches (hybrid methods) attempt to provide stronger performance with sufficient assurance of security against attack.

In addition to exploring new consensus methods, the energy demand associated with the operation of the blockchain has been recognized as an area that can be improved by optimizing the hardware and architecture used in the operation of the blockchain [13], [86].

3.3.3. Importance of Consensus Algorithms

Consensus has particular importance to blockchain technology, because typically transactions are being agreed to by decentralized parties, which do not want to (or cannot) rely on a centralized party to add them to a shared, trusted record. Consensus algorithms validate new transactions, prevent inconsistencies in the shared ledger, and allow blockchain systems to safely operate in an environment in which actors may misbehave or fail. How consensus is achieved can affect security, throughput, of course, but also effects governance, and whether a given blockchain is an open system or closes around a handful of players [87].

This goes beyond technical procedure: consensus design can affect who can add transactions, influence, and if the blockchain can be sufficiently open and wide-reaching, or not. Effectively one can't just study consensus algorithms in terms of computational efficiency, one must study for their implications for governance [87].

Table (3.5): Historical overview of the principal blockchain consensus algorithms considered in this survey, their representative platforms or deployment examples, approximate year of introduction, and primary references.

No.	Consensus Algorithm	Representative Platforms / Deployment Examples	Approximate Year Introduction	Primary of Reference
1	Byzantine Fault Tolerance (BFT)	Early distributed consensus models, conceptual foundation for permissioned consensus systems	1982	[7]
2	Practical Byzantine Fault Tolerance (PBFT)	Hyperledger Fabric, Byzantine-resilient permissioned blockchain designs	1998	[8]
3	Proof of Work (PoW)	Bitcoin, Litecoin, early Ethereum	2009	[1], [88]
4	Proof of Stake (PoS)	Cardano, Ethereum 2.0, other stake-based public blockchains	2012	[26]
5	Proof of Space (PoSpace)	Chia	2013	[33]
6	Proof of Burn (PoB)	Conceptual blockchain systems based on irreversible asset sacrifice	2013	[34], [89]
7	Delegated Proof of Stake (DPoS)	EOS, TRON, Steem	2014	[28]
8	Proof of Authority (PoA)	VeChain, enterprise and permissioned blockchain settings	2017	[31], [90]
9	Proof of Elapsed Time (PoET)	Hyperledger Sawtooth	2017	[35], [91]
10	Proof of History (PoH)	Solana	2018	[36]
11	Proof of Contribution (PoC)	Contribution-based or incentive-driven blockchain models	2018	[37]
12	Proof of Reputation (PoR)	Reputation-aware governance-oriented blockchain environments	2019	[38]

Note: The listed platforms and deployment examples are representative rather than exhaustive. The table is intended to provide historical and contextual orientation only and should not be interpreted as a comparative assessment of performance, security, or energy efficiency.

3.3.4. Historical Development of Consensus Mechanisms

Blockchain consensus mechanism evolution can be traced back to earlier studies into distributed agreement and Byzantine fault tolerance. Core concepts around achieving trustworthy agreement in systems with faulty or malicious nodes were developed through foundational models such as Byzantine Fault Tolerance (BFT) [7] and the Practical Byzantine Fault Tolerance (PBFT) [8], which informed the eventual development of blockchain consensus protocols.

With the introduction of Bitcoin in 2009 came the major milestone of demonstrating how Proof-of-Work (PoW) could enable decentralized consensus within a public and trustless environment [1]. The high energy costs associated with PoW encouraged the development of alternative consensus models, notably Proof-of-Stake (PoS) [26], [92], as well as a number of other mechanisms that seek to improve scalability, efficiency, or suitability to specific applications [73]. The result of several years of development has been the emergence of various consensus algorithms, including Proof-of-Space (PoSpace) [33], Proof-of-Burn (PoB) [34], Delegated Proof-of-Stake (DPoS) [28], Proof-of-Authority (PoA) [31], Proof-of-Elapsed-Time (PoET) [35], [93], Proof-of-History (PoH) [36], Proof-of-Capacity (PoC) [37], and Proof-of-Resilience (PoR) [38], reflective of the rapidly increasing diversity of blockchain requirements. New and innovative proposals are being put forward periodically that seek to improve efficiencies and adaptabilities in specific situations [94], [56].

To assist with understanding the historical context of all these algorithms table's purpose is to provide a description of the primary platforms shown within the table (i.e., where the consensus mechanisms have been implemented), it does not intend to rank the performance of any of the consensus-associated mechanisms relative to one another. Sections later in this report provide more comprehensive detail on analysis relative to functionality, security, and energy consumption using objective analytical measures. Table 3.5 provides a summary of the leading consensus algorithms included in this review with examples of the platforms/protocols where they were used (or will be), the approximate year they were first introduced, and references to the most complete work on each algorithm. Note that while the table's purpose is to provide a description of the primary platforms shown within the table (i.e., where the consensus mechanisms have been implemented), it does not intend to rank the performance of any of the consensus-associated mechanisms relative to one another. Sections later in this report provide more comprehensive detail on analysis relative to functionality, security, and energy consumption using objective analytical measures.

3.4. Security Aspects of Consensus Algorithms

The security of blockchain consensus algorithms is a key aspect of the overall design of these protocols because the consensus mechanism provides a means for

participants to agree on the state of the distributed ledger, regardless of whether or not some of those participants may behave incorrectly or maliciously. The consensus mechanism must provide for both safety (no conflicting states of the ledger) and liveness (the ledger continues to progress even in the presence of failures or adversarial actions).

While an algorithm's ability to defend against attacks will certainly impact the algorithm's overall security, the extent to which an algorithm's security is based upon that algorithm's trust assumptions, communication model, and resource limitations will also significantly contribute to the overall level of security provided by the protocol.

Given these uncertainties in the design of consensus algorithms and the multiple attack vectors associated with consensus mechanisms and their implementation within various blockchain systems, a high-level overview of the various classes of attacks and corresponding security concerns related to those attacks is provided in Table 3.6. The intent of the summary provided in Table 3.6 is to offer a comparative descriptive guide for evaluating the relative security of varying consensus algorithms, rather than to characterize the actual risk associated with each of the attack types described. Therefore, while the risk levels listed in Table 3.6 represent qualitative assessments of the consensus algorithms' overall exposure to each of the attack types listed in the table, the actual magnitude of each of the risks listed in the table is dependent upon various factors, including the algorithm's implementation and operational details, the distribution of validators, the design of validators' incentives, and the implementation context of the consensus algorithm.

Different families of consensus mechanisms utilize very different methods for achieving security. Consensus-based approaches to blockchain security differ primarily in how security is obtained. Proof-of-work (PoW) relies on the computational expense associated with large-scale manipulation of the network and thereby creates a disincentive to attempt it financially, whereas Byzantine fault-tolerant (BFT) approaches rely on a well-defined structure of coordination between the various replicas that allows the system to continue to function even though a limited number of replicas may be malicious or faulty [95], [96]. In contrast, both stake-based and delegated consensus models reduce the amount of computational

resources wasted, yet their overall security relies more heavily upon validator incentives, concentration of stakes, governance, or delegated authority relating to the use of the blockchain. Thus, the security of a given blockchain cannot be measured by a single, universal metric, rather, blockchain security should be perceived as a trade-off among openness, fault tolerance, decentralization, and resistance to attack.

As with all consensus algorithms, another aspect of blockchain security is the relative distinction between external, adversarial attacks and internal vulnerabilities within the protocol itself. Certain consensus algorithms may be subject to several different types of attacks, such as Sybil attacks, a majority-controlled attack, collusion, long-range attacks, network partitioning, or validator compromise. Depending upon their architecture and their respective participation model, different consensus algorithms could experience these threats differently. For example, PoW is primarily defined in association with the existence of a majority-of-hash-power attack, while PoS-based protocols could experience the additional threats of nothing-at-stake, long-range, and concentration attacks. Additionally, while DPoS and proof-of-authority (PoA) have been found to provide greater operational efficiency than other consensus algorithms, they also expose participants to a greater degree of risk associated with collusion or centralization due to the concentrated validation powers exercised by a relatively small number of actors within the system. Therefore, different consensus algorithms should be evaluated based upon their individual attack surfaces rather than as having identical assumptions of security associated with their operations [97], [98].

In addition to considering the security associated with a particular mechanism, operational efficiency must also be evaluated. Although BFT-style algorithms can provide strong consistency guarantees under conditions of adversarial attacks, the communication overhead associated with them could limit the scalability of these systems as the number of participants increases [95], [99]. However, in contrast to this, there exist mechanisms whereby greater throughput or lower latency can be achieved through limiting the number of validators that are participating in the consensus process or relaxing decentralization. Thus, the manner in which a consensus algorithm secures the overall security of a blockchain should be viewed as a mechanism that serves multiple goals within the broader context of the system. For instance, the characteristics of a consensus mechanism that is appropriate for use in a public,

permissionless blockchain would be different than those used in a consortium or private blockchain implementation [100], [101].

3.4.1. Security Features and Vulnerabilities

The way a consensus algorithm distributes authority for validating data and what the algorithm assumes about honesty of participants both have a bearing on that algorithm's security profile. For example, PoW's security comes from its reliance on the computational difficulty of mining blocks and the willingness of miners to engage in competition, and this has provided PoW with a high degree of success in extremely hostile environments, however, PoW can still be vulnerable to 51% attacks in situations when hashing power is sufficiently concentrated or under some other form of collusion [97], [102].

The energy efficiency of PoS can provide for an improvement in the efficiency of the mechanism, but does create new vulnerabilities of its own due to the nature of stake concentration, long-range attacks, and the ability for economic power to produce a disproportionate influence over the protocol [97], [92].

BFT and PBFT-based systems have the capabilities of providing a high level of fault tolerance as long as the percentage of malicious nodes remains below the protocol threshold, thus, they are typically better suited for environments where there are partially trusted and/or permissioned participants rather than public ones [95], [103]. Their effectiveness is also dependent on stable communication and bounded participation, in addition to which message complexity typically limits their ability to be deployed successfully at large scale. DPoS and PoA are differentiated from other consensus algorithms because the validators are either selected in advance or voted onto the role by a participant using some other form of authority, but this restriction ultimately increases the risks of collusion, insider manipulation, and governance capture [100], [104]. Ultimately, there is no single consensus algorithm that can be declared to be universally "safe", rather, each individual consensus algorithm will have unique attributes that define its relative resiliency against attacks and structural concentration.

Table (3.6): Comparative summary of representative security exposures, qualitative risk indication, and potential susceptibility to majority-control attacks across major blockchain consensus algorithms.

No.	Consensus Algorithm	Representative Security Exposures	Qualitative Risk Indication	Potential Susceptibility to Majority-Control Attack
1	BFT	Byzantine faults, message delay, network partitioning, adversarial coordination	Medium	No
2	PBFT	Byzantine behavior, network partitioning, replica coordination failures	Low to Medium	No
3	PoW	51% attack, Sybil attack, mining centralization	Low to Medium	Yes
4	PoS	Majority-stake attack, nothing-at-stake, long-range attack, stake concentration	Medium to High	Yes
5	PoSpace	Majority-resource concentration, Sybil attack, grinding attack	Medium	Yes
6	PoB	Majority-influence concentration, centralization risk, economic manipulation	Medium	Yes
7	DPoS	Delegate collusion, voter apathy, governance capture, centralization	Medium to High	Yes (by collusion)
8	PoA	Validator compromise, centralization, identity abuse, single-point trust dependency	Medium	Yes (by validator compromise)
9	PoET	Insider attacks, malicious SGX enclave behavior, trusted-hardware dependence	Medium	No
10	PoH	Sequencing-layer dependence, centralization risk, validator concentration	Medium to High	Yes
11	PoC	Contribution falsification, insider manipulation, concentration of influence	Medium	No
12	PoR	Reputation abuse, collusion, Sybil attack, concentration through reputation control	Medium	Yes (by reputation manipulation)

Note: The qualitative risk indication in this table reflects broad architectural exposure reported in the literature and does not replace formal security analysis, empirical attack modeling, or deployment-specific assessment. In particular, “majority-control attack” is used here in a generalized sense to include equivalent concentration-based attack pathways, whether through hash power, stake, validator collusion, authority compromise, or reputation manipulation

3.5. Energy Consumption in Consensus Algorithms

It is important to consider a methodological caution when comparing energy consumption of various consensus algorithms. Measurement results for the energy or power consumption of an algorithm can vary considerably based upon several factors - such as how it is implemented, its network size, its validator set size, the type of

hardware used, the type of communications, whether it is being implemented in a public, permissioned, or consortium, the application that is being addressed, etc. Accordingly, this survey provides a comparative and qualitative representation of energy consumptions in order to illustrate general industry trends for various architectural types rather than to provide an accurate comparison of cross-protocols under a common testing methodology [73], [74].

This survey only examines energy consumption from a Comparative Analysis perspective rather than a Universal Standard. Among the major consensus families, proof of work (PoW) is the most energy intensive because it utilizes an ongoing hash-based competitive model to keep the network secure. Current energy estimating methodologies, including the Cambridge Bitcoin Electricity Consumption Index, have developed models that estimate required energy using hardware-based top-down models, thereby offering annualized electricity estimates rather than using fixed protocol-level constants. This makes sense, the environmental impact of PoW cannot be viewed in terms of abstract algorithms, as the type of miner hardware, miner-profitability, source of electricity and neighbor/ competitor actions will all affect how PoW ultimately impacts the environment.

The Energy Information Administration (EIA) recently produced preliminary estimates for the United States, suggesting that cryptocurrency mining may be responsible for anywhere between 0.6% to 2.3% of all the electricity used in the United States — thus emphasising the significant policy implications associated with this activity.

Conversely, proof of stake (PoS) and delegated proof of stake (DPoS) type systems significantly decrease energy consumption because they do not incentivized mining competition, rather, they choose validators based on stake and delegated participation. Ethereum's transition from PoW to PoS is often referred to as the best example to illustrate the differences between the two types of systems and it has been stated that annualized electricity use decreased by 99.9% following the transition from the PoW to PoS protocol.

Additionally, while PoS and DPoS types of protocols result in a lower consumption of electricity, one should not assume that this means that they are the

only types of protocols that generate environmental concerns. Validator infrastructure, hardware production, redundancy in validators, and a concentration of validator governance may also affect the overall sustainability profile of both PoS and DPoS based protocols.

Another methodological consideration includes the use of per-transaction energy metrics. Although these metrics are often used in public discussions, they can create a misleading impression of energy consumption when viewed as a universal metric across all blockchain networks because many blockchain networks use energy to secure the whole of the blockchain rather than to process a specific number of transactions. As such, this survey does not consider per-transaction energy estimates as being a primary comparative metric, and instead focuses on computational intensity and infrastructure usage for each consensus family, and implications for sustainability and deployment suitability.

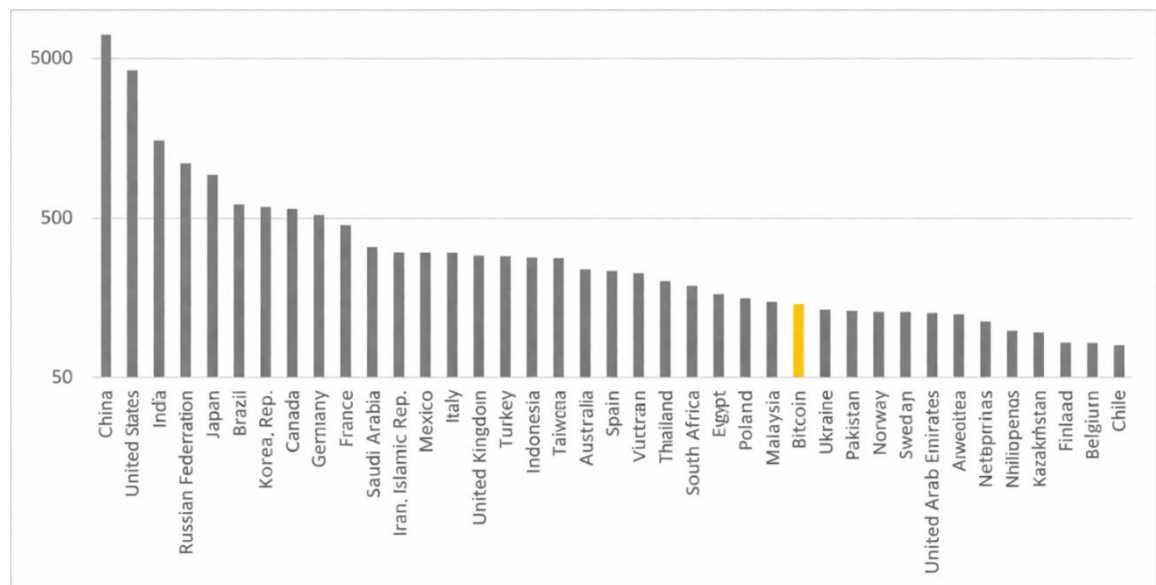


Figure (3.1): Illustrative electricity consumption trend of PoW-based blockchain operation, with Bitcoin used as a representative example of energy-intensive consensus (Annual electricity consumption in TWh) [10].

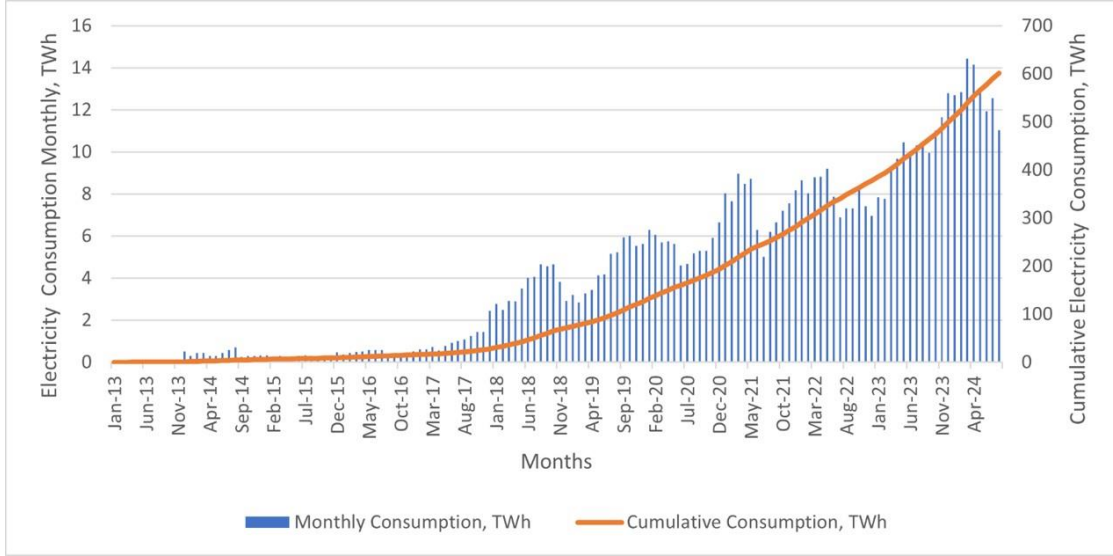


Figure (3.2): Monthly and cumulative electricity consumption associated with Bitcoin mining, illustrating the broader sustainability implications of computation-intensive PoW validation [10].

As a means of offering some initial empirical insights with respect to the energy consumed as a result of a Blockchain's design, the figures presented in Figures 3.1 and 3.2 demonstrate the amount (in kWh) of electricity consumed by the PoW-based Bitcoin Blockchain, with the goal of providing a representative example of the electricity consumption associated with operating on a Blockchain that employs PoW as its consensus mechanism. The purpose of these figures is not to infer a similar consumption profile will also occur for PoS-type consensus mechanisms but rather to represent the enormity of the energy burden that has been a catalyst for the development of more energy-efficient alternatives to PoW including, but not limited to, PoS, DPoS and other low-computation consensus mechanisms. These figures, therefore, represent contextual evidence of the sustainability issues posed by energy-intensive consensus and will also be used to further the ongoing discussion surrounding the energy consumption differences among the various consensus mechanisms through the use of a qualitative assessment lens [105], [106].

3.6. Overview of the Selected Consensus Algorithms

This chapter surveys the major categories of consensus algorithms (CAs), including classical, stake-based, delegated, authority-based, and new or specialized

consensus mechanisms. The survey characterizes the range of trust assumptions, participation models, and deployment environments reported in existing review publications, rather than attempting to compile a comprehensive list of all CA protocols published to date. The evaluation criteria were functionality, security, and energy consumption, these were identified as representative of the trade-offs most important to CA design, namely operational effectiveness, resilience to adversarial behaviour, and sustainability. The evaluation criteria are also consistent with prior IEEE Survey publications that highlighted the need to provide structured, multidimensional evaluations of CAs across a variety of blockchain environments.

Architecturally, we can group the various types of CAs into families: A family of Byzantine Fault Tolerant (BFT) protocols, such as BFT and PBFT, focus on achieving consensus under adversarial circumstances. BFT protocols are generally thought to be best suited for permissioned blockchain implementations. Consensus mechanisms based upon resource association, including PoW, PoS, DPoS, PoA, PoET, PoB, PoH, and PoSpace have varying levels of resources associated with determining how to validate transactions. Some examples of these resources include computing power, economic investment, trusted third-party authority, time passed since a previous event occurred, proof-of-destruction of cryptocurrency, sequentially ordered proof of occurrence, and a promise that storage space has been reserved for that transaction [107], [28]. As well, more recently emerged mechanisms such as PoR and PoC have built on this resource-based approach by adding a layer of validation participation to the basis of a validation participant's trustworthiness as well as their contribution to the overall system via measured behavior or result contributions [37], [38].

The evolution of these mechanisms can be described as the ongoing search for a balance between three competing priorities of consensus mechanisms: strongest possible security, maximum number of transaction validation per second, and least amount of resource consumption required by a consensus validation method to accomplish these goals. The first-generation consensus mechanisms, e.g. PoW, were designed to maximize participation and resilience to attack, but these benefits came at an expense of producing a large amount of energy consumption and having limited scalability [108], [109]. Subsequent mechanisms like PoS, DPoS, and PoA were

designed to reduce the computation power needed to validate a transaction, thus improving through-put, at a greater than normal risk of stake concentration, collusion, or centralization of governance [26], [30]. Likewise, BFT mechanisms provide a high degree of consistency in "bounded trust" environments, but this consistency comes with a cost of communication overhead that limits their scalability in large networks [62], [110].

More recent proposals for consensus mechanisms help to find ways to remove some of the inefficiencies associated with previous generation mechanisms and find new avenues for achieving efficiency from the perspective of both validation mechanisms themselves and how they fit into the overall ecosystem. For example, PoET utilizes trusted execution environments to assign random waiting periods to reduce the amount of computational waste produced by a PoW-type mechanism [111], [112]. PoH combines the concepts of verifiable ordering to decrease the amount of overhead needed to co-ordinate transactions and to provide quantitative empirical metrics to evaluate through-put, thus improving validation throughput [36], [113]. PoSpace improves energy efficiency by replacing computational power with the commitment of storage space, while also creating necessary opportunities for future innovations related to hardware and lifecycle modifications [33], [71]. Reputation- or contribution-based consensus mechanisms like PoR and PoC significantly expand the range of validation mechanisms by tying their validation authority to a history of performance, rather than simply relying upon raw computational power or economic investment [55], [63].

As a whole, these resource-based validation mechanisms highlight that there is no one-size-fits-all consensus mechanism. Instead, every family of consensus mechanisms displays a different opportunity of relative balance between, for example, how to effectively distribute power among users, supporting both fairness of opportunity to access the system, providing for secure access to the system, appropriately handling abuse of systems, governing the structure of the governance of users, and finally maintaining a sustainable model. For this reason, the study of consensus mechanisms should not be conducted as an isolated examination of a particular protocol, but rather should include a comparative analysis of how a consensus mechanism performs under varying levels of operational and security

assumptions. The following subsections will focus on the major consensus families contained within this survey. Each of the sections will discuss their respective mechanisms, security properties, and impact upon the use of computing resources and power consumption [7], [8].

3.6.1. Byzantine Fault Tolerance (BFT)

An agreement mechanism is established through Byzantine Fault Tolerance (BFT), allowing distributed systems to reach consensus even if there are untrustworthy or malicious nodes. In addition, reaching a consensus or agreement among parties that have conflicting interests but are using a Blockchain requires that the parties reach agreement using a method that cannot be impacted by the actions of some nodes in the network. BFT-based designs offer some advantages over Mining based mechanisms as the amount of excess computing power needed to confirm transactions can be reduced, however, the effectiveness of BFT based mechanisms are determined by how many faulty (bounded) node assumptions are held and the degree of coordination among participants who communicate [7], [114]. Therefore, BFT will be used preferably among distribution environments that have a higher degree of structure around validator identity and/or validator participation than do the fully open public distribution networks.

3.6.2. Practical Byzantine Fault Tolerance (PBFT)

PBFT is a typical working system utilizing Byzantine fault tolerance as the best example of this type of system. PBFT can maintain both safety and liveness guarantee as long as one-third of the participants in the system are working properly. The three-phase communication process used by PBFT ensures that all participants in the network reach consensus deterministically, therefore, PBFT is well suited to permissioned blockchains that use relatively small sets of validators. However, the communication overhead included in the three-phase communication process increases significantly as the network grows in size making PBFT difficult to scale, despite its ability to ensure strong consistency [115], [116]. As such, PBFT can be understood as a system with strong assurance of correctness, but limited large-scale applicability due to high communication costs.

3.6.3. Proof of Work (PoW)

Proof of Work (PoW) secures the blockchain networks by requiring miners to solve complex cryptographic puzzles (computationally intensive) before proposing a new block to add to the blockchain. Historically, PoW technology has played an essential role in securing permissionless blockchains, due to the economic cost associated with manipulating the blockchain and providing participants with the ability to participate in a decentralized manner. However, PoW has limitations such as its large energy consumption, high latency, and the risk of forming mining monopolies. As such, while PoW continues to be an effective security model, its sustainability over the long term has increasingly come into question.

3.6.4. Proof of Stake (PoS)

Unlike PoW where miners compete to create each block in the blockchain through the resource-intensive process of mining, PoS is a protocol that employs financial incentives and disincentives based upon an individual's financial investment in the network (their "stake"). Validators in PoS protocols receive rewards for creating new blocks, these rewards are based upon their financial investments in the network. While PoS solves the problem of high energy consumption associated with PoW protocols, it introduces new risks (centralization of stakes, long chain attacks, and nothing at stake), all of which may be mitigated by an effective protocol design (e.g. including provisions for slashing or checkpointing). Therefore, although PoS is more energy efficient when compared to PoW, PoS networks' security depends greatly upon how an effective protocol design incentivizes and distributes participants/validators within the network.

3.6.5. Delegated Proof of Stake (DPoS)

The Delegated Proof of Stake (DPoS) adds upon the features of the Proof of Stake (PoS) method by enabling token holders to select delegates who will be responsible for validating transactions and creating new blocks. The use of DPoS allows for increased throughput, and reduced energy consumption due to the fact that there will be fewer validators to process any given transaction [30], [117]. The downside of this design is that it creates an environment where the delegates have a larger opportunity to collude with one another and can create a situation where there

is a small group that has complete control over the network [118], [119]. DPoS networks can be very appealing for performance sensitive applications, but the ability to decentralize the DPoS network is much weaker than that of other participation models.

3.6.6. Proof of Authority (PoA)

PoA gives the group of approved validators known trust and decentralized identities, allowing for a quicker confirmation process, reduced computational burden and overall suitability for regulated environments that want to ensure accountability and also align with safety standards [120], [121]. The concentration of Authority in the few validators raises questions of centrality, reliance on trust, lack of transparency relative to public-based blockchain models [122], [123].

3.6.7. Proof of Elapsed Time (PoET)

PoET incorporates trusted execution environments that provide randomized waiting times to eliminate the high-energy consumption associated with competing to find a leader and utilizes a hardware-based mechanism for leader selection. Therefore, PoET is much more energy-efficient than proof-of-work while still maintaining a certain degree of probabilistic fairness. The main drawback of this system is that it requires the existence of trusted hardware, which creates a separate security assumption and may become vulnerable if the trusted execution environment is compromised.

3.6.8. Proof of Burn (PoB)

PoB enables network validators to gain validation power through the irreversible destruction of coins (i.e., "burning coins"), thereby displaying an economic commitment to the PoB network [34], [52]. By not requiring continual intensive computational work, PoB is considered to be more energy efficient than PoW for the purposes of achieving validation and obtaining rewards. However, due to the potential for PoB to create concentration of influence among those who can afford to lose assets, there are concerns related to deflation and fairness in participation [124], [125].

3.6.9. Proof of History (PoH)

PoH uses a verifiable ordering of events (i.e., transactions) which allows for minimal coordination overhead among validators when reaching consensus on the order of transactions [113], [126]. Additionally, this feature provides PoH with exceptionally high throughput capabilities, making PoH well suited as a performance-centric blockchain system. However, due to the design of PoH, there is a significant reliance on the security of the timing and sequencing mechanism in place. A compromise at this level could negatively affect the network overall in terms of trust and correctness [113], [127].

3.6.10. Proof of Space (PoSpace)

Whereas PoW trades in computing power to be able to participate in validating transactions on the blockchain, PoSpace allows for storage commitment as the resource requirement for validating participants. Because of this shift in requirements, PoSpace provides a more environmentally sustainable alternative to PoW by using less energy through decreased processing requirements [33], [128]. Nevertheless, when evaluating the practical sustainability of this alternative method, one must consider the device manufacturing processes associated with PoSpace, the life cycle of storage hardware, and the increased risk of grinding attacks or concentration of resources among those with a larger amount of available storage capacity [64], [128].

3.6.11. Proof of Reputation (PoR)

PoR allocates the validation authority to participants in accordance with their reputation built up through their past behavior in the network by encouraging continual honest behaviour and removing reliance on either raw computational ability or financial ability for validation [38], [59]. By improving efficiency and lowering energy consumption, PoR makes possible governance and fairness issues, such as collusion in reputation formation, entry barriers for new entrants, and concerns about the transparency of the process of measuring and updating reputation [129], [130].

3.6.12. Proof of Contribution (PoC)

Validators are chosen in PoC-based systems based on the value of their contributions to the overall operation of the system, such as providing computational

power, providing data, or providing some other contribution of value [131], [132]. The objective of this model of selection is to reward people who have contributed in some way to the operation of the system rather than merely rewarding those who own resources. This model reduces the reliance on energy-consuming forms of consensus, however, the security of the PoC system is based on how well the quality of contributions are assessed, and how well they are protected against manipulation, duplication, or collusion [131], [133].

3.7. Collaborative PoW Approaches

3.7.1. Parallel and Partitioned Mining

Hazari and Mahmoud proposed Parallel Proof-of-Work as an approach for distributing mining computation among cooperating participants through a coordinating manager [134]. The approach retains the conventional hash-target Proof-of-Work puzzle while allocating mining tasks among multiple miners to improve block-generation performance and reduce duplication of assigned work. It also considers coordination and reward allocation within the parallel mining structure.

Parallel Proof-of-Work is the closest architectural predecessor to PoCol because both approaches divide mining work among multiple participants rather than allowing every participant to perform an entirely uncoordinated search. However, PoCol defines the distributed work more narrowly and binds it explicitly to the consensus round. In PoCol, all participating miners use one identified immutable candidate-block template, and the permitted nonce domain associated with that template is deterministically partitioned into mutually disjoint ranges. Thus, honest miners do not evaluate identical serialized header–nonce inputs within the same collaborative round.

PoCol therefore does not claim to be the first approach to parallelize Proof-of-Work or distribute mining computation. Its proposed distinction is the integration of deterministic nonce-range separation, immutable common-template agreement, collaborative reward allocation, and explicit post-range operating policies.

3.7.2. Collaborative and Team-Based Proof-of-Work

StrongChain introduces weak Proof-of-Work headers that make non-winning computational contributions visible and rewardable [135]. Instead of preventing

overlapping work before mining, StrongChain recognizes useful evidence of mining effort after it has been generated. Its main contribution lies in reducing reward variance and making mining participation more transparent.

Collaborative Proof-of-Work forms dynamic mining groups based on participant capabilities and emphasizes secure contribution verification and fair reward distribution [136]. This is directly relevant to PoCol because both protocols seek to reward miners participating in collaborative block production. Collaborative Proof-of-Work, however, focuses primarily on group formation, contribution verification, and reward fairness, while PoCol focuses on deterministic separation of the nonce ranges searched under a common immutable candidate-block template.

Proof of Team Sprint is a recent team-based collaborative consensus algorithm intended to reduce the energy inefficiency of conventional Proof-of-Work [137]. It organizes miners into teams that collaborate in solving the cryptographic mining puzzle and evaluates the effects of cooperation on energy use and reward fairness. Proof of Team Sprint is therefore the most recent directly related peer-reviewed collaborative consensus approach considered in this thesis.

PoCol differs from these approaches by defining collaboration at the level of exact nonce-range assignments associated with a common template. Nevertheless, StrongChain, Collaborative Proof-of-Work, and Proof of Team Sprint provide important precedents for contribution visibility, reward fairness, and organized teamwork. These studies also expose a requirement that PoCol must address more rigorously: a miner should not receive a collaborative reward merely by registering or claiming that its assigned range was completed.

3.7.3. Energy-Aware Mining Policies

Green-PoW reduces energy consumption by restricting mining participation during selected rounds to a smaller group of eligible miners [15]. Its energy-saving mechanism is based primarily on reducing the number of simultaneously active miners. Proof of Team Sprint follows a team-oriented approach in which collaborative execution is intended to reduce the amount of active computation attributed to individual participants [137].

PoCol adopts a different energy-management principle. At the beginning of a collaborative round, all eligible miners may participate and receive deterministic, mutually disjoint nonce ranges. Under the proposed energy-saving policy, a miner that exhausts its assigned range without finding a valid candidate header stops active hashing and enters a low-power idle state until the deadline of the current collaborative mining round.

The energy consumed by miner i under this policy is expressed as:

$$E_i = P_i^{\text{active}} t_i^{\text{active}} + P_i^{\text{idle}} t_i^{\text{idle}} + E_i^{\text{coord}}$$

where $P_i^{(active)}$ is active mining power, $P_i^{(idle)}$ is idle power, and $E_i^{(coord)}$ represents communication, coordination, and state-transition overhead. Accordingly, nonce-range partitioning alone does not guarantee energy reduction. Energy savings arise only when miners spend a non-zero period in a power state that consumes less energy than active hashing.

PoCol may alternatively operate under a continuous-performance policy. In this mode, miners that complete their assigned ranges continue processing newly assigned non-overlapping work rather than entering the idle state. If aggregate hash rate, hardware efficiency, and experiment duration are equal to those of a continuously operating PoW baseline, this mode is expected to consume approximately the same energy. It may, however, improve accepted-block production or transaction throughput under particular evaluated configurations. Such performance improvement must be established experimentally and must not be treated as an automatic consequence of nonce partitioning.

Green-PoW and PoCol therefore reduce active mining through different mechanisms. Green-PoW reduces the number of miners allowed to participate, whereas PoCol reduces the active hashing duration of miners after they complete their assigned work. This distinction allows PoCol to preserve wider initial participation while exposing an explicit trade-off between lower energy use and continued mining performance.

3.7.4. Auditable Mining Contribution

A central challenge in collaborative and partitioned mining is verifying that a miner actually performed its assigned computation. A dishonest miner could claim that it exhausted its range, enter the idle state, and later request a share of the collaborative reward without having evaluated the assigned candidate headers.

Auditable Proof-of-Work provides a mechanism through which miners can produce probabilistic evidence that specified regions of the nonce domain were searched [138]. Although APoW is primarily directed toward detecting work omission and block-withholding behavior in mining pools, its auditing principle is relevant to PoCol. APoW is available as a 2026 arXiv preprint rather than a peer-reviewed journal or conference publication.

A strengthened PoCol design could associate each assigned nonce range with cryptographic commitments, sampled shares, or auditable search evidence. Such mechanisms would help distinguish miners that genuinely completed their assignments from miners that falsely claimed completion, thereby reducing free-riding, false idling claims, and reward manipulation.

3.7.5. Comparison with PoCol and Identified Research Gap

The reviewed approaches address different parts of the problem targeted by PoCol. Parallel Proof-of-Work distributes mining computation and provides the closest architectural precedent for parallel work allocation [134]. StrongChain improves contribution visibility by recording weak Proof-of-Work solutions [135]. Collaborative Proof-of-Work emphasizes dynamic group formation, contribution verification, and reward fairness [136]. Proof of Team Sprint organizes miners into collaborative teams and connects teamwork with energy efficiency and fairness [137]. Green-PoW reduces energy consumption by limiting the number of active miners in selected rounds [15]. APoW provides a possible mechanism for proving that assigned nonce regions were actually searched [138].

However, these approaches do not jointly formulate all of the following elements within one protocol: agreement on one identified immutable candidate-block template, deterministic assignment of mutually disjoint nonce ranges, collaborative reward

distribution, explicit post-range idle and continuous operating policies, and verifiable completion of assigned work.

PoCol is proposed to address this combined design space. Its contribution is not the independent invention of parallel mining, nonce-range distribution, collaborative rewards, or miner idling. Rather, its proposed contribution lies in integrating these mechanisms within one collaborative Proof-of-Work framework and evaluating the resulting energy–performance trade-off.

Table (3.9): PoCol vs. Collaborative PoW Approaches

Characteristic	Parallel PoW	StrongChain	Collaborative PoW	Proof of Team Sprint	Green-PoW	APoW	PoCol
Conventional hash-target puzzle	Yes	Yes	Yes	Yes	Yes	Yes	Yes
Parallel collaborative mining	Yes	Yes	Yes	Yes	Limited by round	Auditing-oriented	Yes
Immutable shared candidate template	Coordinated work	Not central	Not central	Not central	Not central	Not central	Explicit
Deterministic nonce-range allocation	Work allocation	No	Not central	Not central	No	Audited regions	Yes
Explicit mutually disjoint nonce ranges	Implementation - dependent	No	Not central	Not central	No	No	Yes
Contribution visibility or verification	Limited	Weak-header evidence	Explicit	Team-based	Runner-up evidence	Probabilistic audit	Requires extension
Energy-saving mechanism	Not primary	Not primary	Efficiency-oriented	Team cooperation	Fewer active miners	Not primary	Reduced active time
Post-range idle policy	No	No	No	Not explicit	Non-selected miners stop	No	Explicit
Continuous-performance policy	Yes	Yes	Yes	Team-dependent	No	No	Explicit
Main strength	Parallel work allocation	Contribution visibility	Fair collaboration	Team cooperation	Direct active-miner reduction	Auditable search evidence	Integrated work partitioning and

								operatin g policies
Main limitation	Manage r depende nce	Does not prevent overlap	Pool complexity	Team assumpti ons	Restrict ed participat ion	Not complete consensus protocol	a	Contrib ution proof and complet e distribut ed impleme ntation remain incompl ete

The comparison does not imply that PoCol is universally superior to the examined protocols. Parallel PoW provides the closest architectural baseline, Green-PoW provides the most relevant energy-policy baseline, Proof of Team Sprint provides the most recent peer-reviewed team-based collaborative baseline, and APoW provides a relevant contribution-auditing direction.

3.8. Comparative Analysis and Research Question Synthesis

The results presented in this section are intended to provide a context-specific synthesis of these consensus mechanisms, through a comparative analysis of existing studies. A consensus mechanism that is successful in one deployment model may not necessarily be successful in another due to the varying trade-offs between decentralization, fault tolerance and throughput, scalability, governance and energy consumption in different application domains. As such, the subsequent discussion will be structured according to the research questions that were used to guide the synthesis of these mechanisms and the suitability of the mechanisms for their intended applications rather than by means of a single hierarchical or universal ranking of consensus mechanisms [75], [139].

3.8.1. Comparative Analysis

Meaningful comparisons between blockchain consensus algorithms are more than just classifying the types of consensus mechanisms, rather, a more complete comparison needs to look at how each mechanism configured the tradeoffs around functionality, security and energy consumption under different assumptions of trust and deployment of the mechanism. "Consensus protocols vary with respect to their

validator selection model, fault tolerance model, structural characteristics of validator participation, and resources that they demand...This will influence scalability, resistance to attack, and decentralization and sustainability." [140], [141] Thus, there is no way to consider one type of consensus mechanism to be optimal in all contexts. Each type of consensus mechanism needs to be examined in terms of how it is designed in context and based on the objectives and constraints that are placed on a given system.

From the perspective of security, Proof-of-Work consensus mechanisms are some of the most secure for open adversarial environments as they connect control to a level of computation incurred, thus forcing attackers to expend significant resources to execute large-scale attacks. However, Proof-of-Work mechanisms present vulnerabilities to centralization of large amounts of hash power and therefore exhibit high energy consumption. [140], [24] In contrast, Proof-of-Stake consensus mechanisms provide much greater reductions in energy consumption by eliminating mining competition and allowing for "validation" based exclusively on ownership of currency, but will be more dependent on stake distribution, design of validation incentives, and protection against centralization of the validator set. Likewise, Byzantine Fault Tolerant methodologies provide good guarantees of consistency/availability and fault tolerance when there is a defined limited set of validators, however, their communication demands reduce their scalability as the validator set increases.

Diverse tradeoff characteristics can also be illustrated through comparing alternate consensus mechanisms. Proof-of-Delegated-Stake and Proof-of-Authority speed up throughput and reduce costs for block validation by defining the mechanism for validators as quasi-elected delegates or validators defined a priori. The increased efficiency of these mechanisms can be accompanied by a) an increased centralization of validators or b) an increased risk of collusion among validators. Proof-of-Burn, Proof-of-Reputation, and Proof-of-Contribution are additional mechanisms that attempt to align validators with some level of economic sacrifice, behavioral integrity or measurable "contributions" with respect to the operation of the blockchain system. While these mechanisms represent attempts to achieve desirable outcomes, they also create new governance and security vulnerabilities. Thus, while comparative analysis

of consensus mechanisms should not identify one mechanism as “the best”, rather it will help us understand which mechanisms provide the best fit for a specific application environment.

In summary, the comparison of various consensus algorithms demonstrates that the process of designing blockchains is ultimately the process of measuring competing objectives and determining how to be compromise between trust and distrust. In certain instances, the protocols that maximize openness and security will exhibit a higher degree of energy and latency while others that improve throughput and energy efficiency will come with more restrictive assumptions about trust or considerably more limited levels of validator participation. For these reasons, comparative evaluation of consensus protocols should be conducted with an awareness of the application context, level of acceptable centralization of validators, security threats for the blockchain, and sustainability requirements of the blockchain to be designed.

3.8.1.1. Security, Scalability, and Energy Trade-offs

When it comes to consensus mechanisms within blockchain technology, the main form of tradeoff between these systems is based on their ability to achieve security, scalability, and energy efficiency. Proof of Work is able to provide high amounts of security in public blockchains, but it has very large power consumption and limited scalability [142]. Proof of Stake improves efficiency by reducing energy use, but introduces issues associated with criteria for understanding stake concentration and governance [26], [33]. Byzantine Fault Tolerant consensus mechanisms have the ability to provide great consistency and fault tolerance (especially in private blockchains), but do so at the expense of low scalability due to issues relating to communications complexity [7]. New and emerging models such as Proof of Burn, Proof of Resources, and Proof of Certified/Chain Code/Digital Assets aim to create a better balance between these areas by minimizing the waste of resources and broadening the pool of potential validator selection, but still have context and evidence specific questions regarding fairness of validator selection, participation criteria, and ability to be manipulated [22], [37].

3.8.1.2. Comparative Summary

The totality of comparative evidence demonstrates the requirement to treat consensus algorithms as models of coordination that depend on application and not simply as interchangeable technical solutions. While PoW continues to be important in providing security for an open network environment, PoS and DPoS provide more efficient alternatives with different levels of centralization risk. BFT based protocols continue to be useful in providing security in a bounded-trust environment, while newer models (i.e., PoB, PoR and PoC) are continuing to expand the design space. Therefore, comparative evaluation of consensus algorithms supports structured criteria for evaluation of consensus algorithms that consider the operational effectiveness, survivability and sustainability of consensus algorithms in a unified manner.

Table (3.7): Comparative summary of representative security exposures, qualitative risk indication, and potential susceptibility to majority-control attacks across major blockchain consensus algorithms.

Consensus Algorithm	Most Suitable Deployment Context	Main Strength	Main Limitation
PoW	Open public blockchains operating in highly adversarial environments	Strong robustness and openness in permissionless settings	Very high energy consumption and limited scalability
PoS	Public blockchains seeking lower energy use with broad participation	Improved energy efficiency with strong economic incentive structure	Vulnerable to stake concentration and governance imbalance
DPoS	High-throughput decentralized applications and performance-sensitive public platforms	High throughput and low validation overhead	Greater exposure to delegate collusion and centralization
PoA	Enterprise systems, private blockchains, and permissioned IoT deployments	Fast confirmation and low resource consumption	Strong reliance on trusted authorities and limited openness
BFT	Small- to medium-scale permissioned systems with bounded trust	Strong fault tolerance under adversarial conditions	Communication overhead limits scalability
PBFT	Consortium blockchains requiring deterministic finality and strong consistency	High assurance and deterministic agreement	Poor scalability with large validator sets
PoET	Trusted-hardware-based enterprise or consortium environments	Low computational cost and efficient validator selection	Dependence on trusted execution environments
PoH	High-performance blockchain systems prioritizing ordering efficiency and throughput	Very high throughput and reduced coordination overhead	Greater architectural complexity and dependence on sequencing assumptions

PoSpace	Resource-conscious public blockchain systems seeking lower energy use than PoW	Lower energy burden compared with mining-intensive models	Dependence on storage resources and hardware-related constraints
PoB	Specialized public blockchain systems using economic sacrifice as a participation signal	Avoids continuous mining competition	Fairness and concentration concerns linked to asset destruction
PoR	Governance-controlled or reputation-sensitive blockchain environments	Rewards consistent historical behavior	Vulnerable to reputation manipulation and barriers to new entrants
PoC	Application-specific systems where useful contribution can be measured	Aligns validation influence with system contribution	Difficult contribution assessment and manipulation risk

Note: This matrix is intended as a qualitative decision-support summary derived from the comparative analysis presented in this survey. The listed deployment contexts are representative rather than exclusive, and the suitability of a given consensus mechanism may vary depending on validator composition, governance model, threat environment, performance targets, and sustainability requirements

3.8.2. Application-Oriented Selection of Consensus Algorithms

Because different consensus algorithms have different trust assumptions, validator structures, and resource constraints, the best consensus algorithm (the right tool for the right job) can only be determined based on the application requirements that a blockchain will support, not based on which is the best design. Therefore, in practice, selecting a consensus algorithm will be a function of balancing openness of the consensus mechanism, attack resistance, scalability, governance structure, and energy efficiency. The Decision Matrix in Table 3.7 provides the application-oriented decision framework that maps each of the major consensus families reviewed in this survey to representative deployment contexts. The purpose of this matrix is to summarize the types of environments in which each of the consensus mechanisms is generally best suited based on their architectural properties as well as the trade-offs discussed throughout this review [75], [143].

Thus, the purpose of the application-oriented matrix is to provide a contextualized, structured decision support perspective that connects the assumptions behind the various consensus designs with representative deployment environment requirements [2-4].

The decision matrix emphasizes the context dependence of consensus algorithm selection. For example, PoW consensus mechanisms are most appropriate for open

adversarial public blockchain environments, while BFT and PBFT consensus mechanisms have significantly lower levels of trust and are therefore more suited for consortium-blockchain environments with bounded levels of trust. PoA and PoET consensus mechanisms may be more appropriate for permissioned environments where validator identities or trust assumptions regarding their hardware could be managed, while DPoS and PoH consensus mechanisms may have greater appeal for performance-oriented blockchain applications that require higher levels of throughput. PoS and PoSpace consensus mechanisms, on the other hand, may provide alternatives for blockchain applications that desire to lower their energy consumption without fully relinquishing decentralization in the validation process. This assessment leads to the conclusion that the appropriate consensus algorithm deployed in a blockchain application will be determined by the priorities of the application rather than through a single universal benchmark for consensus algorithms [144], [145].

To summarize the analysis performed in this survey, Table 3.8 includes a structured qualitative summary of the consensus algorithms selected in this survey based on the primary analytical dimensions utilized in this survey (i.e., deployment context, fault tolerance, energy consumption, scalability, and security profile). The purpose of this table is to provide a high-level comparative analysis across the various consensus algorithms, rather than serving as a formal quantitative measure of each of the algorithm's performance. As consensus mechanisms rely on different trust assumptions, validator structures, and deployment environments, therefore, direct one to one comparison is not feasible. Consequently, the table serves as an analytical overview providing a general idea of comparative tendencies and design trade-offs as opposed to a definitive ranking of performance among the various consensus protocols.

Table (3.8): Qualitative comparative summary of the selected blockchain consensus algorithms in terms of deployment suitability, fault tolerance, energy consumption, scalability, and security profile.

No.	Consensus Algorithm	Deployment Suitability	Fault Tolerance	Energy consumption	Scalability	Security Profile
1	BFT	Permissioned	Up to one-third faulty nodes	Low	Limited	Strong in bounded-trust settings

2	PBFT	Permissioned	Up to one-third faulty nodes	Low	Limited	Strong with deterministic finality
3	PoW	Permissionless	Resilient unless majority hash power is controlled	Very High	Limited	Strong in open adversarial environments
4	PoS	Primarily permissionless	Resilient unless majority stake is highly concentrated	Low	Good	Strong, but dependent on stake distribution and governance design
5	PoSpace	Permissionless	Resilient unless storage capacity is highly concentrated	Low to Moderate	Good	Moderate to strong, depending on storage distribution
6	PoB	Permissionless	Resilient unless economic influence becomes highly concentrated	Moderate	Good	Moderate, with economic-centralization concerns
7	DPoS	Permissioned / Permissionless	Dependent on delegate honesty and voting structure	Low	Very Good	Moderate to strong, but vulnerable to collusion and governance capture
8	PoA	Permissioned	Dependent on authority honesty	Very Low	Very Good	Strong in controlled environments, but weaker in openness
9	PoET	Permissioned	Dependent on trusted hardware and bounded trust	Very Low	Very Good	Moderate to strong, subject to trusted-execution-environment assumptions
10	PoH	Primarily permissionless	Dependent on validator integrity and sequencing assumptions	Low	Very Good	Strong in high-performance settings, but architecture-dependent
11	PoC	Permissioned / Application-specific	Dependent on contribution assessment integrity	Low	Good	Moderate, with measurement and manipulation concerns
12	PoR	Permissioned / Governance-controlled	Dependent on reputation integrity and governance rules	Very Low	Good to Very Good	Moderate to strong, but sensitive to reputation manipulation

Note: The entries in this table are qualitative comparative indicators rather than universally standardized measurements. Labels such as very low, low, moderate, good, and very good reflect broad tendencies reported in the reviewed literature and the architectural characteristics of each mechanism. Because consensus algorithms differ substantially in trust model, validator composition, implementation design, and deployment context, these labels should be interpreted as analytical summaries rather than as fixed quantitative benchmarks

3.8.3. Answers to the Research Questions

According to the comparative analysis provided in this study, it is possible to respond to the research questions as follows:

RQ3.1: Which principal functional characteristics serve to differentiate blockchain consensus algorithms with respect to their validation model, finality, fault tolerance, throughput, and scalability? [76]

Response to RQ3.1

Blockchain consensus algorithms vary functionally in how they validate transactions, assign rights for producing blocks, achieve finality, and scale with the volume of participants in a network. Whereas PoW employs computational competition as a form of validation, PoS relies upon economic stakes for transaction validation, BFT/PBFT requires validation by the coordination of messages shared by validators, and DPos/PoA employ reduced participation by validators to validate transactions. Each of these functional differences will have an impact on the throughput, latency, governance structure, and the appropriateness of that consensus algorithm for public or permissioned networks [7], [8].

RQ3.2: In what ways do prominent consensus algorithms diverge in their resilience to security threats, including Byzantine faults, majority attacks, Sybil attacks, collusion, and pressures toward centralization? [75], [76]

Response to RQ3.2

Each consensus algorithm utilizes a different assumption of security, resulting in different attack surfaces. PoW has been shown to provide a high degree of protection when operating in an open, adversarial environment, but is still susceptible to the consolidation of a majority of hash power. PoS reduces energy consumption, but introduces risks that are associated with the potential consolidation of stake and relative incentives, BFT Protocols have been demonstrated to produce fault tolerance in trusted environments, while DPoS and PoA create efficiencies, but may lead to increased risk of collusion and/or overall centralization due to limited participation by validators [28], [31].

RQ3.3: How does energy consumption differ across the various consensus families, and which design decisions exert the greatest influence on the sustainability profile of blockchain systems? [73], [74]

Response to RQ3.3

The amount of energy consumed by each family of consensus algorithms is dramatically different, each type of consensus algorithm uses a different validation methodology. In the case of PoW, mining consistently occurs under competition, resulting in a high level of energy required for transaction validation. In contrast,

validation using PoS, DPoS, PoA, PoR, etc., would typically require less total energy because they do not operate under continuous competition to validate transactions [26], [146]. Consequently, the energy consumed will be material in considering the sustainability of any blockchain system.

RQ4: What trade-offs emerge among functionality, security, and energy efficiency in the selection of a consensus mechanism for a given blockchain application? [75], [76]

Response to RQ3.4

There exists a trade-off between functionality, security, and energy efficiency, in general, improving one of these dimensions will result in a restriction of another dimension. A consensus algorithm that maximizes the level of openness and resistance to an adversarial attack (PoW, for example) generally incurs increased energy costs and latency. Conversely, consensus algorithms that provide greater efficiency and power efficiency (i.e. PoS and DPoS) may rely on stronger trust assumptions to ensure security or pose a greater risk of concentration. Consequently, the selection of a consensus algorithm will need to consider the operational requirements and threat matrix associated with the application's intended use case [147], [148].

RQ3.5: Which consensus mechanisms are most appropriate for distinct deployment contexts, such as public blockchain platforms, consortium networks, enterprise environments, and resource-constrained systems? [76], [74]

Response to RQ3.5

There is no one consensus mechanism that will provide the optimal solution for every scenario in which the mechanism could be used. For example, PoW would be optimally suited for highly open and adversarial environments that use public networks, BFT and PBFT algorithms are applicable in governed systems that need to support a limited number of participants, PoS and DPoS algorithms are suited to systems that would benefit from efficiency and reduce energy usage, and PoA, PoR, and PoC would likely be better options in enterprise-data-governed systems or application-specific solutions [149], [38]. Therefore, the suitability of each consensus mechanism will ultimately depend upon the context of the mechanism.

The synthesis of the evidence demonstrates that consensus algorithm selection is primarily dependent upon the application for which it is intended. No one consensus mechanism is universally superior to the other mechanisms across all functional dimensions, and changes made to improve scalability or efficiency with respect to the amount of energy consumed will likely create a new set of trust assumptions and risks of centralization as well as security compromises associated with the consensus mechanism. Therefore, it is important to locate the major utility of performing a comparative study in the clarification of the structural conditions of the consensus mechanisms that will be more and less appropriate for a specific blockchain solution [76], [83].

3.9. Conclusion

The current study examined twelve widely used types of consensus algorithms that are used in blockchain systems. The authors provide a comparative analysis of these consensus algorithms using three distinct dimensions: Security, Functionality, and Energy Consumption. The authors have concluded that no single consensus mechanism can be considered superior to other consensus algorithms across all blockchain systems, however, different consensus algorithms provide an alternative set of trade-offs associated with each of their characteristics, for example, their validation logic, trust assumptions, governance structure, fault tolerance model, scalability attributes, and resource requirements. Especially, the authors note that the algorithm Proof of Work exhibits higher levels of robustness when it is deployed in a publicly-accessible adversarial environment, however, it does require significant amounts of energy. On the other hand, other types of consensus algorithms such as Proof of Stake, Delegated Models, Authority-based Consensus Mechanisms, and Byzantine Fault Tolerance all provide alternative trade-offs involving security, efficiency, and operational control, depending on how they are implemented and how they are used [73], [75].

The authors also found that making comparisons between the various types of consensus mechanisms is challenging from a methodology standpoint due to a lack of uniformity in terminology and terms used to evaluate consensus mechanisms, also, there are many different assumptions made regarding the properties of each consensus

mechanism, there are significant differences in the metrics that are used to evaluate various types of consensus mechanisms, and there are many different contexts in which the various types of consensus mechanisms are empirically evaluated or tested in the literature. Therefore, the analysis and comparison of the different types of consensus mechanisms in this research effort are intended to be considered as a qualitative evidence-based synthesis, rather than as a universally-quantitative comparative benchmark of different types of consensus mechanisms. This is critical to understand the differences between the advantages and disadvantages of different types of consensus mechanisms [82], [85].

Chapter 4

System Model and Problem Formulation

Chapter 4

System Model and Problem Formulation

4.1. Overview of the Proposed Setting

The problem statement, threat assumptions, performance and energy metrics, and system model of the Proof-of-Collaboration (PoCol) consensus mechanism are developed in this chapter. The initial assumption is that PoCol operates on a permissionless blockchain network derived from classical Proof-of-Work (PoW), as used in Bitcoin [1]. PoCol retains Nakamoto-style cumulative-work chain selection and a hash-based proof-of-work condition, but coordinates the generation of candidate block headers. All miners in a round use the same canonical block template and the same nonce-free header template, and the permitted nonce domain is partitioned into disjoint ranges. Under this common-template model, disjoint nonce ranges prevent honest miners from evaluating the same serialized candidate-header input, this is more precise than describing the protocol as partitioning an unspecified hash-output or search domain [1], [140].

The PoCol system comprises miners, a peer-to-peer overlay that disseminates transactions and blocks, and clients that submit transactions. In PoW, a miner repeatedly instantiates candidate block headers and evaluates their hashes until one header satisfies the proof-of-work target, the successful miner broadcasts the corresponding block and may receive the protocol reward [1], [13]. In PoCol, miners evaluate candidate headers derived from the common header template, with each miner restricted to its assigned nonce range.

The outcome of PoCol is anticipated to be a decrease in redundancy in terms of hashes and, therefore, energy consumption per block. Building on the taxonomy and energy models presented in Chapters 2 and 3 of [39], [20], this chapter creates a quantitative framework that will eventually be generated for the classical PoW alone and the PoCol method. The framework consists of: (i) an analytical network model that outlines the behavior of the nodes, (ii) an enemy model based on the Byzantine fault tolerance [7], [8] and Nakamoto consensus [1], [49], and (iii) performance and energy metrics for comparing PoW and PoCol based on prescribed conditions [20], [10].

4.2. Network Model

$M = \{m_1, m_2, \dots, m_n\}$ is the set of miners participating in the blockchain network, where $n = |M|$, that is the number of miners. In this case, each miner m_i contributes a fraction of hashing power of the network h_i such that $\sum_{i=1}^n h_i = 1$ in the baseline PoW system. Practically, h_i reflects the computational capability of miner m_i and is determined by the mining hardware of miner m_i (ASICs, GPUs, or special accelerators) and energy cost assigned to miner m_i . The power consumption of the network can be denoted by P (in W), and the process of generating a block is modeled as a Poisson process with the arrival rate of λ blocks in a second, where $\frac{1}{\lambda}$ is the expected delay time between blocks as in standard Bitcoin analysis.

The network can be modeled as a partially synchronous peer-to-peer network following the general approach in blockchain and BFT analysis. Messages, including transactions, block headers, complete blocks, and control messages, propagate using a gossip protocol. A bound Δ applies to message delay after the network's global stabilization time. Before stabilization, the network may behave asynchronously, while the stated safety properties are intended to hold. Each miner maintains a local view of the blockchain state. The blockchain is an append-only chain beginning at the genesis block, and the state-transition function applies the transactions in committed blocks to derive the current ledger state. Clients submit transactions that are propagated and stored in local mempools. In PoCol, the protocol aims to reconcile the eligible transaction set so that every honest miner derives the same canonical block template. That block template fixes the transaction list, Merkle root, previous-block hash, deterministic timestamp, target, and other consensus-critical fields. From it, every miner derives the same nonce-free header template. During a round, candidate headers are instantiated by inserting nonce values into the nonce field, no other consensus-critical field may vary. A candidate header satisfies the proof-of-work condition when $H(\text{Serialize}(\text{header})) \leq T$, where T is the target threshold. The corresponding block is valid only if it also satisfies all non-PoW consensus rules [1], [19].

In the controlled common-template PoW baseline used in this thesis, miner m_i evaluates candidate headers obtained by instantiating the same nonce-free header

template with nonce values from the permitted nonce domain N . Let Q_i denote the number of block-header hash evaluations performed by miner m_i in a round, and let $Q_{\text{total_PoW}} = \sum_{i=1}^n Q_i$. A duplicated evaluation occurs only when two miners hash the same serialized candidate header. Reuse of the same numeric nonce is not a duplicate if any other header field differs. The baseline deliberately fixes a common template to isolate coordinated versus uncoordinated selection of nonce values, it does not claim that conventional Bitcoin miners normally hash identical headers. The SHA-256 hash-output space $\{0, 1\}^{256}$ is not partitioned among miners. PoCol partitions the nonce domain N . For a fixed header template, each assigned nonce range therefore determines a disjoint set of serialized candidate-header inputs. $Q_{\text{total_PoCol}}$ denotes the number of candidate-header hash evaluations performed by PoCol miners during the round.

Time is represented in discrete rounds or epochs. During each round, miners evaluate candidate headers generated from their assigned nonce ranges. When a miner obtains a header hash satisfying the target, it broadcasts the corresponding block, and nodes apply the protocol's cumulative-work chain-selection rule [1]. After a block is accepted, the block subsidy and transaction fees are distributed according to the protocol-defined reward function described later in the thesis [15], [22].

4.3. Adversary and Threat Model

The adversary can be considered as a PPT entity capable of corrupting a fraction of miners to manipulate their behavior. Corrupted miners (called Byzantine miners) can deviate from the protocol in any manner, such as withholding blocks, publishing contradictory histories, trying to double-spend, and colluding in order to get majorities to get more rewards [7], [8]. Honest miners adhere to the specified protocol, such as nonce-domain assignment, hashing, block propagation and reporting their rewards.

We assume the adversary can corrupt up to a fraction β of the total hashing power of the network, i.e., the total amount of h_i which applies to corrupted miners is at most β (where $\beta < 0.5$ in the case of Nakamoto-style security) [1], [49], however, it will be less than $1/3$ when reasoning about BFT [7], [8]. In the analysis and evaluation, we can conclude that the adversary can coordinate corrupted miners to execute certain attacks:

- Double spending. This involves broadcasting a transaction, getting it into a block, and attempting to revert the block by creating a longer private chain which excludes the transaction (applicable to classic PoW) [1], [49].

- Selfish mining or block withholding mining. This involves withholding found blocks to create forks which change rewards or reduce the effective contribution of the honest miners [140], [22].

- Non-collaborative behavior in PoCol, which includes ignoring assigned nonce ranges, mining a modified block template that deviates from the canonical one, and copying nonce ranges from honest miners to claim unfair rewards, failing to report the number of hashes done or distributing block rewards amongst collaborators.

- Eclipse attacks or network partition attacks, which involve isolating groups of honest miners from the rest of the network to achieve their aims [49], [19].

It is also assumed that the adversary has partial control of the network channels of corrupted miners, which enables them to delay messages or drop them selectively. However, the adversary cannot overcome cryptographic primitives, such as hash functions or digital signatures, as well as forge messages from honest miners [1], [7]. The network is expected to be eventually synchronous with a bounded delay, Δ , thus ensuring liveness under the honest-majority assumption [7], [8].

The PoCol protocol must be secure from the aforementioned attacks. In particular, the protocol has to ensure that corrupt miners are prevented from (i) receiving excessive rewards due to their behavior, (ii) causing a significant increase in the likelihood of a successful double spending as compared to baseline PoW, and (iii) using the collaboration scheme to mount new types of censorship or DoS attacks.

4.4. Performance and Energy Metrics

We establish quantitative measures for PoW as well as PoCol in order to compare and assess their proposed consensus mechanism both in terms of the classical performance characteristics (throughput, latency, and finality) and energy consumption, measured as power, energy per transaction/block, and energy-latency trade-offs. The measures employed in this study are aligned with earlier empirical and

analytical studies of energy consumption in blockchains and their performance [20], [10].

4.4.1. Throughput, latency, and finality

Throughput or transaction throughput refers to the total number of valid transactions confirmed by the blockchain in a given period of time, where it is measured as transactions per second. In order to calculate $N_{tx}(t_1, t_2)$ as the total number of transactions that have been included in the canonical chain in the time interval of t_1 and t_2 , the average throughput in this time interval can be calculated as

$$\theta = \frac{N_{tx}(t_1, t_2)}{t_2 - t_1}$$

The steady-state throughput for a blockchain whose blocks are composed of on average λ Btx transactions is, thus, $\theta \approx \lambda$ Btx, with the assumption that the network is not congested and blocks are filled almost completely [1], [11],[57].

Latency reflects the time taken from the moment when the transaction is broadcasted until the moment when it is confirmed in the network. The time $t_{broadcast}$ used here is the point when the client broadcasts the transaction and $t_{confirm}$ is the time of its inclusion in the block that forms part of the canonical chain to a sufficient depth k (for instance, it is 6 confirmations in Bitcoin). Therefore, latency is defined as $L = t_{confirm} - t_{broadcast}$, which is a function of the block interval $\frac{1}{\lambda}$ and network delays Δ as well as the number of confirmations k to be reached in order to be effective against attacks conducted by adversaries with a hashing power of β .

Finality refers to the moment when the transaction is practically irreversible, meaning that the likelihood of its rollback by an adversary with a hash power of N less than β lowers below a threshold ε (e.g., 10^{-6}). In the case of probabilistic-finality systems like PoW and PoCol, it is expressed in the number of k blocks that are generated on top of the block where the transaction is contained. The studies of Bitcoin as well as analogous protocols allow the derivation of formulas for the probability of an adversary catching up with the legitimate chain, which diminishes exponentially with an increase of k in case $\beta < 0.5$ [1]. In deterministic-finality systems that include PBFT-style consensus, this moment is achieved when a block becomes accepted by

the supermajority (2/3 of validators) and does not depend on the future formation of blocks. PoCol follows the same probabilistic-finality principles as PoW, and k is chosen so as to target security guarantees at least comparable to those of the baseline technology, establishing formal equivalence of these guarantees is left to future work.

4.4.2. Energy Metrics and Trade-offs

Power (P) represents the rate at which the network consumes electrical energy, measured in watts (joules per second). It aggregates the consumption of all miners' hardware, including compute, memory, networking, and cooling overheads [13], [14]. For a given consensus protocol with block-generation rate λ , the **expected energy consumed** per block is

$$E_{\text{block}} = \frac{P}{\lambda} \quad (4.1)$$

This relation follows directly from modeling block generation as a Poisson process with a rate λ : in expectation, one block is produced every $\frac{1}{\lambda}$ seconds, during which the network consumes power P [99]. If each block contains, on average, B_{tx} transactions, the expected energy per transaction is

$$E_{\text{tx}} = \frac{P}{\lambda B_{\text{tx}}} = \frac{P}{\lambda \cdot B_{\text{tx}}} \quad (4.2)$$

The notation below distinguishes hash evaluations from the domain of values being searched [99], [19]. Under the common-template PoW baseline, $Q_{\text{total_PoW}} = \sum_{i=1}^n Q_i$ is the total number of candidate-header hash evaluations performed across all miners in an epoch. Let $U_{\text{total_PoW}}$ be the number of distinct serialized candidate-header inputs among those evaluations. The redundancy factor of PoW is then

$$R_{\text{PoW}} = \frac{Q_{\text{total,PoW}}}{U_{\text{total,PoW}}} \quad (4.3)$$

In the PoCol model, the canonical block and header templates are fixed for the round, and the permitted nonce domain is partitioned into disjoint ranges. Consequently, every candidate-header input evaluated by an honest miner is distinct from those evaluated by other honest miners in that round:

$$Q_{\text{total,PoCol}} = U_{\text{total,PoCol}}, \quad R_{\text{PoCol}} = 1 \quad (4.4)$$

Under these assumptions, and letting P_{PoW} and P_{PoCol} denote the total network power under PoW and PoCol, respectively, the expected **energy consumed to generate one block** can be written as

$$E_{\text{PoW}} = \frac{1}{\lambda} P_{\text{PoW}} R_{\text{PoW}} \quad (4.5)$$

$$E_{\text{PoCol}} = \frac{1}{\lambda} P_{\text{PoCol}} R_{\text{PoCol}} = \frac{1}{\lambda} P_{\text{PoCol}} \quad (4.6)$$

The **energy-savings ratio** comparing PoW to PoCol is therefore

$$\text{ESR} = \frac{E_{\text{PoW}}}{E_{\text{PoCol}}} = \frac{P_{\text{PoW}}}{P_{\text{PoCol}}} R_{\text{PoW}} \quad (4.7)$$

If we assume that power per miner is similar in both systems so that $P_{\text{PoW}} \approx P_{\text{PoCol}}$, then $\text{ESR} \approx R_{\text{PoW}}$. Using the approximation $Q_{\text{total_PoW}} \approx n \cdot Q_{\text{avg}}$, where Q_{avg} is the average number of candidate-header hash evaluations performed by each miner, the corresponding expression is given in Equation (4.8).

$$\text{ESR} \approx \frac{n Q_{\text{avg}}}{U_{\text{total,PoW}}} \quad (4.8)$$

As the number of miners n increases in the evaluated common-template baseline, duplicate candidate-header evaluations may increase, while PoCol maintains $R_{\text{PoCol}} = 1$ by construction [9], [19]. This property depends on two conditions: (i) all honest miners use the same immutable header template in a round, and (ii) their assigned nonce ranges are disjoint. If either condition is violated, duplicate serialized candidate-header inputs can reappear.

The energy–latency trade-off arises because block-generation rate λ affects both confirmation latency and energy consumption. For a fixed total power P , increasing λ (shorter block intervals) reduces the time to include transactions in blocks but increases the number of blocks (and thus total energy) required to process a given number of transactions if redundancy remains high [1], [49]. Conversely, decreasing λ reduces the number of blocks and, potentially, total energy but increases latency and the time to finality. In probabilistic-finality systems, the security parameter k also interacts with λ : *shorter* block intervals may require larger k to maintain the same attack success probability, further affecting latency [1], [49].

The aim of PoCol is to decouple energy consumption from competitive redundancy. By reducing R_{PoW} to 1, which is the value of R_{PoCol} , PoCol aims to decrease both E_{block} and E_{tx} for specified values of λ and B_{tx} , thereby enhancing energy efficiency while maintaining throughput and latency. The boundary of this claim is stated precisely as follows: PoCol eliminates duplicate hash inputs among honest miners within a round when all miners use the same committed block template, PoCol does not claim to eliminate every possible form of redundant computation in all PoW implementations. In later chapters, these metrics will be used to quantitatively compare PoW and PoCol using simulation and analytical models [20], [10].

For the PoCol energy-saving policy, total network energy must include both active and idle periods:

$$E_{PoCol,idle} = \sum_i (P_i^{active} t_i^{active} + P_i^{idle} t_i^{idle} + E_i^{coord})$$

For the continuous-performance policy, if all miners remain active for the complete experiment duration T :

$$E_{PoCol,continuous} \approx \sum_i P_i^{active} T + \sum_i E_i^{coord}$$

Accordingly, deterministic nonce-range separation does not independently imply lower electricity consumption. Energy reduction requires a measurable transition from active hashing to a lower-power state. Continuous operation may improve performance under a tested configuration, but under equal active power and

duration it is not expected to consume less energy than a continuously operating PoW baseline.

4.5. Goals and Needs for Design

The PoCol consensus protocol has been developed to meet the requirements and objectives defined in the literature review and issue descriptions included in the proposal [39], [20]:

- G1 – Safety and consistency: The PoCol protocol must ensure that honest miners will always see the same results in the blockchain. The probability of honest miners obeying the same canonical chains will be very high. This corresponds to the common-prefix and consistency principles in longest-chain protocols and safety in BFT protocols [1], [8].

- G2 – Liveness: Transactions created by honest users, spread among honest miners, must become part of the canonical chain and eventually be considered final (with a certain probability) within a bounded time if the situation in the network is partially synchronous and the adversary has the power less than threshold fraction β [1], [8].

- G3 – Energy efficiency: For the fixed block-generation rate of λ and transaction throughput of θ , PoCol must decrease the expected energy consumption by each block and transaction compared to standard PoW processes. Specifically, the value of energy-savings ratio ESR should exceed 1 under real network conditions and miner populations [19], [15].

- G4 – Fairness and decentralization: The distribution of rewards must be proportional to the effective contribution of the miners (e.g. assigned hash-space share, real shares submitted) in order to prevent the centralization of mining power in the hands of a small number of big players.

- G5 – Compatibility and deployability: The PoCol mechanism must be compatible with existing blockchain architectures and operative systems and must allow for the usage of available hardware.

- G6 – Protection from strategic behavior: The PoCol protocol has to be incentive-compatible for honest miners who have incentives to follow the rules of

nonce assignment and reward-reporting rather than break the rules (i.e. ignore the assignments or duplicate the nonce assignments of others).

The functional specifications define how nonce ranges are generated and assigned, how a node verifies that the successful nonce lies in the proposing miner's assigned range, how blocks are validated and disseminated, and how rewards are computed and distributed. This range-membership check does not by itself prove that non-winning miners exhaustively evaluated their assigned ranges. The non-functional specifications include throughput, latency, energy consumption, and security objectives such as a bounded double-spend probability [19], [20].

4.6. Problem Statement (Formal)

We now formalize the design problem addressed by the PoCol consensus algorithm. Consider a permissionless blockchain network with:

- A set of miners $M = \{m_1, \dots, m_n\}$, where each miner m_i contributes hashing power fraction h_i and consumes power P_i , so that total network power is $P = \sum_{i=1}^n P_i$.
- A target block-generation rate λ · (e.g., 1 block per 600 seconds) and an associated transaction throughput requirement θ · (transactions per second) determined by application needs [1], [49].
- An adversary that can corrupt a subset $C \subseteq M$ of miners with total hashing power at most β (i.e., $\sum_{m_i \in C} h_i \leq \beta$), where β is a design parameter (e.g., $\beta < 0.5$ for Nakamoto-style security [1], [49]).
- A security threshold ε specifying the maximum acceptable probability that the adversary can successfully perform a double-spend attack against a k-confirmation transaction (e.g., $\varepsilon = 10^{-6}$) [1], [49].

Let Π denote the space of consensus protocols that (i) use the same underlying hash function H and difficulty-adjustment mechanism as the baseline PoW system, (ii) implement the longest-chain rule for resolving forks, and (iii) differ only in how they assign block-header hashing work and distribute rewards among miners. A protocol $\pi \in \Pi$ is characterized by:

- A work-assignment function A_π that, in each epoch e , maps miner identities and possibly past history to disjoint (or minimally overlapping) subsets of the permitted nonce domain for honest miners.

- A verification mechanism V_π that allows miners and clients to verify that a given block was produced according to valid assignments and that miners claiming rewards contributed to the corresponding search effort.
- A reward function R_π that maps observed contributions (e.g., number of nonces searched, shares submitted, or time spent in assigned ranges) to reward shares for miners when a block is confirmed [15], [22].

For a given protocol π , define the following quantities in steady state:

- $E_{\text{block}}(\pi)$: *expected* energy consumed to produce one block, as per Equation (4.1) and the redundancy-aware model (4.5)–(4.7).
- $E_{\text{tx}}(\pi)$: *expected* energy consumed per transaction, as per Equation (4.2).
- $L_{\text{avg}}(\pi)$: *average* transaction-confirmation latency for k confirmations, given λ , Δ , β , and the protocol's fork behavior [1], [49].
- $P_{\text{ds}}(\pi, k, \beta)$: *probability* that an adversary controlling hash fraction β can execute a successful double-spend attack against a k -confirmation transaction under protocol π [1], [49].
- $F_{\text{dev}}(\pi)$: a fairness metric capturing the deviation between miners' long-run reward shares and their hashing-power fractions, for example

$$F_{\text{dev}}(\pi) = \max_i |r_i(\pi) - h_i|,$$

where $r_i(\pi)$ is the long-run fraction of total rewards earned by miner m_i under π [19], [61].

The design problem can then be expressed as a constrained optimization problem:

Minimize $E_{\text{block}}(\pi)$.

over $\pi \in \Pi$

subject to

1. $\lambda(\pi) \approx \lambda$ and $\theta(\pi) \geq \theta$ (performance constraints),
2. $P_{\text{ds}}(\pi, k, \beta) \leq \varepsilon$ (security constraint),
3. $F_{\text{dev}}(\pi) \leq \delta$ (fairness constraint),
4. $ESR(\pi) = \frac{E_{\text{block}}(\text{PoW})}{E_{\text{block}}(\pi)} \geq S_{\text{min}}$ (energy-savings constraint),
5. π is implementable without specialized trusted hardware and is compatible with standard blockchain data structures (deployability constraint).

Here, δ is a small constant bounding the acceptable deviation from proportional fairness, and $S_{\text{min}} > 1$ is the minimum energy-savings factor required to justify deployment (e.g., S_{min} in the range 1.5–2 based on energy analyses in [19], [15]).

$E_{\text{block}}(\text{PoW})$ is the expected energy per block of the baseline protocol measured under the same λ and hardware assumptions using the models in Section 4.4.2 [20], [10].

Intuitively, the problem is to design a consensus protocol that (i) preserves the security and performance guarantees of the existing PoW-based blockchain, (ii) ensures fair and decentralized reward distribution, and (iii) significantly reduces redundant hashing and hence energy consumption. The PoCol protocol proposed in this thesis is one concrete candidate solution $\pi_{\text{PoCol}} \in \Pi$, whose design, detailed operation, and evaluation are developed in the following chapters [39], [20].

4.7. Summary

In this chapter, the problem formulation and the system model for the proposed PoCol consensus mechanism are presented. Through the network model, the authors define an open system of blockchain comprised of a group of miners, a communication network that allows free peer-to-peer interaction, a chain of blocks that varies, and separation of the baseline and real PoW protocol and its modification in PoCol in terms of control of hash-population and rewards given to people working or mining [1], [39]. The adversary and threat model defines the powers of Byzantine miners and the power to execute some currency manipulations involving a certain percentage of hash capacity including illegal money transfer, selfish mining, or breaking the rules of collaboration in PoCol [7], [61].

The efficiency indicators (throughput, latency, and finality) and energy indicators (electricity consumption, energy used per block and transaction, and energy-latency relationship) were determined in such a way that they make it possible to compare PoW and PoCol under one idea, based on the previous experience of analysis and experiment in this field [20], [10]. The chapter then showed what are the requirements of the PoCol protocol concerning safety, liveness, energy effectiveness, fairness, adaptability, and reliability and summarized them into a formal optimization problem of how to reach lower energy consumption levels while maintaining the safety of the protocol. Future chapter will describe the detailed design of PoCol protocol, including partitioning of nonce-domain, collaboration mechanisms, and reward assignment methods based on the system and problem statement as defined earlier in

this research. Then showed what are the requirements of the PoCol protocol concerning safety,

Chapter 5

The Proof-of- Collaboration Consensus Algorithm

Chapter 5

The Proof-of-Collaboration Consensus Algorithm

5.1. Introduction and Design Goals

At the core of a blockchain system is a consensus mechanism that enables mutually distrustful nodes to agree on the ordering and validity of state transitions. Nakamoto-style Proof-of-Work (PoW) provides strong security properties but can require substantial energy because miners continuously evaluate large numbers of candidate block headers in parallel. The relevant input domain is not merely the 32-bit header nonce field: in Bitcoin, miners can also obtain new candidate headers by changing the coinbase extra nonce, which changes the Merkle root, and by updating other permitted header fields. This energy requirement has motivated alternative mechanisms such as Proof-of-Stake (PoS), Delegated PoS (DPoS), Proof-of-Authority (PoA), and blockchain adaptations of Byzantine fault-tolerant protocols [48], [2].

In Bitcoin-style PoW, miners repeatedly hash serialized candidate block headers. The 32-bit header nonce is one mutable field, but exhausting its values does not exhaust the broader candidate-header input domain: a miner can change the coinbase extra nonce, thereby changing the Merkle root, and may update other permitted template fields before scanning the nonce domain again. Therefore, two miners using the same numeric nonce do not necessarily evaluate the same hash input. Exact duplication occurs only when the complete serialized headers are identical. The probability that a miner finds the next block depends primarily on its share of the total hash rate, and network energy consumption scales with aggregate mining activity and hardware efficiency [1], [24]. Empirical studies report substantial energy use by major PoW networks [9], [99]. PoS- and BFT-style protocols reduce continuous block-header hashing but introduce different assumptions and mechanisms, including stake management, slashing, committee selection, and partial synchrony [7], [8].

This thesis describes Proof of Collaboration (PoCol), which seeks to retain the hash-based proof-of-work structure while coordinating candidate-header evaluation. In each round, PoCol defines one canonical block template and one nonce-free header

template. The permitted nonce domain is partitioned into non-overlapping ranges that are assigned deterministically and pseudo-randomly to registered miners. Each miner evaluates only candidate headers formed by inserting nonce values from its assigned range. A nonce value is successful only relative to a specific header template and target, namely when the resulting serialized header hashes to a value at or below the target. Under the common-template invariant, disjoint nonce ranges ensure that honest miners do not evaluate the same serialized candidate-header input during the round.

The main design goals of PoCol are: first, to reduce duplicate candidate-header evaluations relative to the evaluated common-template PoW baseline, second, to preserve decentralized derivation of work assignments by allowing every miner to compute its nonce range locally from public protocol state, third, to support a collaboration-aware distribution of the block subsidy and transaction fees, and fourth, to limit the changes required to PoW-style block and node structures. These are design objectives rather than claims of equivalence with Bitcoin's existing mining workflow.

In this chapter, a comprehensive formal definition of PoCol, with respect to system and threat model, is provided as well as detailed information on how miners register with PoCol, how the distribution of nonce domain occurs deterministically, how collaborative work is accomplished through PoCol, how rewards are distributed among miners, and how the miner set is updated dynamically. An informal analysis of the security and reliability of PoCol is provided, which shows how PoCol can be compared with existing consensus algorithms, along with some of the problems associated with inactive miners and the effects of churn.

5.2. System and Threat Model

5.2.1. Network and Node Assumptions

We model a blockchain network as comprising miners who keep a copy of the replicated blocks stored in the entire ledger. Out of all the available nodes, miners are the group that plays a pivotal role in the consensus by proposing and validating blocks. The communication within the network occurs within the context of a framework that is partially synchronous such that the messages are delivered within a finite amount of time but the time taken can vary.

Each of the nodes has its own ledger containing blocks from the single blockchain maintained by all the other miners. The nodes interact using messages that are sent and received in a peer-to-peer manner. The messages include pending transactions which are distributed through the gossip mechanism.

5.2.2. Miner Identities and Cryptographic Primitives

Long-term public key is the only means of identification for each of the miners which the miner uses both for registration and also deregistration plus to indicate the address to which block rewards would be sent. The registration transaction helps to identify a miner's public key with a unique miner ID and a network address. Each signature uses a proper digital signature method (for example, ECDSA and EdDSA) based on elliptical curves as stated in blockchain systems. InPoCol blockchain technology, the hash function is constructed such that it behaves like a random oracle, and it is usually constructed using SHA-256 or SHA-3, as seen in other blockchain technologies.

PoCol relies on the preimage resistance and output unpredictability of the hash function. For a fixed header template and target, miners are not assumed to have a shortcut that derives a nonce value whose instantiated candidate header hashes below the target, they must evaluate candidate headers according to the proof-of-work procedure. Collision resistance is relevant to preventing distinct inputs from being deliberately constructed with the same digest, but the mining condition itself is a target-preimage condition rather than a collision-finding problem.

5.2.3. Adversary Capabilities

Adversaries are assumed to be in charge of taking over a few miners, and they take part in developing Byzantine faults which give them full control over the miners to conduct any activities of their choice. The adversary can also combat the miners through network attacks. However, adversaries do not conduct unusual work on normal cryptographic processes (signature creation) through forgery or reverse engineering the hash function.

The total hashing rate which is assumed to be within the limit of fraction α of the total network hash rate, and this rule has been used in previous work of Nakamoto

consensus protocol [46], [1]. For PoCol to be effective, there is no need to conduct questions of stake or long-range attack which are included in PoS processes.

5.2.4. Security Properties

We informally target the following standard security properties for PoCol:

- **Safety (Consistency):** With overwhelming probability, two honest nodes do not permanently disagree on the prefix of the blockchain, aside from a bounded number of most recent blocks that may be reverted due to temporary forks.
- **Liveness (Termination):** Transactions broadcast by honest clients are eventually included in the blockchain and become stable, provided the network is not permanently partitioned and the adversary's hash power is bounded.
- **Fairness:** Over sufficiently long time horizons, reward shares are intended to follow the contribution measure defined by the protocol, such as hash-power estimates or verifiable participation in assigned nonce-range evaluation, and no miner should be able to bias nonce-range assignments systematically in its favour.
- **Energy Efficiency:** For a given difficulty level and target block interval, PoCol is intended to reduce duplicate serialized candidate-header evaluations relative to the evaluated common-template PoW baseline, thereby improving the reported energy-per-confirmed-transaction metrics.

The remainder of the chapter describes the PoCol protocol and argues informally that these properties hold under the specified system and threat model.

5.3. PoCol State Representation on the Blockchain

5.3.1. Blockchain Ledger and Global State

The blockchain ledger maintained in PoCol consists of a series of blocks with B0 being the genesis block created during the genesis round of PoCol. Each block consists of a header and a set of transactions. The global state of $S(B_h)$ for height h contains (a) balances of accounts along with any special data related to the application, (b) details of all the miners in M_h , and (c) other details needed by the reward distribution function.

The execution of transactions in block B_h follows state $S(B_{h-1})$. The standard transactions may alter the balances, deploy or invoke smart contracts, or make changes to application-specific states. Additionally, PoCol introduces miner registration and

deregistration transactions in order to alter the set of miners through the processes discussed in Section 5.4.

5.3.2. Registered Miner Set

At each height h , M_h is a list of public keys of the miners who are eligible to participate in PoCol mining rounds in order to earn rewards. The list M_h at block h is derived from the historical record of miner registration and deregistration transactions recorded in blocks until the block height of h . In the simplest version, registration is permanent unless canceled, i.e., if a miner's registration transaction is confirmed, its public key will remain on the list of M_h until a deregistration transaction is executed.

5.3.3. Balance and Reward State

Balances are updated through standard transfer transactions and also by reward distribution transactions present in every block. Unlike PoW where one coinbase transaction pays the entire block reward, in PoCol, reward distribution transactions can have one output for each miner where the block reward amount can be distributed based on some reward rules. Therefore, PoCol ledger is the only official source for both user balances and miner earnings.

5.4. Miner Life-Cycle: Registration, Activity, and Leaving

5.4.1. Miner Registration Protocol

The miner's lifecycle in PoCol starts with their registration at genesis. During that phase, the first miner (Miner A), registers locally on the network and possibly also adds a genesis registration in B0 to the register, depending on the deployment. Following that, the next miners join the network by making a miner registration transaction by providing their public key, network address, reward address, and possibly some optional information (e.g., declared hash power or jurisdiction). Afterward, this transaction is broadcast across the network in the same manner as other valid transactions and is included in a block by any miner. When the registration transaction is confirmed in block B_h , then the miner set of honest nodes is updated so that M_h now contains the new miner. From then on for the next PoCol round, starting from $h+1$, the new miner is now eligible for nonce-range assignments and reward sharing. This design replicates how validator sets are updated in PoS and BFT-like

blockchains where membership is only changed after having staked and governance transactions included [27], [8].

5.4.2. Explicit Miner Deregistration

Miners who do not want to participate in PoCol anymore (such as when they are decommissioning their equipment or transitioning to another network) are able to carry out a deregistration transaction. This transaction is signed by the miner with their public key and shows that the miner wants to be removed from their registered set. Nodes first check the validity of the signature and ensure that the miner currently belongs to the set M_h before recording the transaction in their mempools. After the deregistration transaction has been executed in a block, the miner is removed from M_h in the current state. Depending on the settings in the protocol, deregistration can take effect immediately with the next round of miners contributing to M_h or it can take place after k blocks to avoid abuse. After deregistration, the miner will no longer receive any nonce range or share of rewards. However, previously acquired rewards will still be recorded in the ledger.

5.4.3. Handling Silent Leaves and Inactive Miners

Due to the nature of open networks, it is not realistic to expect all miners to deregister as soon as they become inactive. It is necessary for PoCol to cover silent leave situations. In such cases, a miner can simply cease hashing or go offline without making a deregistration transaction. In this situation, an identifier of that miner remains in M_h , meaning that it no longer contributes towards consensus.

From the point of view of the nonce distribution function used in PoCol, inactive miners will continue to receive nonce ranges. However, these ranges will not be used at all. This situation is similar to the drop in total hashing power of classical PoW, during which certain miners stop using their hardware causing the effective hashing power of the network to decline. The action of PoCol will remain correct, but blocks in this case will be produced at a lower frequency until the correct difficulty setting is achieved.

The presence of inactive miners in M_h raises fairness concerns about the distribution of rewards. A simple policy for awarding miners who have registered is to

provide them with rewards regardless of whether they participated in the recent round. More elaborate rules can include activity measurements and “proofs of hashing” to deny rewards to consistently inactive miners.

5.4.4. Optional Inactivity-Based Removal

In order to avoid having a number of inactive miners recorded in M_h , the protocol could be augmented with an inactivity-based removal feature. In this feature, for each miner, the protocol monitors the index of the last block in which that miner was actively involved (as a winner, co-signer of a heartbeat transaction, etc). If the miner remains inactive for more than T blocks, then any node in the network can suggest an inactivity removal transaction. As soon as this transaction has been included in the block, the miner is no longer considered to be registered.

This action resembles penalties for inactivity in PoS/BFT systems, however, PoCol allows implementing it without requiring the user to pay a penalty in stake. T can be adjusted so that either stability (no continuous removals of inactive miners from M_h) or responsiveness (quick removal of longtime inactive miners).

5.5. Overview of the PoCol Mining Round

5.5.1. Round Structure and Inputs

PoCol operates by having rounds, in which any round is meant to yield a single valid block of a specific height. Rounds are indexed to the blocks by having round r producing block B_r such that its predecessor is hash of the valid block of height $B_r - 1$. Any round follows the same process and as such the miners will access consistent information on all the possible blocks produced from 0 to $r - 1$ and derive the required past state of the previous block’s hash ($prev_{hash}$), set of active miners M_{r-1} as well as the current pending transactions (mempool).

Since all miners reference the same committed information from past states of the blockchain, PoCol requires them to derive the block template for B_r in an identical, deterministic manner. All miners participating in the same PoCol mining round must mine exactly the same block: they use a single canonical, immutable block template (Section 5.8.2), verify that they share the same template identifier $TemplateID_r$, and may vary only the nonce during the active round. Identical templates are therefore not

merely an expected behaviour but a consensus rule, and the nonce distribution function of Section 5.6 is defined with respect to this common template.

5.5.2. High-Level Steps

At a high level, a PoCol round comprises four phases:

- 1) Canonical block template construction: all miners deterministically derive the same immutable block template for the round — containing the previous-block hash, the round number, the protocol version, the difficulty target (set by the protocol from the desired block interval and recent block production rate), a deterministic round timestamp, the ordered transaction list, its Merkle root, and the reward-distribution transaction — and verify that they share the same template identifier $TemplateID_r$ (Section 5.8.2).
- 2) Nonce-domain partitioning: miners apply the PoCol allocation function to compute a pseudo-random ordering of miners and assign each miner a non-overlapping nonce range within the permitted nonce domain. The assignment is bound to the common block and header templates through $TemplateID_r$ (Section 5.8.3).
- 3) Collaborative proof-of-work evaluation: each miner iterates only over its assigned nonce range. For each nonce n , the miner instantiates the common nonce-free header template to obtain a complete candidate header and evaluates its hash. If the header hash is at or below the target, the miner publishes the already committed block template together with the successful nonce value, the reward-distribution transaction is already included in that template.
- 4) Verification and commitment: upon receiving a proposed block, nodes verify its proof of work, confirm that its non-nonce consensus-critical fields match the committed canonical template, validate its transactions, verify that the successful nonce lies within the proposer's assigned range, and verify the predetermined reward distribution. If the block is valid and selected by the chain-selection rule, it is appended to the ledger and the round ends. If every assigned nonce range is exhausted without producing a header hash at or below the target, the round ends without a block, a new round and a new canonical template are then derived before mining resumes (Section 5.8.4).

The following sections formalize the nonce-domain distribution and the behaviours in genesis and normal rounds.

5.6. Deterministic Nonce-Domain Distribution

In PoCol, the goal of nonce-domain distribution is to ensure that, in each round r , the permitted nonce domain is partitioned into disjoint sub-ranges and each

registered miner receives exactly one sub-range, without any trusted coordinator. All honest miners must be able to recompute the assignment locally and obtain identical results from public inputs only.

Let B_{r-1} be the last committed block at the start of round r , with header hash

$$h_{r-1} = H(\text{header}(B_{r-1})) \quad (5.1)$$

Let $M_r = \{m_0, m_1, \dots, m_{n-1}\}$ be the registered miner set extracted from the ledger state S_{r-1} , where each m_i is a unique miner identifier (e.g., a public key). Let $N = \{0, 1, \dots, N_{\max} - 1\}$ be the permitted nonce domain, with fixed size $N_{\max}$ (e.g., 2^{32} or 2^{64} depending on implementation constraints).

5.6.1. Round Seed

For each round $r \geq 0$, every miner computes the round seed as

$$\text{seed}_r = H(\text{"PoColNonce"} \parallel h_{r-1} \parallel r \parallel \text{TemplateID}_r) \quad (5.2)$$

where $\parallel$ denotes concatenation and the ASCII string "PoColNonce" is used as a domain separator. Here $\text{TemplateID}_r = H(\text{BlockTemplate}_r)$ is the identifier of the canonical block template of round r (Section 5.8.2), so that the nonce allocation is bound to the common template. Because h_{r-1} , r , and TemplateID_r are fixed once B_{r-1} is committed and the canonical template is derived, seed_r is deterministic for all honest miners and unpredictable before B_{r-1} is finalized, assuming H behaves as a random oracle [49], [11].

5.6.2. Pseudo-Random Ordering of Miners

Given the miner set M_r and seed seed_r , PoCol defines a per-round ticket for each miner identifier $m_i \in M_r$ as

$$\text{ticket}_r(m_i) = H(\text{seed}_r \parallel \text{id}(m_i)) \quad (5.3)$$

where $\text{id}(m_i)$ is a canonical byte encoding of the miner's identifier (e.g., its public key). All miners then sort M_r by increasing ticket value and obtain a round-specific permutation

$$L_r = [m_{(0)}, m_{(1)}, \dots, m_{(n-1)}] \quad (5.4)$$

such that

$$\text{ticket}_r(m_{(0)}) \leq \text{ticket}_r(m_{(1)}) \leq \dots \leq \text{ticket}_r(m_{(n-1)})$$

The list L_r is a round-specific ordering of M_r . Under the random-oracle model, this permutation is statistically close to uniform and cannot be biased by any miner unless it can control h_{r-1} in a way that already violates the underlying PoW security model. Similar validator shuffling and VRF-based leader selection mechanisms have been used in PoS protocols such as Ouroboros, Algorand, and Snow White [27], [29].

5.6.3. Partitioning the Permitted Nonce Domain

Let $n = |M_r|$. PoCol divides the permitted nonce domain N into n contiguous sub-ranges using simple integer partitioning:

$$\text{start}_r(i) = \left\lfloor \frac{i N_{\max}}{n} \right\rfloor \quad (5.4a)$$

$$\text{end}_r(i) = \left\lfloor \frac{(i+1) N_{\max}}{n} \right\rfloor - 1 \quad (5.4b)$$

for each position $i \in \{0, \dots, n-1\}$. The nonce sub-range for miner $m_{(i)}$ in round r is then

$$R_r(m_i) = [\text{start}_r(i), \text{end}_r(i)] \quad (5.5)$$

By construction:

- $R_r(m_{(i)}) \cap R_r(m_{(j)}) = \emptyset$ for all $i \neq j$,
- $\bigcup_{i=0}^{n-1} R_r(m_{(i)}) = N$,
- $|R_r(m_{(i)})| \in \left\{ \text{floor}\left(\frac{N_{\max}}{n}\right), \text{ceil}\left(\frac{N_{\max}}{n}\right) \right\}$.

Thus, each miner receives a disjoint portion of the nonce domain, and the assignment is uniquely determined by $(M_r, h_{r-1}, r, \text{TemplateID}_r)$, remains

unchanged for the duration of the round, and is independently verifiable by every node. No coordination or message exchange is required for nonce-range assignment beyond the blockchain state and standard block propagation.

5.6.4. Fairness Across Rounds

For a fixed nonce-free header template and target, the position of a miner's assigned range within the nonce domain does not make individual nonce values intrinsically favourable. A miner's probability of producing a successful candidate header depends on the number of candidate headers it can evaluate within its assigned range and on the per-header success probability. The pseudo-random ordering L_r changes the locations of assigned ranges across rounds. Under the common-template invariant, disjoint assigned ranges also ensure that honest miners do not evaluate the same serialized candidate header in the same round.

5.6.5. Handling Unevaluated Nonce Ranges

When a registered miner is inactive, its assigned nonce range $R_r(m_i)$ is not evaluated. The set of nonce values actually examined in the round is therefore the union of the ranges assigned to active miners, and the effective aggregate hash rate of the active miner set is reduced.

In the worst case, all nonce values that would instantiate headers satisfying the target for the current template may lie inside ranges assigned to inactive miners. Active miners can then exhaust their assigned ranges without finding a successful candidate header. In that event, the permitted nonce domain for the current template has been exhausted by the active miners, and PoCol starts a new round with a newly derived common template, $TemplateID_r$, and nonce-range assignment as specified in Section 5.8.4 [1], [46].

Figure 5.1 illustrates a PoCol round in which Miner C is registered but inactive. All miners derive the same canonical block and header templates, verify the common $TemplateID_r$, and compute the disjoint nonce ranges locally. Miners A and B evaluate candidate headers using nonce values from their assigned ranges, while Miner C's assigned range is not evaluated. The result is a temporary reduction in effective aggregate hash rate. Under the baseline reward policy, Miner C may nevertheless

remain eligible for a reward share while it remains registered, this reward-policy issue is distinct from verification of the successful nonce's range membership.



Figure (5.1): PoCol consensus round with an unevaluated nonce range caused by an inactive miner. All miners use the same immutable canonical block and nonce-free header templates, only the nonce value varies when candidate headers are instantiated.

5.6.6. Post-Range Operating Policies

PoCol distinguishes nonce-range allocation from the miner's operating state after completing the assigned search. The protocol may apply either an energy-saving

idle policy or a continuous-performance policy. These policies are alternatives and must be identified explicitly for every experimental or deployment configuration.

5.6.6.1. Energy-Saving Idle Policy

Under the energy-saving policy, a miner searches only the nonce range assigned to it for the current immutable template. If no valid block is found before the range is exhausted, the miner stops active hashing and enters a low-power idle state until the end of the current collaborative mining round. The miner remains connected and may receive protocol messages, including notification that another miner has produced a valid block, but it performs no further mining hashes during the remaining round time.

The phrase “end of the current collaborative mining round” refers to the round deadline or target block interval defined by PoCol. It does not refer to a multi-block difficulty-adjustment epoch.

The energy benefit of this policy depends on the difference between active and idle power and on the duration of the idle interval. The policy may also reduce the active network hash rate as miners complete their ranges, therefore, its effects on block interval, range-exhaustion probability, liveness, and security must be evaluated together with its energy savings.

5.6.6.2. Continuous Performance Policy

Under the continuous-performance policy, a miner that completes its assigned range does not enter the low-power idle state. Instead, it may continue processing newly assigned, non-overlapping work defined by the protocol. The new assignment must be bound to an explicitly identified template or sub-round so that honest miners do not receive overlapping assignments.

When PoCol and the PoW baseline use the same aggregate hash rate, hardware efficiency, active power, and experiment duration, continuous operation is not expected to provide an inherent energy reduction. Its purpose is performance-oriented: it may improve accepted-block production or transaction throughput under the evaluated configuration. Such improvement must be reported as an empirical result rather than as a guaranteed theoretical consequence of nonce-range separation.

5.7. PoCol Genesis Round

5.7.1. Genesis Preconditions and First Miner Setup

The purpose of the PoCol genesis stage is to determine the first block B_0 and organize the first miner development set which has been registered in the ledger and has its balances displayed. In the simplest implementation approach, a bootstrapping authority or consortium has determined the primary development information. The bootstrapping authority selects the certain number of founding miners and provides them with the first tokens.

As for methods of bootstrapping, the PoCol refers to a more decentralized approach when the first miner self-registers independently from others only in order to have its transaction registered initially.

Regardless of what kind of method of bootstrapping is used, the genesis state $S(B_0)$ has been utilized, representing M_0 which is the first registered miner set. For the time being, M_0 encompasses several miners who agreed to work together during the PoCol genesis stage. There has been one additional transaction in the genesis block which records the allocation of first balances to the registered miners or application-specific accounts.

5.7.2. Genesis Miner Registration Sequence

In a typical situation, the process of registration in the genesis sequence goes as follows. Miner A, who desires to carry out local registration record, proceeds with executing genesis registration transaction possibly by disclosing its public key, network address, and reward address. However, after registration by Miner A, the two other miners – B and C – send registration requests to Miner A, which are validated by Miner A. As soon as Miner A has approved the requests, it registers them into the system.

No matter however many details exist concerning the manner of displaying genesis registration messages, there are two main ways to do this. The addressed message could be published as an explicit transaction or through configuration files. Nevertheless, it is essential that all the miners have to agree upon M_0 prior to start nonce domain distribution.

5.7.3. Genesis Round Nonce Distribution and Collaborative Search

In the genesis round ($r = 0$), miners read the genesis hash as $prev_{\text{hash}}$, deterministically derive the canonical genesis block template and its nonce-free header template, compute $TemplateID_0$ over the canonical template representation with the nonce value excluded or fixed to its canonical placeholder, and compute $seed_0 = H(\text{"PoColNonce"} \parallel genesis_{\text{hash}} \parallel 0 \parallel TemplateID_0)$. They then derive the ordered miner list L_0 and assign nonce ranges R_i as in Section 5.6.

The canonical genesis block template is identical for every miner and includes the initial reward-distribution transaction and any bootstrap transactions. Every miner verifies $TemplateID_0$ before beginning proof-of-work evaluation. Each miner then instantiates candidate headers using nonce values from its assigned range. When a miner obtains a header hash at or below the target, it publishes B0 by combining the committed template with the successful nonce value. Other miners verify the complete header, proof of work, transactions, and template commitment before accepting B0 and S(B0).

5.7.4. Genesis Reward Distribution and Ledger Initialization

The reward distribution in the genesis block may follow a different policy than in normal rounds—for example, allocating initial balances to founding entities or to a treasury account. Nonetheless, the structure of the reward distribution transaction is the same: it is a standard transaction that creates outputs paying to specified addresses. After B0 is committed, the ledger contains S(B0) with initialized balances and M_0 , and PoCol proceeds with normal rounds for subsequent blocks.

The genesis round of PoCol can be seen in Figure 5.2, and the definition includes processes in Sections 5.7.1, 5.7.2, 5.7.3, and 5.7.4. Each of these explanations contains a description of how the bootstrap registration process occurs. The bootstrap registration process consists of two phases. The first phase is called the bootstrap registration process, and this phase starts with the following events: the miner, does its own registration, where it keeps its own record. After it is done with the registration, it allows for miner #2 and miner #3 to enter the miners that are registered, which means that it confirms what is going on and updates the information. After the client has submitted the transaction (which has been validated and relayed to the miner), each of

the miners is able to view the genesis hash that is made for the genesis round. It then derives the common round 0 block template and verifies $TemplateID_0$. From there, it calculates its round seed by using the predetermined combination, and after that makes a ticket and nonce range by using the nonce distribution function. The process continues with the collaborative endeavor and shows what will happen when miner #2 creates the final winning nonce and makes the genesis block to confirm that the payment has been made.



Figure (5.2): PoCol consensus genesis round (round 0), including miner registration, transaction submission, collaborative nonce search, and commitment of the genesis block. All miners derive the same canonical genesis template ($TemplateID_0$) before the collaborative search.

5.8. PoCol Normal Rounds ($r \geq 1$)

5.8.1. Reading State at the Start of Round r

At the beginning of a normal round $r \geq 1$, each miner reads the current state $S(B_{r-1})$ from its local blockchain view. This state provides: (i) the previous block hash $prev_{hash}$, (ii) the registered miner set M_r (derived from registration and deregistration transactions up to height $r-1$), and (iii) current account balances and application-specific data. Miners also maintain a mempool of pending transactions received since the last block.

5.8.2. Canonical Block Template Construction

In PoCol, round r does not involve independently constructed candidate blocks. Before nonce evaluation begins, all participating miners deterministically derive, or receive and verify, the same canonical block template $BlockTemplate_r$ from the committed state $S(B_{r-1})$ and the eligible transaction set. The block template fixes the transaction list, reward-distribution transaction, Merkle root, previous-block hash, round number, protocol version, target, deterministic timestamp, and every other consensus-critical field except the round's nonce value. From this block template, miners derive a nonce-free header template $HeaderTemplate_r$. Candidate block headers are produced only by inserting nonce values into the nonce field of $HeaderTemplate_r$.

The ordered transaction list is produced deterministically by every miner from a common eligible transaction set, which is established as follows. Round r starts deterministically at the moment block B_{r-1} is accepted. The round then opens a bounded transaction-collection (synchronization) window of protocol-defined length Δ_{tx} , during which nodes exchange pending transactions through the normal gossip mechanism, under the propagation assumption of Section 4.2, honest mempools converge during this window. A transaction is eligible for round r only if it was received before the close of the window and is valid with respect to $S(B_{r-1})$, transactions arriving after the cutoff remain in the mempool and become eligible in later rounds.

From the eligible set, the canonical transaction set is selected and ordered by a protocol-fixed deterministic rule. As the reference policy, this thesis proposes selection in decreasing order of fee density up to the protocol block-size limit, with ties broken by lexicographic ordering of transaction identifiers. This specific rule is a proposed protocol requirement, not a feature of the original PoCol prototype: the BlockSim prototype of Chapter 6 sorts pending transactions by decreasing fee when a block is created, but it defines no tie-breaking rule, performs the selection only after the winning event rather than before mining, and includes no cross-miner agreement step. Because the selection rule, its inputs, and the tie-breaking order are fixed by the protocol, any two miners holding the same eligible set obtain the same ordered list and hence the same Merkle root, any other deterministic rule (for example, a committed transaction list announced together with block B_{r-1}) is equally admissible, provided it is adopted as a consensus rule.

Residual mempool differences are reconciled before mining rather than tolerated, because deterministic sorting alone does not resolve divergent mempool views: determinism guarantees identical outputs only for identical inputs. A miner that observes, in a peer's announcements, the identifier of an eligible transaction it does not possess requests the missing transaction by its identifier and must obtain and validate it before constructing the template. $TemplateID_r$ serves as the agreement check: a miner whose locally derived $TemplateID_r$ differs from the $TemplateID_r$ announced by its peers does not start mining, it resynchronizes its transaction view (fetching missing transactions or discarding ineligible ones) and re-derives the template until the identifiers agree. Mining in round r may begin only after the miner has verified that its $TemplateID_r$ equals the network-consistent value. This agreement is reached entirely through peer-to-peer gossip of transactions and template identifiers, without any centralized coordinator.

In summary, the common canonical transaction set is the product of four combined mechanisms: (i) the bounded collection window with a deterministic round start, (ii) the eligibility cutoff against $S(B_{r-1})$, (iii) identifier-based retrieval of missing transactions, and (iv) the $TemplateID_r$ agreement check that gates the start of mining. In the discrete-event prototype of Chapter 6, the transaction-collection window, mempool reconciliation, and template agreement are not implemented as

executable mechanisms, they form part of the corrected protocol specification and are listed in Chapter 6 and Section 8.5 as required implementation extensions.

Each round is associated with a block-template identifier

$$TemplateID_r = H(\text{Encode}(BlockTemplate_r \text{ without a round-specific nonce value}))$$

Every miner recomputes $TemplateID_r$ locally and verifies that it refers to the same canonical block and nonce-free header templates before beginning nonce evaluation. During an active round, those templates are immutable, miners instantiate candidate headers only by varying the nonce value.

Common-Template Invariant: For every PoCol round r , all honest miners use the same immutable block template T_r and the same derived nonce-free header template. A candidate block is valid for round r only if all non-nonce consensus-critical fields match T_r , the nonce value is inserted solely to instantiate the candidate header evaluated for proof of work.

5.8.3. Nonce-Domain Distribution in Round r

With the canonical block and nonce-free header templates fixed and $TemplateID_r$ verified, miners compute the round seed $seed_r = H(\text{"PoColNonce"} \parallel prev_{\text{hash}} \parallel r \parallel TemplateID_r)$, derive the ordered list L_r , and compute the nonce ranges R_i as described in Section 5.6. Including $TemplateID_r$ binds each nonce-range assignment to the canonical template. Each miner computes its own assigned range locally. The ranges are pairwise disjoint, collectively cover the permitted nonce domain, are independently verifiable from public inputs, and remain unchanged for the duration of the round.

This partitioning prevents duplicate candidate-header inputs because every honest miner uses the same nonce-free header template while different miners receive disjoint nonce ranges. For a fixed $HeaderTemplate_r$, a proof-of-work input is the serialized complete header $Header_r(n)$ obtained by inserting nonce n . Therefore, distinct nonce values produce distinct serialized header inputs, and no two honest miners evaluate the same $Header_r(n)$ during the round.

Disjoint-Evaluation Invariant: For every pair of distinct honest miners m_i and m_j in round r , $R_r(m_i) \cap R_r(m_j) = \emptyset$. Hence they never evaluate the same serialized candidate header $\text{Header}_r(n)$.

5.8.4. Collaborative Proof-of-Work Search

Miners perform proof-of-work evaluation by iterating over nonce values $n \in R_i$. For each n , a miner constructs the complete candidate header $\text{Header}_r(n)$ by inserting n into the nonce field of the common nonce-free *HeaderTemplate_r*, then evaluates $H(\text{Serialize}(\text{Header}_r(n)))$. The transaction list, Merkle root, previous-block hash, target, timestamp, protocol version, reward-distribution transaction, and all other consensus-critical fields remain fixed by *BlockTemplate_r*. Altering any such field produces a different template and is prohibited during the current round. If $H(\text{Serialize}(\text{Header}_r(n))) \leq \text{target}$, the miner has produced a proof-of-work-valid header for B_r and broadcasts the committed block template instantiated with the successful nonce value.

If a miner exhausts its assigned range R_i without producing a header hash at or below the target, it marks that range as exhausted locally. If all assigned ranges are exhausted and no valid block has been received, the round ends without a block. No miner may alter a non-nonce field and continue under the same round identifier. Instead, the protocol creates a new round, derives a new canonical block and nonce-free header template, recomputes *TemplateID*, generates new nonce ranges, and resumes only after agreement on the new template. Thus, only the nonce value varies within one round, changes to timestamp, transactions, or other header fields require a new template and a new round.

5.8.5. Block Proposal, Verification, and Commitment

Upon receiving a proposed block B_r from some miner, each node verifies it as follows:

- 1) It checks that B_r extends the current longest valid chain (or a competing chain of comparable length) by verifying that $\text{prev}_{\text{hash}}$ matches the hash of B_{r-1} in its local ledger.

- 2) It recomputes the canonical block template, its nonce-free header template, and $TemplateID_r$, then checks that every non-nonce consensus-critical field of the received block and header matches the committed template.
- 3) It checks that the transaction list and the Merkle root of the received block match the common template, and that the reward-distribution transaction is identical to the predetermined $T_{reward}(r)$ committed in the template (Section 5.9), so that the block contains no unauthorized modification of the template.
- 4) It recomputes $seed_r$, L_r , and the nonce ranges R_i , and verifies that the successful nonce value lies within the proposing miner's assigned range. This verifies range membership of the successful candidate header, it does not prove exhaustive evaluation of other miners' ranges.
- 5) It validates all transactions in Br against $S(B_{r-1})$, ensuring no double spends or contract violations.
- 6) It verifies the reward distribution transaction according to the PoCol reward distribution function (Section 5.9).
- 7) It verifies that $H(\text{Serialize}(\text{Header}_r(n))) \leq target$ for the received complete header and that the header's Merkle root and other committed fields are consistent with $BlockTemplate_r$.
- 8) A header that satisfies the target but was constructed from different non-nonce fields is not a valid PoCol result for round r . Such a header derives from a different template, so its $TemplateID$ does not match the committed $TemplateID_r$ and the corresponding block is rejected regardless of its proof of work.

If all checks pass and the chain selection rule selects this chain as the canonical one (e.g., by longest-chain or most-work rules), the node appends Br to its local ledger, updates $S(Br)$ accordingly, and removes included transactions from its mempool. Conflicting blocks are handled using standard fork resolution rules from Nakamoto-style consensus [1], [46].

5.9. Reward Distribution Mechanism in PoCol

Let round r attempt to produce block B_r extending B_{r-1} . We denote by $M_r = \{m_0, \dots, m_{n-1}\}$ the registered miner set used for round r . Let $R_{blk}(r)$ be the protocol-defined base block reward (subsidy) and $F_{tx}(r)$ the total transaction fees included in B_r . The total distributable reward in round r is

$$R_{tot}(r) = R_{blk}(r) + F_{tx}(r) \quad (5.6)$$

PoCol introduces a reward distribution function

$$f_r: M_r \rightarrow [0,1]$$

which maps each miner identifier to a reward share $s_i = f_r(m_i)$. The shares satisfy

$$\sum_{m_i \in M_r} f_r(m_i) = 1, \quad s_i = f_r(m_i) \geq 0 \quad (5.7)$$

The monetary reward paid to miner m_i in round r is then

$$\text{Reward}_r(m_i) = s_i R_{\text{tot}}(r) \quad (5.8)$$

In the simplest equal-sharing baseline, PoCol sets

$$f_r(m_i) = \frac{1}{|M_r|}, \quad \forall m_i \in M_r \quad (5.9)$$

so, every registered miner receives an identical fraction of each block reward. This maximally reduces income variance for small miners and provides strong incentives to remain registered, at the cost of rewarding miners even if their participation in a given round cannot be directly measured. More advanced designs can use f_r to encode hash-rate estimates, historical activity, stake or reputation, as long as their inputs are observable and verifiable from on-chain state and are fixed before the round begins, so that $T_{\text{reward}}(r)$ can be computed deterministically prior to mining [38], [27].

5.9.1. Reward Distribution Transaction

The reward distribution transaction $T_{\text{reward}}(r)$ generalizes the coinbase transaction in PoW systems [49]. It is computed before mining from $S(B_{r-1})$ and the protocol rules, included in BlockTemplate_r , and committed through the Merkle root. The winning miner does not create or alter this transaction after producing a successful header, it broadcasts the committed block template instantiated with the successful nonce value and any required proof or signature. $T_{\text{reward}}(r)$ has:

- One output per miner $m_i \in M_r$, paying amount $\text{Reward}_r(m_i)$ to the on-chain reward address registered for m_i ,

- Inputs that represent the newly created subsidy $R_{\text{blk}}(r)$ and the collected transaction fees $F_{\text{tx}}(r)$, according to the underlying UTXO or account model.

Conceptually, $T_{\text{reward}}(r)$ is a multi-recipient coinbase: instead of paying the entire reward to the single winner, PoCol encodes collaborative rewards as multiple outputs in a single transaction. For ease of parsing and verification, $T_{\text{reward}}(r)$ is placed as the first transaction in B_r , following the usual convention for coinbase transactions in PoW blockchains [38], [49].

5.9.2. Verification Rules

When a node receives a proposed block B_r , it recomputes:

- 1) $R_{\text{tot}}(r)$ from the protocol rules and the included transactions,
- 2) The miner set M_r from the state S_{r-1} ,
- 3) The share vector $\{s_i\}$ using the agreed function f_r and any on-chain metrics it depends on.

The node then verifies that:

- *Completeness* : $T_{\text{reward}}(r)$ contains exactly one output per miner in the eligible set (typically all $m_i \in M_r$, or a filtered subset if activity-based policies are used),
- *Correctness*: for each m_i , the payment amount equals $s_i \cdot R_{\text{tot}}(r)$ up to negligible rounding error,
- *Conservation*: the sum of all reward outputs equals $R_{\text{tot}}(r)$, no extra coins are created or destroyed,
- *Validity* : $T_{\text{reward}}(r)$ is well-formed under the base ledger rules (no double spends, non-negative output values, etc.).

If any of these conditions fails, the block is rejected even if its proof-of-work is valid. This prevents a malicious leader from biasing the reward allocation in its favour. The pattern is similar to the validation of coinbase transactions and miner fees in existing PoW networks, but extended to multi-recipient rewards [38], [11].

5.9.3. Inactive Miners and Policy Options

Let $A_r \subseteq M_r$ denote the set of active miners in round r (e.g., those that are online and hashing). The baseline PoCol design uses the full M_r in (5.7)–(5.9), meaning inactive miners still receive rewards until they explicitly deregister or are removed by inactivity rules. This keeps the protocol simple but may be undesirable if large numbers of registered miners are persistently inactive.

Deployment-specific variants can instead define a filtered set $M_r \subseteq M_r$ consisting only of miners that satisfy some activity predicate (for example, having mined at least one block or emitted signed heartbeats within a sliding window of recent rounds). The reward function is then evaluated over M_r and re-normalized so that $\sum_{m_i \in M_r} f_r(m_i) = 1$. In this variant, miners that are offline for too long effectively lose their share of rewards until they become active again or are formally removed from the registered set. This approach strengthens incentives for continuous participation and aligns with reputation- and credit-based schemes studied for DPoS and BFT-style consensus [28], [52].

5.9.4. Interaction with Miner Set Dynamics

Because PoCol's reward mechanism is driven entirely by M_r and the deterministic function f_r , it naturally adapts to miner set changes. When a new miner joins (via registration) and becomes part of M_r , the next round automatically includes it in $T_{\text{reward}}(r)$ with a non-zero share. When a miner deregisters or is removed due to inactivity, it disappears from M_r and thus receives no further rewards. There is no need for additional coordination or side agreements: the on-chain state fully determines who is eligible to receive a share of $R_{\text{tot}}(r)$ in each round [150].

Figure 5.3 shows how rewards are distributed according to Section 5.9, illustrated as a scenario in which Miner B became the winning miner on arbitrary round r . Before mining begins, every miner reads the registered miner set M_r from its ledger view, applies the reward distribution function f_r , and computes the deterministic reward-distribution transaction committed in the canonical block template, with each registered miner's share drawn from the block subsidy plus the transaction fees. When the complete candidate header instantiated by Miner B's nonce value hashes at or below the target, Miner B does not create or modify any transaction, it publishes the already committed block template instantiated with the successful nonce value. The reward distribution transaction $T_{\text{reward}}(r)$ has one output for each registered miner and is entered into the block before being transmitted. Verification rules described in Section 5.9.2 are followed, meaning that any receiving miner must confirm that the beneficiaries are equal to M_r , thus confirming that every share is equal to the prescribed

value f_r , that the sum of outputs equals $R_{\text{tot}}(r)$, and that neither value was created nor lost, prior to committing the block and updating balances accordingly.



Figure (5.3): PoCol reward distribution and verification in round r , from production of a proof-of-work-valid candidate header by the successful miner to balance updates at all registered miners. $T_{\text{reward}}(r)$ is computed before mining and committed in the canonical block template, the miner publishes that template instantiated with the successful nonce value.

5.10. Dynamic Miner Set Management in PoCol

In this section we reuse the notation from the system model in Chapter 4. In particular, let

$M(r)$ denote the set of miners that are registered at the beginning of round r , with cardinality

$n(r) = |M(r)|$. For each miner $m_i \in M(r)$ we denote its hash rate by $H_i(r)$, and the aggregate network hash rate by

$$H_{\text{tot}}(r) = \sum_{m_i \in M(r)} H_i(r)$$

Chapter 4 defined the expected block-arrival rate $\lambda_{\text{blk}}(r)$ and the expected block interval $T_{\text{blk}}(r)$ as functions of $H_{\text{tot}}(r)$ and the difficulty parameter D_r , for example via

$$\lambda_{\text{blk}}(r) = H_{\text{tot}}(r) p_{\text{succ}}(D_r)$$

$$T_{\text{blk}}(r) = \frac{1}{\lambda_{\text{blk}}(r)}$$

where $p_{\text{succ}}(D_r)$ is the per-hash success probability at difficulty D_r [1], [46]. Dynamic changes in the miner set $M(r)$ thus directly impact the performance and energy model derived in Chapters 3–4, because they modify both $H_{\text{tot}}(r)$ and the size of the registered set used by the PoCol nonce distribution function.

5.10.1. Joining New Miners

A new miner joins PoCol by issuing a registration transaction that is included in some block B_h . As in Section 5.3, this transaction carries the miner's public key, network address and reward address. Let $M(h)$ be the miner set after applying all transactions in B_h . Then, for any round $r > h$, the new miner appears in $M(r)$ and is consequently:

- included in the registered miner list used by the nonce-domain distribution function (Section 5.6), and
- included in the miner set used by the reward distribution function f_r (Section 5.9).

From the analytical point of view, adding a miner m_j at height h does two things:

- 1) It increases the total hash rate:

$$H_{\text{tot}}(r) \leftarrow H_{\text{tot}}(r) + H_j(r), \quad \forall r > h$$

which shortens the expected block interval $T_{\text{blk}}(r)$ at fixed difficulty until the difficulty-adjustment mechanism responds [9], [11].

- 2) It increases the size of the registered set $n(r)$, so the nonce domain $\{0, \dots, N_{\text{max}} - 1\}$ is partitioned into more sub-ranges (Equations (5.4a)–(5.5)). This does not change the aggregate probability of success for honest miners, but it refines the granularity of the PoCol assignment and slightly reduces the size of each individual miner's nonce range.

If the reward distribution function f_r uses equal sharing (Equation (5.9)), the arrival of a new miner also reduces the per-block share of all existing miners: each m_i receives $1/|M(r)|$ of the total reward in future rounds. If f_r is weighted by hash rate or stake, the impact depends on the chosen weighting scheme but remains completely determined by on-chain information and the formal definition of f_r .

5.10.2. Leaving Miners (Explicit and Silent)

Miners may leave the system explicitly or silently:

- In explicit leave, the miner issues a deregistration transaction that is confirmed in some block B_h . From that point on, the miner is removed from the registered set, so for all rounds $r > h$ it holds that

$$m_i \notin M(r), \quad H_i(r) = 0$$

Consequently, the miner is no longer included in the nonce assignment (Section 5.6) nor in the reward distribution f_r (Section 5.9).

- In a silent leave, the miner simply stops hashing or goes offline without deregistering. In this case, its identifier remains in $M(r)$, and its nominal hash rate may still be modelled as $H_i(r) = 0$ in the analytical model of Chapter 4. Practically, the miner continues to appear in the PoCol miner list, but its assigned nonce range $R_r(m_i)$ is never explored.

In terms of the performance model, both explicit and silent leaves reduce the effective total hash rate from $H_{\text{tot}}(r)$ to

$$H_{\text{eff}}(r) = \sum_{m_i \in A(r)} H_i(r)$$

where $A_r \subseteq M(r)$ denotes the active subset of miners in round r . The block-arrival rate becomes

$$\lambda_{\text{blk}}^{\text{eff}}(r) = H_{\text{eff}}(r) p_{\text{succ}}(D_r)$$

which may be strictly smaller than the nominal $\lambda_{\text{blk}}(r)$ used in Chapter 4 if inactive miners are still counted in $M(r)$. This is analogous to hash-rate drops in classical PoW, where miners switching off hardware increase the actual block interval until the difficulty-adjustment algorithm reacts [9], [49].

From the incentive perspective, explicit leaving removes the miner from the reward function f_r , while silent leaving still allows the miner to collect rewards in the baseline equal-sharing design (because $m_i \in M(r)$ so long as it is not deregistered). The inactivity-aware variants of f_r discussed in Section 5.9.3 remove or down-weight such miners by conditioning on activity predicates, thereby aligning the economic model with the energy model.

In Figure 5.4, the explicit leaving procedure is illustrated according to sections 5.4.2 & 5.10.2. During this process, the miner who leaves (miner C) creates the deregistration transaction, with a signature, identifier (ID), network address, and timestamp on it, and sends it to the network. The receiving miners validate the transaction by ensuring that miner C's signature is valid and that miner C is still a member of the current registered miner set. Once validated, the transaction is accepted into a miner's memory pool (mempool) by each of those receiving miners. Once all the miners validate the original deregistration transaction, the deregistration transaction itself will be added to the next block created by collaborative mining. Once the block has been created and validated, all miners will now be able to derive the new registered miner set that includes only miner A and miner B. Therefore, miner C will no longer participate in any future rounds of the blockchain validation process or receive an assigned nonce range from the nonce-domain partitioning function. Additionally, the next invocation of the nonce distribution function will repartition the permitted nonce domain among the remaining miners, so no nonce range is assigned to Miner C after deregistration.



Figure (5.4): PoCol miner leaving sequence, showing explicit deregistration of Miner C and the resulting update of the registered miner set.

5.10.3. Long-Term Inactivity and Forced Removal

To avoid unbounded growth of $M(r)$ with many zero-hash-rate miners, PoCol can be extended with an inactivity-based removal rule. We introduce a function

$$\ell_i(h)$$

that records, for each miner m_i , the last block height at which its activity has been observed (e.g., as a block proposer, participant in a heartbeat mechanism, or contributor of verifiable work evidence—depending on the specific deployment).

Given an inactivity threshold $\theta_{\text{inact}} > 0$ (in blocks), a miner is declared inactive if, at height h ,

$$h - \ell_i(h) > \theta_{\text{inact}}$$

Any honest node may then propose a forced-removal transaction that, once included in a block, updates the state so that $m_i \notin M(r)$ for all subsequent rounds $r > h$. In the analytical model, this operation removes the miner from the sum defining $H_{\text{tot}}(r)$ and from the normalization in (5.7)–(5.9), all subsequent performance, energy and reward equations evolve with the new, reduced set $M(r)$.

This mechanism is conceptually similar to liveness penalties or slashing for non-participating validators in PoS and BFT-style protocols, but in PoCol it affects eligibility and reward sharing only, not stake balances [21], [92]. Tuning θ_{inact} allows system designers to balance stability (rare changes in $M(r)$) against responsiveness (quick removal of long-term inactive miners), in line with the workload and energy constraints analysed in Chapters 3–4.

5.11. Security and Robustness Analysis of PoCol

We now reinterpret the qualitative security discussion of PoCol using the formal notation and properties introduced in Chapters 3–4, in particular the Bitcoin backbone-style properties of common prefix, chain growth, and chain quality for PoW systems [46], [47]. Let $C_v(t)$ denote the chain adopted by an honest node v at time t , and let $\text{len}(C_v(t))$ be its length. The analysis in Chapters 3–4 assumed that:

- honest miners collectively control at least a fraction $(1 - \alpha)$ of the total hash rate, with $\alpha < 0.5$, and
- the network eventually behaves as a partially synchronous channel with bounded delay after a stabilization point [7], [47].

Under these assumptions, we argue that replacing vanilla PoW with PoCol’s deterministic nonce-domain partitioning does not invalidate the backbone results, it only changes how the honest hash power is organized.

5.11.1. Resistance to Double Spending and Chain Reorganizations

In the classical PoW model, the security of transaction finality is captured by the common-prefix property: once an honest node sees a block sufficiently deep in its chain (e.g., with k confirmations), the probability that this block will ever be removed from the common prefix of all honest chains decays exponentially in k [46], [34].

In PoCol, the effective per-round block-finding race is unchanged from the perspective of the backbone abstraction:

- Honest miners still contribute a total hash rate $H_{\text{hon}}(r) = (1 - \alpha) H_{\text{tot}}(r)$.
- Adversarial miners control the remaining $\alpha H_{\text{tot}}(r)$.
- A successful block in round r requires a hash output below the difficulty threshold D_r , with per-hash success probability $p_{\text{succ}}(D_r)$.

The only difference is that honest miners are restricted to disjoint nonce ranges determined by the PoCol distribution function, adversarial miners can deviate but will be detected if they claim ranges outside their assignment (Section 5.11.2). As long as all honest miners respect their ranges, the distribution of successful blocks between honest and adversarial miners is still governed by their hash-rate proportions, not by nonce range positions. Thus, the standard probability that an adversary with fraction α of the total hash rate can successfully build a longer private chain than honest miners over a window of length k remains the same asymptotically as in the backbone model.

Informally, PoCol is designed to preserve the conditions under which it can be argued that the double-spend probability for a transaction with k confirmations decreases exponentially in k , for $\alpha < 0.5$ and partially synchronous networks, in analogy with the Bitcoin backbone analysis [46], [34], a complete formal proof of these properties for the full common-template protocol remains future work.

5.11.2. Nonce-Domain Violations Analysis

By definition of the partitioning function (Equations (5.4a)–(5.5)), the assigned ranges $R_r(m_i)$ of honest miners are pairwise disjoint and cover the permitted nonce domain N_r . A duplicated proof-of-work evaluation occurs only when the same serialized complete header is hashed more than once. Reuse of the same numeric nonce is not a duplicate if the header template differs. For a fixed round r , let $HeaderTemplate_r$ be the common nonce-free header template and define the

candidate-header input domain $D_r = \{\text{Serialize}(\text{Header}_r(n)) \mid n \in N_r\}$. The partitioning function assigns miner m_i the subset $D_r(m_i) = \{\text{Serialize}(\text{Header}_r(n)) \mid n \in R_r(m_i)\}$. Because $R_r(m_i) \cap R_r(m_j) = \emptyset$ for $i \neq j$, the subsets $D_r(m_i)$ are also disjoint. Thus, honest miners do not evaluate the same serialized candidate-header input within a round. This is an input-duplication statement, not a statement about cryptographic hash collisions.

A malicious miner might attempt to:

- 1) Evaluate nonce values outside its assigned range, thereby enlarging the nonce subset that it searches, or
- 2) Claim a block where the winning nonce lies in another miner's range.
- 3) Mine a modified template $T'_r \neq T_r$ (for example, with a different transaction set, timestamp, or reward transaction) in order to search outside the authorized round domain D_r .

PoCol's verification rules (Section 5.8.5) explicitly recompute $R_r(m_i)$ for the miner who claims to have produced the block and check that the advertised nonce indeed lies in the corresponding range. If this check fails, the block is rejected by honest nodes. As a result, range deviations are detectable and cannot give adversaries an undetected advantage. In particular, an adversary cannot covertly steal another miner's range without being noticed by all honest validators. Likewise, a miner that changes T_r is no longer searching the authorized round domain D_r , its proposal carries a *TemplateID* different from the committed TemplateID_r and is rejected during verification (Section 5.8.5), even if the proof-of-work satisfies the difficulty target.

Reuse of the same numeric nonce range in different rounds is benign because the canonical block and header templates, and therefore the serialized candidate headers, change between rounds. The hash function is applied to different header inputs even when a numeric nonce value is reused [46], [34].

Enforcing the Common-Template Invariant prevents unauthorized changes to non-nonce header fields, divergence of the committed transaction set, duplicate serialized candidate-header inputs among honest miners, unauthorized timestamp changes, and post-mining modification of the reward-distribution transaction. This clarifies why nonce-domain partitioning alone would not prevent duplicate inputs if miners were permitted to use different header templates.

At the same time, the common-template mechanism introduces a coordination requirement: all miners must obtain the same template even though their mempools and network views may differ transiently. PoCol achieves this agreement without a centralized coordinator through the deterministic derivation rule of Section 5.8.2, whose inputs — the committed state $S(B_{r-1})$ and the propagated transaction set, under the mempool-consistency assumption of Section 4.2 — are public and identical for all honest nodes, a miner whose local view temporarily diverges detects the mismatch through $TemplateID_r$ and resynchronizes before searching. Where mempool synchrony is imperfect in a real network, template agreement may transiently fail for some miners, making this agreement robust (for example, through a committed transaction list announced together with the previous block) is identified as an implementation requirement in Chapter 6 and as a mandatory extension in future work.

5.11.3. Effects of Inactive and Partially Active Miners

Chapters 3–4 modelled the relation between total hash rate $H_{\text{tot}}(r)$, block interval $T_{\text{blk}}(r)$, and energy-per-block $E_{\text{blk}}(r)$. PoCol introduces an additional distinction between the nominal miner set $M(r)$ and the active subset A_r . The effective hash rate driving consensus is

$$H_{\text{eff}}(r) = \sum_{m_i \in A(r)} H_i(r) \leq H_{\text{tot}}(r)$$

If inactive miners remain in $M(r)$ but contribute $H_i(r) = 0$, then:

- The security analysis (common prefix, chain growth, chain quality) still holds when re-parameterized with $H_{\text{eff}}(r)$, because security depends on relative honest vs adversarial hash rate, not on the size of $M(r)$.
- The performance and energy model must use $\lambda_{\text{blk}}^{\text{eff}}(r) = H_{\text{eff}}(r) p_{\text{succ}}(D_r)$ and the corresponding $T_{\text{blk}}^{\text{eff}}(r)$, otherwise the system may experience longer block intervals than anticipated [9], [49].

Partially active miners, whose hash rate varies over time, are naturally captured by time-dependent functions $H_i(r)$. In the limit, the expected number of blocks acquired by miner m_i over a long window is proportional to its time-average share of $H_{\text{eff}}(r)$, in line with the backbone chain-quality property. From the energy perspective,

Chapters 3–4 can interpret these dynamics as time-varying operating points of $E_{\text{blk}}(r)$ under PoCol, just as they do under classical PoW.

5.11.4. Sybil Attacks and Miner Identity Management

Because PoCol distributes nonce ranges per identity rather than directly per unit hash rate, a naive implementation could be vulnerable to Sybil attacks: an adversary could split its hardware into many registered identities $\{m_i\}$ and attempt to obtain a larger share of the nonce domain and reward. To prevent this, PoCol must be paired with identity and registration policies that tie miner identities to some resource:

- In permissioned settings, $M(r)$ may be curated by an administrator or consortium, and identities correspond to vetted organizations (similar to PoA or PBFT-style deployments) [31], [37].
- In permissionless settings, the registration process can require economic or computational costs, such as a proof-of-work or a security deposit, so that spawning many identities becomes expensive. This mirrors the role of stake in PoS protocols and deposits in hybrid PoW/PoS designs [27], [29].

Additionally, the design of the reward function f_r is crucial. If f_r depends only on aggregate hash rate or stake (for example, via weights proportional to observed hash power), splitting a given hash capacity across many identities does not increase the total long-term reward, it merely redistributes it among identities controlled by the same adversary. Used together, (i) costed registration and (ii) an f_r that is not trivially identity-proportional make Sybil attacks economically unattractive.

5.11.5. Comparison with Existing Consensus Mechanisms

Finally, we position PoCol vis-à-vis the consensus mechanisms surveyed in Chapters 2–3:

- Relative to Nakamoto-style PoW, PoCol keeps the same backbone security assumptions and analytical structure, but modifies the candidate-header evaluation process (a common header template with disjoint nonce ranges) and reward allocation (collaborative coinbase). The formal properties of common prefix, chain growth and chain quality are expected to remain valid under the same conditions on hash power and network synchrony — a preliminary argument rather than a proven equivalence, as a formal proof for the complete common-template protocol has not been carried out, while the energy model in Chapter 3 can be adapted to show reduced redundant hashing and potentially improved energy efficiency [9], [49].

- Relative to PoS protocols such as Ouroboros, Algorand, and Snow White, PoCol avoids stake management, slashing and long-range-attack issues by staying in the PoW security model, at the cost of retaining some energy intensity. However, by reducing duplicate work, PoCol can shift the system along the energy–security trade-off curve toward more sustainable operating points highlighted in recent blockchain energy studies [99], [11].
- Relative to BFT / PBFT-style permissioned consensus, PoCol scales more naturally to large, open miner populations because it avoids quadratic communication and explicit quorum formation in each round. PBFT and its descendants provide fast finality under partial synchrony and known validator sets, but they are harder to deploy in permissionless settings with frequent joins and leaves. PoCol trades instantaneous finality for probabilistic finality, but remains compatible with the open, dynamically changing miner sets analysed in this chapter [31], [37].

In summary, when re-expressed in the notation and analytical framework of Chapters 3–4, PoCol can be viewed as a structural refinement of PoW that keeps the same backbone security model, refines the control of energy and fairness via deterministic nonce-domain partitioning and collaborative rewards, and offers a middle ground between classical PoW, PoS, and BFT-based consensus.

Parallel Proof-of-Work provides the closest architectural comparison to PoCol because both approaches distribute mining computation among cooperating participants [134]. PoCol differs by binding deterministic, mutually disjoint nonce ranges to one identified immutable candidate-block template and by defining explicit miner behavior after an assigned range is exhausted.

Green-PoW provides the most relevant energy-policy comparison [15]. Green-PoW reduces energy use by reducing the number of miners allowed to participate in selected rounds, whereas PoCol’s idle policy initially permits broad participation and subsequently reduces each miner’s active hashing duration after its assigned range is completed.

Proof of Team Sprint provides the most recent peer-reviewed team-based collaborative comparison [137]. Collaborative Proof-of-Work and StrongChain provide relevant precedents for reward fairness, contribution verification, and contribution visibility [136], [135]. APoW provides a potential direction for verifying whether an assigned nonce region was actually searched [138].

These comparisons do not establish universal superiority. PoCol’s proposed distinction is the integration of immutable template agreement, deterministic non-overlapping nonce allocation, collaborative reward distribution, and selectable idle or continuous operating policies. Its complete distributed implementation and contribution-verification mechanism remain areas requiring further development and evaluation.

5.12. Summary

This chapter has presented Proof of Collaboration (PoCol), a PoW-inspired protocol in which all registered miners in a round use one immutable canonical block template and one derived nonce-free header template. The permitted nonce domain is deterministically partitioned into assigned ranges, enabling coordinated candidate-header evaluation and reward sharing without a trusted range coordinator.

The core of PoCol is its deterministic nonce-range allocation function. It computes a round-specific seed from the previous block hash, round number, and $TemplateID_r$, derives a pseudo-random miner ordering, and assigns each miner a non-overlapping nonce range. Together with the Common-Template Invariant, this ensures that, for the fixed nonce-free $HeaderTemplate_r$, no two honest miners evaluate the same serialized candidate header $Header_r(n)$ in a round. PoCol retains a hash-based proof-of-work condition and cumulative-work chain structure. The preliminary analysis in Section 5.11 suggests compatibility with PoW-style assumptions, while formal equivalence of common-prefix, chain-growth, chain-quality, and double-spend guarantees remains future work.

A key contribution of PoCol is its collaborative reward distribution mechanism, which allocates block rewards and fees among all registered miners according to a configurable reward function. This design reduces income variance, can mitigate incentives for mining centralization, and provides a basis for incorporating additional collaboration metrics in future extensions. Finally, we sketched an informal security and robustness analysis, arguing informally that PoCol is designed to maintain standard blockchain security properties while addressing energy efficiency and fairness concerns, formal security equivalence remains to be established.

In the following chapters, PoCol’s performance is evaluated quantitatively through simulation, comparing the throughput, approximate confirmation time, stale-block behaviour, and energy consumption of the idealized PoCol abstraction against a classical PoW baseline, reward-fairness evaluation and comparison with further consensus mechanisms remain future work.

Chapter 6

Implementation and Experimental Setup

Chapter 6

Implementation and Experimental Setup

6.1. Chapter Scope and Objectives

This chapter outlines the execution of the Proof-of-Collaboration (PoCol) mechanism and how its associated experiments were designed, executed, recorded and analyzed. The chapter does not evaluate the results, this will be done in the Evaluation chapter. Instead, this chapter documents the implementation of the PoCol mechanism and the environmental conditions under which the output files for this thesis were produced. The framework for the simulation workflow used in the PoCol implementation conforms to the precedent set within the blockchain research community regarding the controlled and repeatable testing of blockchain networks and behaviours using configurable network parameters (for instance, bandwidth) and consensus parameters (such as gas price or tokens/day) [40], [41].

BlockSim, an event driven blockchain simulator, was used for the implementation of PoCol. BlockSim has been utilized extensively in previous research as a tool for simulating the operation of Bitcoin-style networks and extending consensus and incentive calculation methods [40], [151]. The decision to utilise this simulator was based upon the findings of previous research that simulation is an acceptable method for conducting blockchain network evaluations when direct experimentation of large, geographically dispersed blockchain networks is impractical, difficult to control and cannot be replicated [41], [152]. The outputs of the BlockSim simulations were exported into formatted log workbooks, which serve as the basis of this chapter, as well as the comparative evaluations presented in Chapter 7.

6.2. Simulator Platform and Baseline Model

BlockSim, the implementation platform, is based on a discrete event process for modeling blockchain functionality. Mining, propagation, block reception, and chain selection are mapped explicitly to BlockSim's model. Prior research has demonstrated

BlockSim's effectiveness in exploring network propagation, the mining process, and miner incentives as well as having the capability of supporting custom blockchains not just limited to Bitcoin [151]. Similarly, numerous researchers have noted that BlockSim is one of the most advanced and extensible blockchain simulators for conducting research on protocol design [41].

The baseline model investigated in this research represents a Bitcoin-type proof of work (PoW) solution and is defined in the worksheet associated with the research as *Model = 1*. The *Model = 1* refers to the mining model of competition where individual miners explore the proof space independently and the first miner to find a block receives the block reward. The PoW model has historically been used for benchmark security and performance analyses of longest-chain solutions and has been well studied both analytically and in simulations [140].

6.3. PoCol-Specific Implementation in BlockSim

The PoCol implementation was introduced as a different way of getting consensus in the same simulation framework of Model 3. The ability to find the longest chain has been maintained from the Bitcoin-type simulator but with a new way of assigning jobs and accounting for work in four ways. First, instead of having miners worldwide compete in a global race, PoCol miners are assigned their search to do in disjoint and coordinated rounds. This is exactly as Chapter 5 proposed: working together in a nonce-domain that could be divided into many shared parts to work on all at the same time. Second, the logging of the rounds extended so that you can tell the difference between collaboration and ordinary mining events. In the EnergyLog sheet for both PoW and PoCol, there were specific events for each type of mining recorded, worked by who, what round, and when the blocks were mined. This allows a direct audit of the collaborative mining events during its execution. Third, the per-miner financial model has also been extended so that each successful run created an overall summary of how many times miners had attempted or hashed, the number of mined blocks created, the amount of profit created, the energy consumed, and the number of carbon emissions created. Fourth, the output for the blocks that were mined

was also put into the main-chain log so that the canonical chain can be reconstructed independently of the aggregate simulation summary.

Relative to the common-template protocol of Chapter 5, the prototype represents the template implicitly as shared round state rather than as byte-level block and header objects. For each round identified by a parent block, the simulator creates one abstract nonce domain sized to match the configured expected block time, partitions that domain into disjoint per-miner ranges, and samples one successful nonce value from the domain. The miner whose assigned range contains that value becomes the winner. Because all simulated miners refer to one shared round state and disjoint ranges, duplicate abstract candidate inputs are excluded by construction. The simulator does not, however, serialize or hash actual block headers.

However, the discrete-event prototype does not construct byte-level block templates: it does not materialize a canonical transaction ordering before mining, does not compute a `TemplateID`, and does not pre-commit the reward-distribution transaction inside a Merkle root (transactions and rewards are attached to the block object when the winning event is executed), consequently, it does not mechanically verify template immutability or reject blocks built from modified templates. The common-template enforcement of Sections 5.8.2–5.8.5 — deterministic template derivation, the transaction-collection window with distributed transaction-set agreement and mempool reconciliation (Section 5.8.2), fixed transaction ordering, the pre-committed reward-distribution transaction, computation and verification of $TemplateID_r$, rejection of proposals whose `TemplateID` differs from the committed one, and deterministic regeneration of a new template after nonce-range exhaustion — is therefore a required protocol correction that forms part of the PoCol specification but is not yet implemented as executable checks in the current prototype. The reported results therefore evaluate the idealized behaviour of disjoint nonce-domain allocation under a shared round context: the abstraction idealizes away transaction-set disagreement, mempool divergence, `TemplateID` mismatches, template-reconciliation and transaction-retrieval delays, synchronization failures, the communication overhead of template agreement, miners temporarily unable to participate because their template differs, and any additional stale blocks caused by template disagreement. The results do not measure the communication, synchronization,

latency, energy, or fork-related overhead introduced by a full distributed implementation of canonical-template agreement. Consequently, the reported performance and energy results should be interpreted as proof-of-concept results for the collaborative search abstraction rather than measurements of a complete production implementation, completing and measuring these mechanisms is a mandatory item for future implementation work (Section 8.5).

These decisions were based on the findings in the wider blockchain evaluation literature, which states that a simulation model should allow for not only aggregate results but also the lower-level elements needed for reproducibility, sensitivity testing, and debugging [41], [152]. These methods are also aligned with performance-based research models for blockchain that divide consensus levels, network effects, and resource requirements into Producer/Producer or Peer/Peer and mined/blockchains or cluster blocks/resource levels observable from the three different scale perspectives of the consensus effect [40], [140].

6.4. Experimental Design

The experiment was designed to compare the performance of two consensus protocols on a controlled basis. The main independent variable was the number of miners participating in the protocol. Five different miner populations – 100 miners, 200 miners, 300 miners, 400 miners, and 500 miners – were created for each consensus protocol. Therefore, there were 10 main log files generated during the experiment (5 for PoW and 5 for PoCol).

All simulation inputs other than the miner population remained unchanged between test runs, these are included in the InputConfig sheet of each workbook. The parameters included in the InputConfig sheet were: target block *interval* = 600 seconds, block propagation *delay* = 0.42 seconds, total simulation *time* = 10,000 seconds, total network hash *rate* = 141×10^{12} hashes per second (H/s), miner *efficiency* = 21.5 joules per terahash (J/TH), and grid emission *factor* = 0.445 kilograms CO₂ equivalent per kilowatt hour (kgCO₂e/kWh). Because all of the miners in the ProfitEnergy logs were designed to be homogeneous (i.e., they were all configured to have the same nominal miner participation weight), there was no

difference in hardware class capabilities among the simulated miners. As indicated, this design decision was made in order to isolate the effect of collaborative partitioning of miner work from any other confounding factors introduced by having unequal miner hardware capability.

The use of fixed network and energy parameters follows the established practice prescribed in the blockchain evaluation literature where one modifies only a limited number of variables while maintaining all other variables the same as possible in order to retain fitness [41], [152]. In addition, using the same target block interval (600 seconds) and explanation of block propagation delay as the previous research on interaction between security, block generation, and network latency in PoW-style blockchain implementations [140].

Table (6.1): Fixed experimental parameters and output structure used across all scenarios.

Parameter	Value
Consensus models	PoW (Model 1) and PoCol (Model 3)
Miner populations	100, 200, 300, 400, and 500 miners
Target block interval	600 s
Block propagation delay	0.42 s
Simulation time	10,000 s
Total network hash rate	141×10^{12} H/s
Miner efficiency	21.5 J/TH
Grid emission factor	0.445 kgCO ₂ e/kWh
Output workbook sheets	InputConfig, SimOutput, ProfitEnergy, Chain, EnergyLog

The operating policy must be stated explicitly for each PoCol experiment. A complete comparative design distinguishes among: (1) a continuously operating PoW baseline, (2) PoCol under continuous operation, and (3) PoCol under the post-range idle policy. Energy accounting for the idle scenario must separately record active mining time, idle time, active power, idle power, and coordination overhead.

Parallel PoW, Green-PoW, Proof of Team Sprint, Collaborative Proof-of-Work, StrongChain, and APoW are treated as conceptual and architectural comparisons unless their complete algorithms are independently implemented under identical

simulation parameters. Numerical superiority must not be claimed from cross-paper results obtained using different hardware, network sizes, workloads, difficulty settings, or energy models.

6.5. Logged Outputs and Data Collection Pipeline

For all simulations, a single workbook was produced containing five (5) types of Data Sheets as described below:

1) Input Config - This Data Sheet contains the configurations of the scenarios performed during the simulation. It also helps to repeat runs later with the same parameters.

2) Sim Output - This Data Sheet contains the aggregate output of all simulations, which includes total numbers of blocks (main and stale), the number of transactions, the total number of attempts/hashees made, the total kWh used, and the total amount of CO₂ emitted.

3) Profit Energy - This Data Sheet allows you to view Profit Energy statistics for each miner, including averages, rewards, and CO₂ and kWh values associated with each miner's activity.

4) Chain - This Data Sheet provides a complete history of the canonical chain for the entire run which allows reconstruction of main-chain depth, proposer sequences for each block and the time each block was proposed (timestamp).

5) Energy Log - This Data Sheet shows the complete time-based execution events for all activities performed during the simulation with particular reference to differences in event semantics that exist between PoW and PoCol.

The aforementioned logging structure is consistent with recent findings regarding how to evaluate public blockchains where both the Network and miner activities have a combined impact on the resulting Consensus Protocol observed behavior as studied in previous research [41], [152]. In addition, the energy and CO₂ accounting data included with this logging structure reflects an early stage of research where the evaluation of blockchain protocols is no longer solely focused on throughput and security, but now considers energy use and environmental footprint [20], [12].

6.6. Derived Performance, Energy, and Sustainability Indicators

The primary logs were analyzed, which allowed for a calculation of a few generated data types that can be used in the comparative study. The first derived value was accepted main-chain block number, which came from the SimOutput and Chain sheets. The second value was stale-block count and stale rate, calculated from the simulation output. The third value was total transaction volume accepted during the run. The fourth value was the total computing work represented through the number of hashes/i.e. attempts that are recorded.

To normalize the energy outputs, two new variables were created. The energy per accepted block was calculated by finding the total energy over the number of main blocks. Furthermore, the energy per transaction was calculated from the total energy divided by the total accepted transactions. Carbon footprint work-level was calculated from the CO₂ totals logged in the workbook and consists of the direct correlation with recorded CO₂ emissions. These values allow for a protocol-neutral means of comparison because the work it takes to secure accepted blocks can be analyzed in terms of energy needed to perform transaction procedures and the carbon footprint that each design has based on the same electricity assumptions [20], [12].

The decision to focus more upon the normalized energy values is also backed by literature since the comparative studies indicate that using the total energy consumed in a network is not sufficient when there are various acceptance behaviors and transaction packing. Therefore the energy per the accepted unit of useful work must be determined [20].

6.7. Scenario Matrix and Traceability of Executed Runs

Table 6.2 contains the summary of the ten runs from the logs that were uploaded. The table will help to demonstrate experimental reproducibility ahead of evaluation where the end results would need to be explained.

Observations can already be made that PoCol implementation provides a far different set of results from ‘normal’ PoW. A comparison of the PoW and PoCol results shows that while the latter exhibits much lower aggregate energy consumption, it also exhibits a much higher occurrence of stale blocks. This phenomenon could

already be shown here for the current observation without interpreting the reasons why it happens or whether it can be considered good or bad. Further analysis will be done in Chapter 7.

Table (6.2): Summary of executed PoW and PoCol scenarios derived from the uploaded log files.

Protocol	Miners	Main blocks	Stale rate (%)	Total energy (kWh)	Energy/main block (kWh)	Energy/tx (Wh)
PoW	100	18	0.00	7.9018	0.4390	0.2452
PoW	200	14	0.00	15.5580	1.1113	0.6195
PoW	300	16	0.00	24.4860	1.5304	0.8533
PoW	400	27	0.00	33.0783	1.2251	0.6830
PoW	500	19	0.00	40.4078	2.1267	1.1844
PoCol	100	18	35.71	0.1308	0.0073	0.0039
PoCol	200	36	68.14	0.3001	0.0083	0.0045
PoCol	300	62	78.40	0.4695	0.0076	0.0041
PoCol	400	59	82.75	0.4542	0.0077	0.0042
PoCol	500	46	85.67	0.3350	0.0073	0.0039

6.8. Reproducibility Procedure

To reproduce a run, one must load the corresponding consensus model, set the number of miners to be used, and keep all other parameters listed in Table 6.1 constant. Once the experiment is complete, the simulation will produce outputs in a five-sheet format identical to the one produced in this thesis. Post-processing will yield normalized ratio results from SimOutput, which will be validated against Chain and EnergyLog. Specifically, the accepted blocks will be validated using findings from SimOutput and the canonical-chain sheet, while event semantics will be validated by reviewing the EnergyLog reason field.

In the field of simulation-based blockchain research, the ability to reproduce results depends not only on disclosing the parameters used but also on preserving the exact roll-forward of event traces and subsequent transformations of simulation outputs [41], [152]. Consequently, the workbook is treated as a primary empirical record for each experimental run within this thesis.

6.9. Threats to Validity and Limitations of the Setup

The limitations of current research should be highlighted. To start with, every scenario corresponds to just one logged run. That means that current chapter demonstrates just the observed effect, not its statistical significance. Future work should introduce additional runs using different random seeds. Secondly, the miner population is homogeneous in the current research. It implies that the current research does not resemble real mining eco-systems since the hash rate, network position, and energy source vary among miners in reality. Third, the simulation time of 10,000 seconds is sufficiently long for controlled comparison purposes, however, it is of short length concerning the need for studying long-term block-chain behavior, like changes in difficulty or structural centralization tendencies.

Fourth, both energy and carbon provided by the simulation depend on the inner accounting model used by the simulator and on the selected emission factor. In previous studies, the problem of energy evaluation has been raised and proved to be sensitive to assumptions, efficiency of hardware, and the definition of useful work. Fifth, the analysis of the logs indicates the high level of stale blocks in PoCol simulation at high miner populations. While this finding is important, it is still not clear whether this effect is caused by consensus procedure, scheduling, propagation methods, or the peculiarities of collaborative rounds [20], [152].

Sixth, and most fundamentally, the prototype implements an idealized shared-round abstraction of PoCol rather than the complete common-template protocol of Chapter 5: byte-level templates, TemplateID computation and verification, the transaction-collection window, distributed transaction-set agreement, mempool reconciliation, and pre-committed reward enforcement are not executable in the current code. The reported results therefore evaluate the idealized behaviour of disjoint nonce-domain allocation under a shared round context and do not measure the communication, synchronization, latency, energy, or fork-related overhead of full canonical-template agreement, they should be read as proof-of-concept evidence for the collaborative search abstraction rather than as measurements of a complete production implementation.

6.10. Summary

This chapter provided documentation of the implementation of PoCol in BlockSim as well as a definition of the experimental setup to be used for comparing PoCol with the baseline PoW system. The chapter described the simulator platform, defined the PoCol-specific logging and accounting functions, identified the controlled experimental scenarios, specified the parameters to be kept constant throughout the experimental setup, described the structure of the result workbooks generated by the simulator, and identified the normalized performance measurements to be used for subsequent analyses.

Furthermore, the experiment produced ten result workbooks — five PoW scenarios and five PoCol scenarios, corresponding to miner populations of 100, 200, 300, 400, and 500 — each containing the configuration, aggregate information, miner accounting data, chain history, and energy-consumption data.

Thus, the thesis has a proof-of-concept simulation basis — one run per configuration, analysed descriptively without significance testing — for analyzing the behavior of PoCol and PoW in terms of accepted blocks, stale performance, transaction processing, energy consumption, and carbon emissions [40], [41].

Chapter 7

Experimental Results and Evaluation

Chapter 7

Experimental Results and Evaluation

7.1. Introduction

This chapter reports the empirical evaluation of the proposed Proof-of-Collaboration (PoCol) consensus mechanism using the modified BlockSim framework introduced in Chapter 6. The evaluation focuses on the direct comparison between PoCol and classical Proof of Work (PoW) under matched simulation horizons and miner populations. The analysis considers performance, energy consumption, carbon footprint, and the main trade-offs that emerge as network size increases [40], [150].

Unless stated otherwise, the direct experimental baseline in this chapter is PoW. References to PoS and PBFT are used only as contextual points of comparison from the literature, not as matched simulator runs under the same parameter settings. This distinction is important for avoiding over-interpretation of cross-protocol differences that were not measured in an identical test harness [76], [141].

7.2. Evaluation Objectives

The primary purpose of the evaluation is to ascertain whether PoCol can reduce energy use and carbon emissions compared to PoW while still maintaining acceptable performance of the blockchain. Specifically, the experiments were designed to answer three questions: Does collaborative nonce-domain allocation increase throughput or shorten the approximate confirmation time as the number of miners grows? To what extent are total energy consumption, energy per block, energy per transaction, and total CO_2 reduced? What ancillary effects arise, especially with respect to stale-block behaviour and scalability at larger miner populations [24], [67].

The simulator outputs presented in this chapter come from the instrumented BlockSim logs obtained for the following miner numbers: 100, 200, 300, 400 and 500. In this analysis, the total blocks, main-chain blocks, stale blocks transaction number, total energy and total carbon emission the simulation workbook belongs to every miner.

Table 7.1 presents a summary of the various metrics that have been taken from the simulations output, the metrics presented in this table indicate a definite pattern, as it can be observed that PoCol consumes relatively much less energy than PoW as well as obtaining lesser energy per accepted block and per transaction, and has higher throughput levels. However, it has been noted that with the increase of the number of miners, the rates of stale blocks in PoCol have risen considerably, indicating that the collaborative mining in the present form creates considerable fork-management difficulties.

Table (7.1): Summary of the main PoCol and PoW evaluation metrics extracted from the simulation logs.

Alg.	Miners	TPS	Approx. 6-conf (s)	Stale rate (%)	Energy (kWh)	Energy/block	CO ₂ (kg)
PoCol	100	3.32	3189	35.7	0.131	0.0073	0.058
PoW	100	3.22	3128	0.0	7.902	0.4390	3.516
PoCol	200	6.67	1567	68.1	0.300	0.0083	0.134
PoW	200	2.51	3959	0.0	15.558	1.1113	6.923
PoCol	300	11.33	957	78.4	0.470	0.0076	0.209
PoW	300	2.87	3635	0.0	24.486	1.5304	10.896
PoCol	400	10.78	974	82.7	0.454	0.0077	0.202
PoW	400	4.84	2182	0.0	33.078	1.2251	14.720
PoCol	500	8.53	1261	85.7	0.335	0.0073	0.149
PoW	500	3.41	3033	0.0	40.408	2.1267	17.981

7.3. Performance Results

7.3.1. Throughput under varying network sizes

Figure 7.1 depicts the average throughput across the entire simulation duration. In simulations with 100 miners, PoCol performs equally as well as PoW in throughput measure, but in all higher miner conditions, PoCol exceeds PoW in average throughput. The throughput in TPS for PoCol and PoW at 200 miners is 6.67 TPS and 2.51 TPS, respectively. The throughput gap at 300 miners between PoCol and PoW is 11.33 TPS to 2.87 TPS, respectively. The throughput figures of PoCol and PoW at 400 miners are 10.78 TPS and 4.84 TPS, respectively. It is clear, therefore, that collaborative work enables more blocks and transactions to accumulate on the main chain within the same simulation time period than in the current simulation configuration.

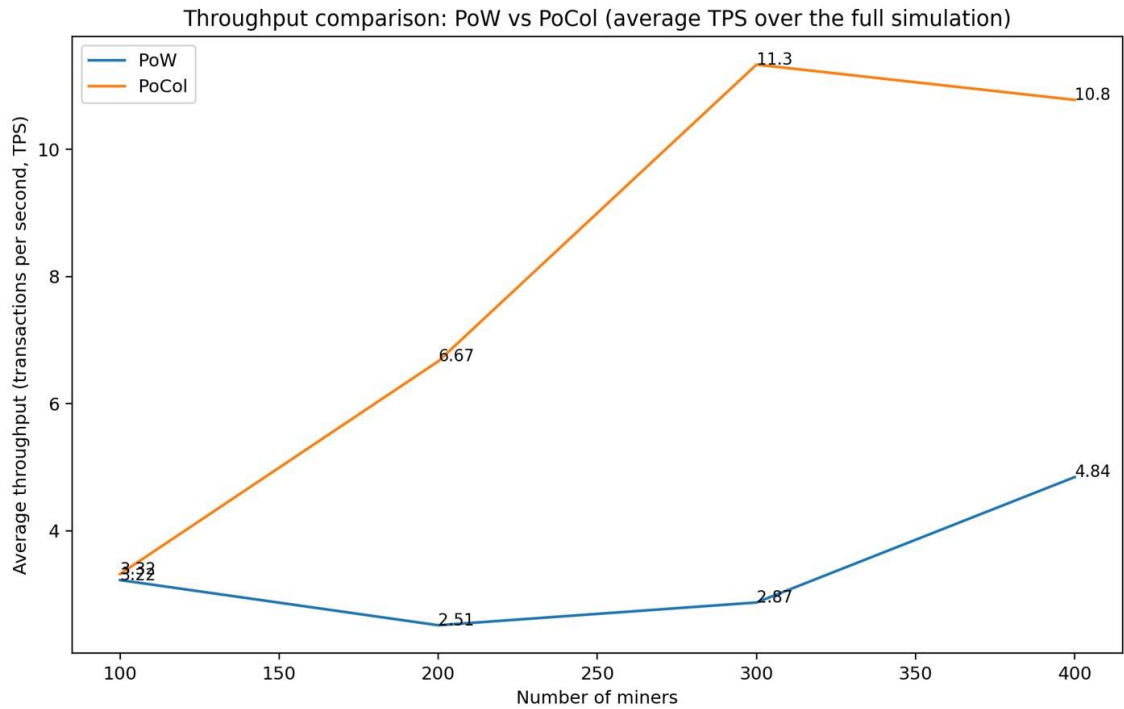


Figure (7.1): Average throughput over the full simulation horizon for PoCol and PoW.

The improved throughput performance is indicative of collaborative work allocation positively impacting the reduction of redundant search efforts, and thus increasing the quantity of useful work that was part of the accepted blocks. However, throughput alone does not provide a sufficient basis to evaluate the protocol, as the protocol may process many transactions, while concurrently exerting significant forking pressure. Therefore, when analysing these throughput results, they should be considered in conjunction with stale block performance and other finality metrics [140], [150].

7.3.2. Latency and approximate confirmation time

Both PoW and PoCol provide probabilistic rather than deterministic finality. This section therefore reports the approximate time to obtain six main-chain confirmations, derived from the average main-chain block time per run. At 100 miners, this approximate six-block confirmation time was approximately 3189 seconds for PoCol and 3128 seconds for PoW (see Figure 7.2). The approximate confirmation time for PoCol begins to drop rapidly in comparison to PoW as the number of miners increases beyond 100, with approximately 957 seconds for PoCol and 3635 seconds

for PoW at 300 miners, approximately 974 seconds for PoCol and 2182 seconds for PoW at 400 miners and approximately 1261 seconds for PoCol and 3033 seconds for PoW at 500 miners.

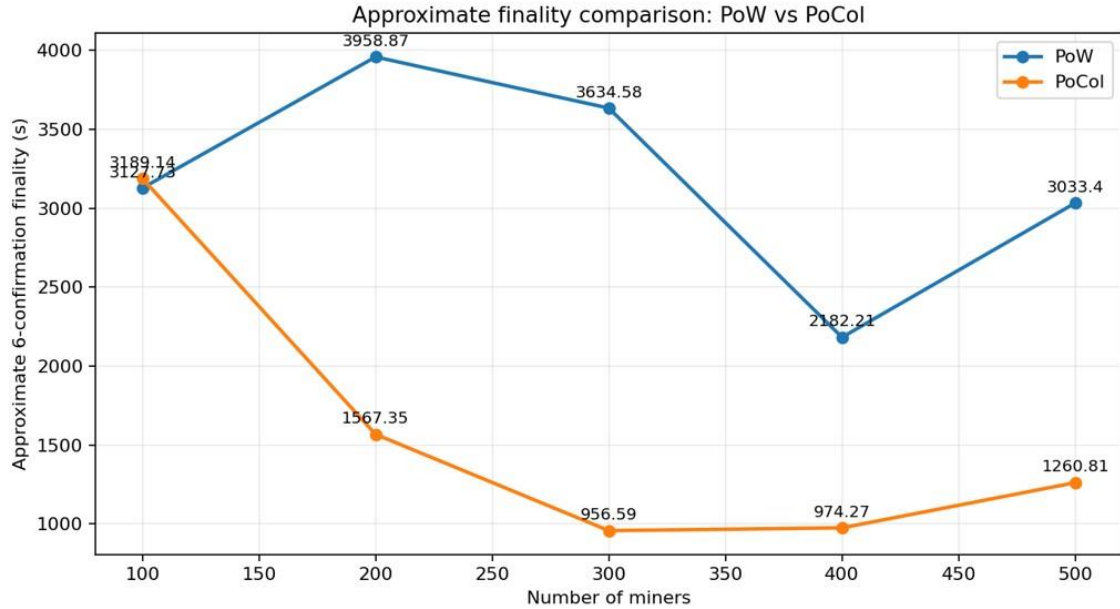


Figure (7.2): Approximate six-block confirmation time derived from the average main-chain block interval.

These findings indicate that PoCol produces accepted blocks on the main chain more frequently and thus is positioned to shorten effective confirmation times for all accepted blocks. However, since stale rates can significantly impact the effective security margin created by confirmation depth, the finality estimates given in this section should be interpreted with caution. For example, while a rapidly growing chain may have a high rate of staleness, it does not necessarily confer the same level of security confidence as a chain that grows slowly but has an extremely low rate of stale blocks [140], [76].

7.3.3. Scalability and fault-tolerance

PoCol's scalability is being evaluated based on its behavior using a miner set and observing how throughput grows and how stale rates behave during the mining experience. Stale-block rates for the PoW and PoCol protocols can be seen in Figure 7.3. Stale rates for PoW stay at 0% for all tested runs and the PoCol stale rate increases from 35.7% with 100 miners to 68.1% with 200 miners, 78.4% with 300 miners, and

82.7% with 400 miners. The trends established for the PoCol mining are continued in the logs for 500 miners. The overall conclusion is that PoCol is scalable in terms of throughput and energy-based leverage but that PoCol's chain cleanliness is unfavorably scaled based on its implementation to date.

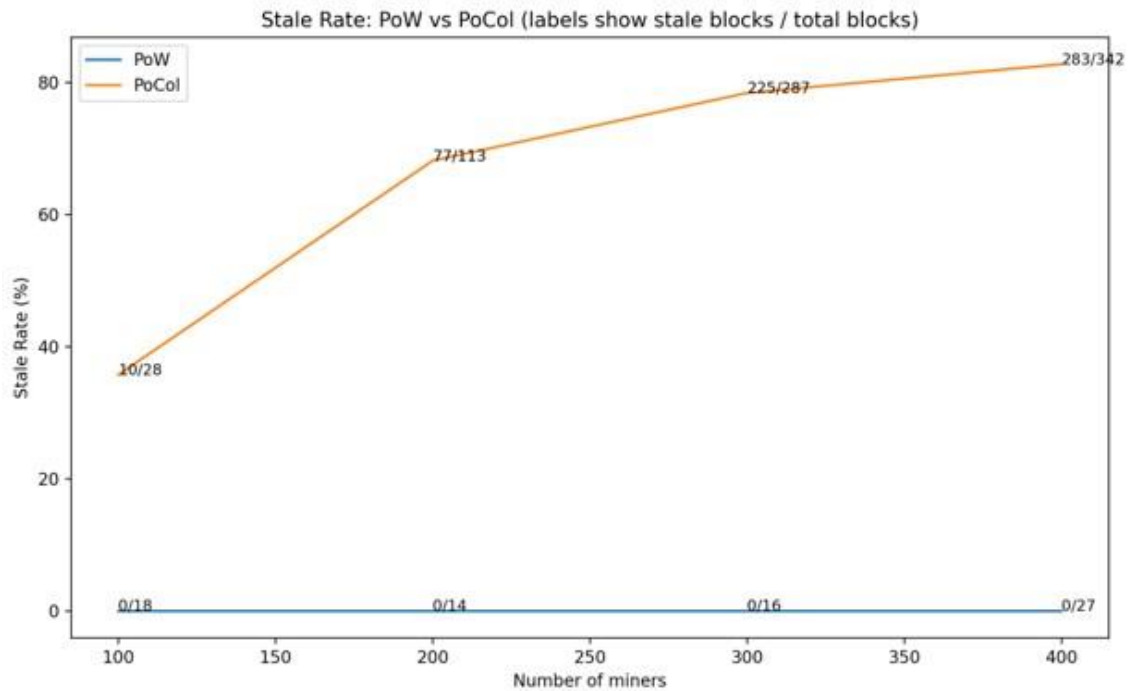


Figure (7.3): Stale-block rate for PoCol and PoW. Labels indicate stale blocks over total blocks.

While the perspective of fault-tolerance does not indicate a protocol-failure, PoCol's result does indicate sensitivity issues with respect to coordination over-head, propagation timing or round-synchronization. In practice, for PoCol, a stale-rate above 70% creates complications regarding wasted network resources, difficulties in aligning incentives and may also adversely impact confirmation-depth's effective security. Therefore, while PoCol displays strong potential in terms of operation due to designed efficiency, this efficiency is also limited by an implementation bottleneck that must be resolved prior to considering PoCol to be robust in a production environment [140], [150].

7.4. Energy Consumption Results

7.4.1. Total energy per block / per transaction

The largest improvement observed for the simulated PoCol abstraction concerns energy consumption. Figure 7.4 illustrates the total cumulative energy consumption of PoCol and PoW under varying numbers of miners. In simulations with 100 miners, PoW consumed approximately 7.90 kWh of electrical power, where PoCol consumed approximately 0.131 kWh (reduction of approximately 98.3% from PoW). With 400 miners, PoW consumed approximately 33.078 kWh whereas PoCol continued to consume approximately 0.454 kWh (reduction of approximately 98.6% from PoW). At 500 miners, PoW used approximately 40.408 kWh to supply the network while PoCol consumed 0.335 kWh.

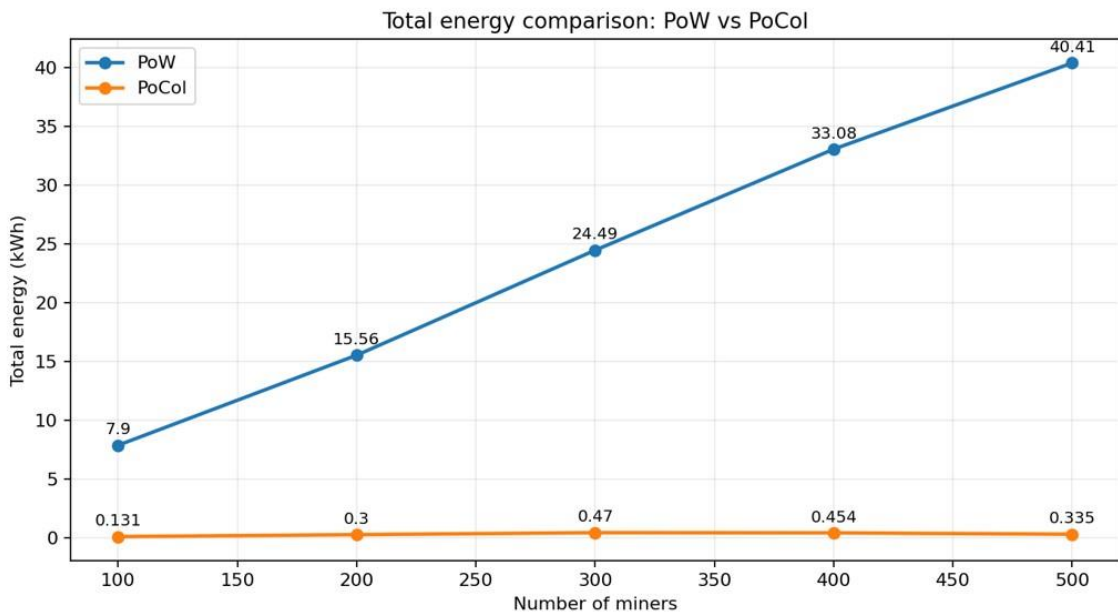


Figure (7.4): Total energy consumption for PoCol and PoW across miner populations.

When we normalize energy consumption by accepted blocks and accepted transactions, it becomes clearer that PoCol has a much lower cost per block than PoW. This is displayed in the energy requirements for every main-chain block in Figure 7.5. Since PoCol allows for many main-chain blocks to be created while keeping energy usage low, it has a much lower cost per block than PoW, which has costs that can be orders of magnitude higher than those of PoCol. The energy required per transaction

for PoCol is also indicated by Figure 7.6 and shows similar results to those found in Figure 7.5. In all cases, PoCol's energy usage per transaction remains relatively constant at approximately 0.004 - 0.005 Wh/transaction, whereas the energy required per PoW transaction varies from about 0.245 Wh/transaction at 100 miners to over 1.18 Wh/transaction at 500 miners.

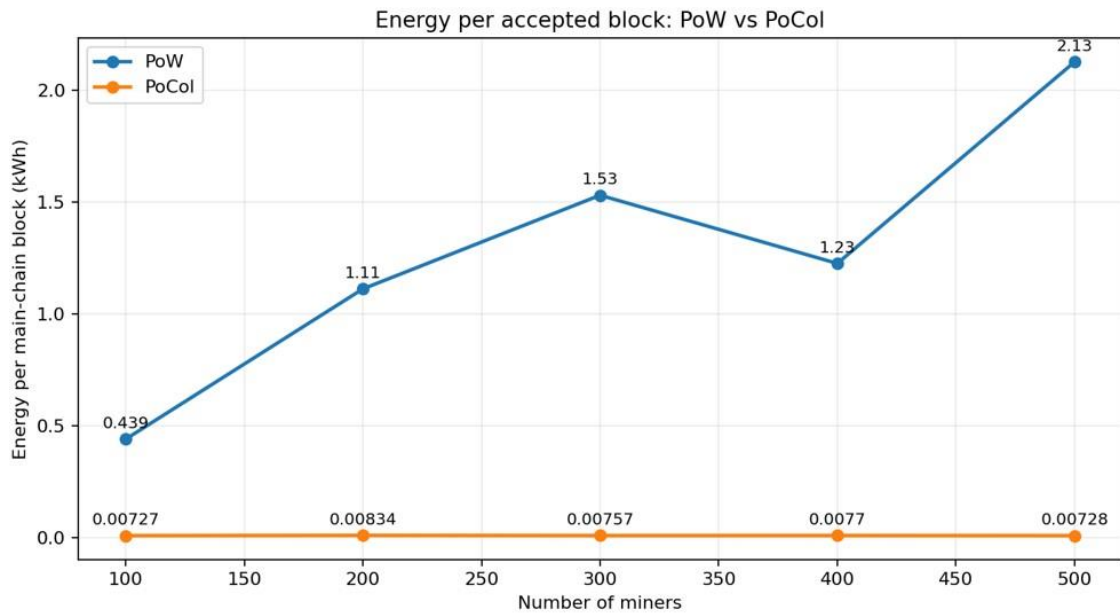


Figure (7.5): Energy per accepted main-chain block for PoCol and PoW.

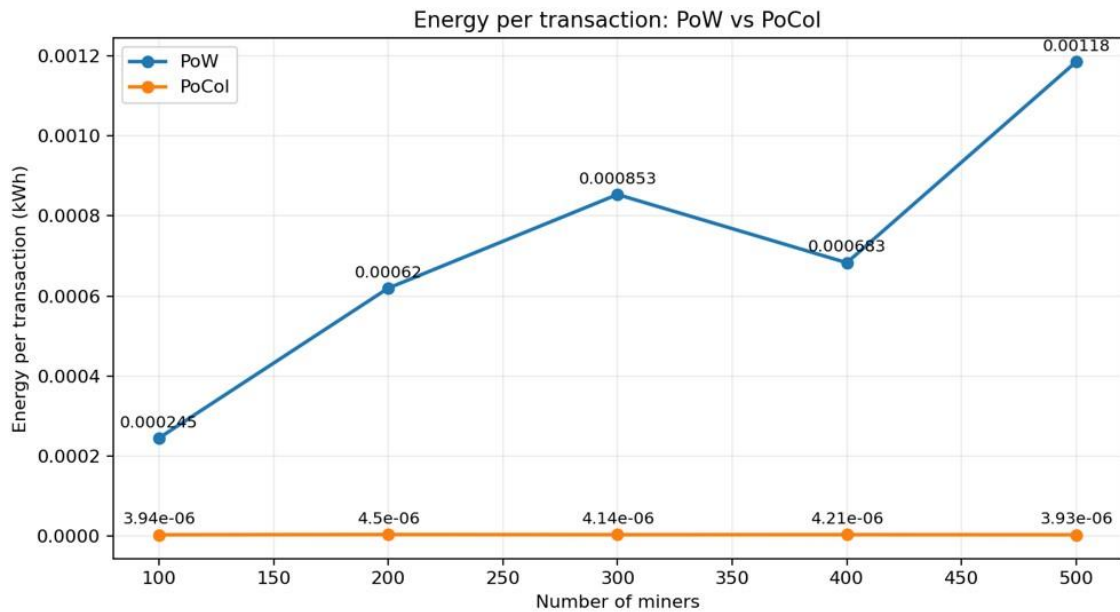


Figure (7.6): Energy per transaction for PoCol and PoW.

The time-series diagrams reinforce the same conclusion. Figure 7.7 illustrates cumulative energy over simulation time. The PoW curves rise steeply with miner count, whereas the PoCol curves stay compressed near the horizontal axis. This indicates that the event-driven collaborative model suppresses most of the redundant work that dominates PoW energy expenditure.

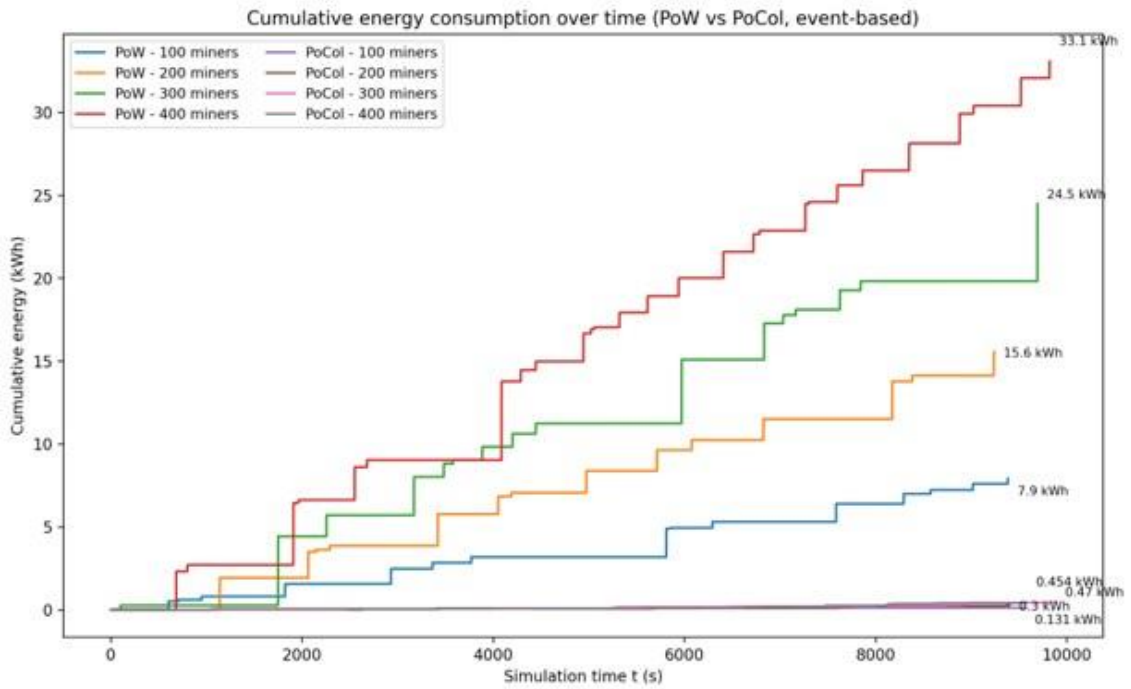


Figure (7.7): Cumulative energy over time for PoW and PoCol under the event-based accounting model.

7.4.2. Comparison with PoW, PoS, PBFT, etc.

When comparing PoW to PoCol, the simulated PoCol abstraction achieved significantly higher energy efficiency through greater energy and carbon cost reductions than the PoW baseline. This chapter shows that the operational behaviour of both PoCol and PoW show similar trajectories of lowering energy and carbon costs while concurrently maintaining the same style of permissionless chain growth process as PoW, however, to a lesser extent than PBFT.

However, at this time, we cannot categorically state that PoCol is superior to PoS or PBFT in overall energy efficiency because we did not perform matched runs of PoS or PBFT under the same simulator inputs as PoCol. It is worth noting, based on current evidence, that the simulated PoCol abstraction has achieved a substantial improvement

over the PoW baseline, and has narrowed some of the gap between traditional high-energy PoW model and lower energy consensus models discussed in the existing literature [76], [65].

7.4.3. Energy–performance trade-offs

Figure 7.8 compares total carbon footprint, which follows the same shape as the energy curves because the simulator converts energy into CO_2 using a fixed grid emission factor. At 400 miners, the total CO_2 of PoW is about 14.7 kg, compared with about 0.202 kg for PoCol. Thus, the reduction in carbon footprint is not a marginal improvement but a structural one.

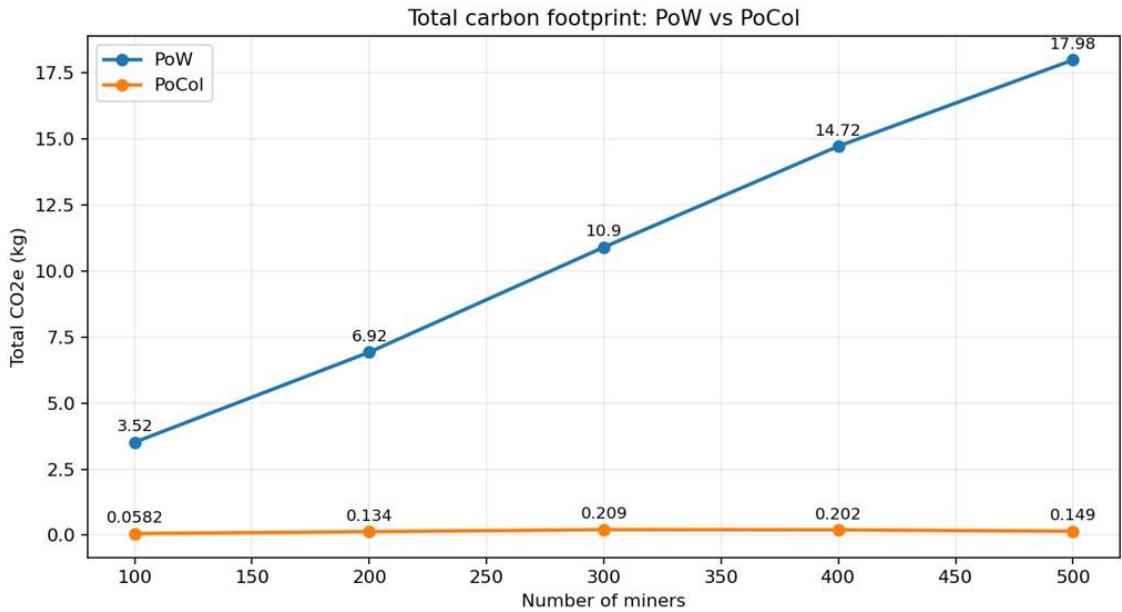


Figure (7.8): Total carbon footprint for PoCol and PoW.

The trade-off appears when these gains are compared with stale-block behavior. PoCol simultaneously improves throughput and reduces energy, which is unusual and desirable, however, it pays for this advantage with a sharp increase in stale blocks. Therefore, the present prototype should be interpreted as energy-efficient and performance-promising, but not yet fully optimized in protocol stability. The core research contribution is that energy reduction is achieved without sacrificing useful throughput, even though the resulting coordination dynamics still require refinement.

The energy-saving and continuous-performance policies represent different operating objectives. Under the idle policy, miners stop active hashing after

completing their assigned ranges, and energy savings depend on the measured idle power and idle duration. Under the continuous policy, miners remain active and may process further non-overlapping work. If active power and duration are equal to the PoW baseline, this mode should not be interpreted as inherently energy-saving. Any observed throughput advantage must be limited to the implemented configuration and reported together with the corresponding stale-block rate, block interval, active hash rate, and total energy.

7.5. Sensitivity Analysis

7.5.1. Impact of topology, delay, churn

While logs identified from previous studies did not include separate factorial experiments to identify topology, delay and churn, the measured growth of stale rates appears to indicate that there is a sensitivity to the variables measured. Simulation studies with discrete-event blockchain simulators have indicated that for whatever reason the growth in the number of stale blocks correlates with a larger percentage of block discovery interval when propagation delay is larger than it was previously or when node coordination is less consistent [40], [140]. By virtue of creating a larger number of proposed-block events in the same time period, the implementation of PoCol seems to be subject to greater propagation and coordination effects than PoW at this time. This therefore indicates that any future studies should differentiate delay distributions, peer degree assumptions, and churn processes on an experimental basis. By conducting the studies in this manner, the researcher will be able to identify whether the recent increase in the number of stale blocks has been caused primarily by propagation from the network, round synchronization overhead, or by the logical task of working collaboratively together. For instance, if PoCol is being considered for deployment on an open network with heterogeneous links and intermittent node participation, then it is critical that any studies conducted to evaluate PoCol study these sensitivity factors.

7.5.2. Impact of collaboration parameters

The current experiments vary miner population while using the prototype's abstract nonce-domain partitioning. They do not separately vary nonce-domain size,

range-allocation policy, or template-agreement parameters. The high stale-block rates therefore characterize the implemented event and propagation model under the tested configurations, they should not be described as a direct consequence of the mathematical nonce-domain partition itself.

To enhance PoCol's ability to provide optimal results with minimum energy costs, future research will require changes in Parameters Associated with Collaborations, including Nonce Partition Size and Timing, Collaborative Round Success Proposal Rule (after collaboration), Validation Delay Tolerance Threshold prior to advancement of Collaborative Rounds, Reward for Stale Block Acceptance vs. Accepted Collaborative Outcome. These variables will have a great impact on determining the optimal tradeoff between Chain Quality and Energy Savings with regards to PoCol.

7.6. Discussion of Results

7.6.1. Interpretation relative to research questions

The results answer the thesis research questions in a nuanced way. Regarding the question of whether collaboration can reduce redundant work and energy consumption, within the evaluated simulation setting, the answer is yes: by requiring a common immutable block template in each round and assigning disjoint nonce ranges, PoCol eliminates duplicate serialized candidate-header inputs among honest miners within a mining round, and this is the mechanism behind the observed reductions. Across all evaluated miner populations, PoCol drastically lowers total energy, energy per block, energy per transaction, and total CO_2 relative to PoW. Regarding the question of whether collaboration can preserve useful performance, the answer is also broadly yes: PoCol generally improves throughput and shortens the approximate six-block confirmation time. However, regarding robustness and scalability, the answer is conditional. The present prototype exhibits very high stale-block rates at larger miner counts, which means the collaboration concept is validated, but its operational design still requires refinement before it can be considered mature for production-scale deployment. Moreover, because the prototype evaluates an idealized shared-round abstraction, the reported figures do not include the communication, synchronization, latency, energy, or fork-related overhead of full

distributed canonical-template agreement, and should be interpreted as proof-of-concept results for the coordinated candidate-header evaluation abstraction rather than as measurements of a complete production implementation.

Therefore, the chapter supports the thesis claim that collaborative nonce-domain allocation is a powerful path toward sustainable blockchain consensus. At the same time, it shows that the design problem is not solved by energy efficiency alone. Chain quality, fork control, and synchronization discipline remain central requirements for a practical protocol [140], [76].

7.6.2. Practical implications for real-world deployments

The findings from this research indicate two major implications for future use cases of PoCol in production environments. For example, it may be possible to develop an environmentally-friendly PoW-like cryptocurrency using a permissionless mining model that maximally minimises the environmental impacts of blockchain (i.e. it offers an appealing opportunity to further research into developing more efficient mining processes). We recommend a substantial increase in the amount of both research funding and engineering development to continue improving upon PoCol.

In addition, it will also be necessary to implement a more rigorous and consistent mechanism for block propagation and coordinating rounds than what is currently demonstrated by the PoCol prototype, to minimise the high rate of stale blocks created by excessive forking and unpredictable reward mechanisms, potentially leading to poor security in an adverse environment and thus undermining the anticipated operational efficiencies provided by PoCol's efficient mining capabilities.

In conclusion, PoCol is currently positioned as a proof-of-concept and a valuable contribution to the overall development of greener blockchain systems. However, it is not yet a viable replacement for commercial-scale deployed public PoW-based ecosystems. As such, the next step in PoCol's development should concentrate on further reducing the number of stale blocks while maintaining PoCol's operational efficiency.

7.7. Summary

In this chapter, experimental data and assessment of PoCol versus PoW were presented through the results obtained through experimentation. When comparing PoCol with PoW using the BlockSim version of the extended BlockSim framework, PoCol provided significant benefits in the areas of reduced energy use (total energy consumption, energy per block, energy per transaction & total carbon footprint) across all miner groups included in the assessment, improved throughput (generally) and shorter approximate confirmation times within the main chain.

In summary, the results provide simulation-based proof-of-concept evidence supporting the thesis hypothesis that collaborative consensus can substantially improve the energy profile of PoW-style consensus, subject to the idealized template-agreement assumptions of the prototype.

However, this chapter also identified a key limitation to the PoCol process, the stale-block rate (i.e., number of stale blocks as a percentage of total blocks) increases exponentially as the miner population increases. As a result, the PoCol prototype worked well as a mechanism for creating efficiencies in mining and/or processing transactions, but still requires improvements at the protocol level to process chains at a better quality and scale. Therefore, the overall results of the study validate the primary contribution of the thesis, while identifying the biggest engineering challenges that need to be addressed in the future.

Chapter 8

Conclusion and Future Work

Chapter 8

Conclusion and Future Work

8.1. Summary of Research Work

This thesis examined the key problem that motivated this research: the means by which to achieve the open participation and secure characteristics features of Nakamoto-type blockchain consensus while reducing material redundancy in the workload and the resulting energy consumption from traditional proof of work (PoW) methods [1], [40]. In addressing this challenge, it developed a new consensus framework called Proof-of-Collaboration (PoCol) by maintaining hash-based block identification but changing candidate-header evaluation from an entirely competitive process to a coordinated approach in which all miners of a round mine the same immutable block template, the permitted nonce domain is divided into disjoint ranges bound to that template, and the collaborating miners share the rewards.

The research has evolved through five sequential steps. In the first place, the thesis compiled the literature on the topic of blockchain consensus, including classical block verification technology (BFT) as well as PBFT protocols, PoW methods among others, and energy-friendly alternatives like useful work and green blockchain [7], [65]. Next, the thesis established the system model, adversary model, energy use measurements, and optimization issue that make up the framework for using PoCol. The third stage established the specifications for the PoCol mechanism including miner registration, nonce-range allocations, coordinated candidate-header evaluation, reward distribution, and miner dynamic behavior of miners. Finally, the simulation was accomplished by expanding on the BlockSim framework to include energy use and carbon modeling measurements [150], [153].

The resulting outcomes indicate a combination of facts. On one hand, the simulated PoCol abstraction significantly reduced total energy consumption and CO₂ emissions in the simulation scenarios investigated, indicating an environmentally-friendly alternative to PoW of more traditional types. However, the same experiments show that the PoCol process in its current form has utilized high numbers of stale blocks as

the number of miners increased indicating that savings of energy do not produce superior levels of performance design.

8.2. Revisited Research Questions and How They Were Answered

The first research question examined how collaboration and resource sharing could be incorporated into a PoW-derived blockchain while retaining the intended advantages of open participation and adversarial resilience. PoCol addresses this question by requiring all miners in a round to use the same canonical block and nonce-free header templates and by assigning disjoint nonce ranges through a deterministic function of publicly verifiable blockchain state. Rather than saying that miners traverse the same unspecified search domain, the precise statement is that each miner evaluates a disjoint subset of the candidate-header input domain $D_r = \{\text{Serialize}(\text{Header}_r(n)) \mid n \in N_r\}$. Consequently, no two honest miners evaluate the same serialized candidate header within the round. This preserves the hash-based proof-of-work condition while reducing duplicate candidate-header inputs in the evaluated common-template model and supporting the proposed collaborative reward mechanism.

RQ2 focused on the question of how much energy could be saved when collaboration is used instead of the traditional model of competition, based on the simulation evidence provided in Chapters 6 and 7 about the energy-saving potential of PoCol. The results indicate that the simulated PoCol abstraction requires substantially lower energy expenditures than the PoW baseline under the evaluated simulation assumptions, this leads to fairly predictable carbon-footprint ratios, based on energy consumption values obtained for the energy expenditures presented. While the absolute figures vary by the models used, the sign and order of the magnitude of reduction present confirm the assumption made in this thesis that duplicate candidate-header evaluation is a modeled source of waste in the evaluated common-template PoW baseline, and that reducing those duplicate evaluations can lead to significant savings both from the perspective of sustainability and cost-effectiveness. Stated precisely, PoCol eliminates duplicate serialized candidate-header inputs among honest miners within a mining round by requiring a common immutable block template and

assigning disjoint nonce ranges, it does not eliminate every conceivable form of redundant computation in all PoW implementations.

Finally, the third research question addressed how realistic conditions influence fairness, throughput, scalability, and resilience. RQ3 was only partially answered: the simulation results indicate improvements in energy sustainability and throughput, but fairness, decentralization (for example, Gini coefficients over miner incomes), adversarial robustness, and pool-centralization behaviour were not experimentally evaluated in this thesis and remain future work, together with the evaluation of distributed template-agreement overhead under realistic network conditions [67], [27].

8.3. Contributions Revisited

Three levels of contributions are identified by thesis: conceptual, methodological, and empirical. With regard to the first level, the thesis presents PoCol as a new consensus method in the class of various blockchain mechanisms. The basic idea is reducing energy consumption not only by applying some new factor or more efficient computer hardware but showing that the initial assumption about open blockchain consensus having to be organized in fully duplicated computation race can be challenged. Thus PoCol offers us a new design principle: collaborative partition of proof-of-work computations.

In terms of methodology, the thesis delivers a built-from-scratch sustainability-conscious evaluation framework itself. The enhancements of BlockSim concerning the building of event-based model reflecting energy consumption and carbon emissions allow us to conduct the experimental comparison not limited only to throughput and latency. This is important because performance parameters alone are insufficient to estimate the sustainable blockchain design properly. Fast consensus mechanism consuming significantly more energy might be unacceptable in many particular situations, while low-energy protocol might cause chain instability. The combined methodology used for the evaluation is important in and of itself [150], [153].

At the empirical level, the thesis provides simulation-based comparative evidence. The conducted experiments show that the offered collaborative model allows achieving noticeable decrease in energy consumption and carbon emissions compared to PoW based system at the given assumptions while maintaining the throughput at a

satisfactory level as well. However, it was found out that there might be significant negative impact due to the stale blocks effect. This somewhat negative result is also a contribution itself. It shows where PoCol is functioning successfully and where the problems exist and provide information for further processing of the issue. In blockchain research this kind of clear demarcation is essential since it helps us to avoid making too general conclusions and provides sound basis for future research [24], [67].

8.4. Recommendations for Practitioners

When examining energy-efficient blockchain implementations, practitioners should approach the selection of consensus mechanisms as multi-objective engineering challenges, rather than simply using optimization methods based upon a single criterion (energy consumption). Practitioners should consider the interdependence of energy consumption, throughput, latency, stale-block rate, operational complexity and governance assumptions in their analyses when choosing between consensus mechanisms. The results of this thesis indicate that in those circumstances where sustainability is a priority objective, collaborative proof of work (PoW) type designs can be beneficial, however, the deployment conditions must limit the potential for coordination overhead to impede block propagation and permit chain divergence.

Practitioners of blockchain technology should incorporate simulation and instrumentation into routine protocol design tests. Before implementing changes to a consensus mechanism, practitioners should evaluate various network sizes, delay assumptions, and workload patterns to determine how the proposed mechanism will perform under these varying conditions. During their testing, practitioners should also utilize an evaluation framework that provides both performance and environmental metrics. The BlockSim extension developed for this research provided an example of how the integration of energy and carbon accounting into evaluation work can provide a comprehensive view of a blockchain's impact on the environment, instead of simply providing an estimate of the environmental impact at a later date [150]. For sectors where energy costs or public resources are paramount, such as supply chain operations, public services, and the coordination of renewable energy resources, a pre-deployment evaluation must be an integral part of the overall implementation process [76], [65].

Practitioners should exercise caution in their interpretation of energy savings as indicating immediate deploy ability. As demonstrated in this study, PoCol's sustainability benefits were accompanied by increased rates of stale blocks at the larger network sizes. As such, while collaborative consensus may be viewed as providing promise, practitioners should not assume that a diverse range of collaborative consensus mechanisms can be implemented in every environment. Deployments should be conducted within a controlled or semi-controlled environment, including a limited number of participants, stable connections, and significant monitoring, before proceeding to wider implementation within open networks. This careful and methodical approach aligns with the overarching learning of blockchain engineering that the soundness of any protocol is determined by its interaction between its mechanism design and the actual behavior of networks, not the mechanism alone [7], [140].

8.5. Future Research Directions

The next immediate step in improving the scalability performance of PoCol is implementing full common-template enforcement in the prototype (explicit canonical templates, TemplateID verification, and pre-committed reward-distribution transactions, as specified in Chapter 5), measuring the communication, synchronization, latency, and energy overhead of distributed template agreement under realistic network conditions, and reducing stale-block occurrences through enhanced nonce allocation coordination logic, refining the chain-selection and tie-breaking policies, developing lighter-weight synchronization windows, or combining collaborative mining with more efficient block-distribution methods. Because the current prototypical vulnerabilities revolve around how collaboration and block distribution interact, this area represents an appropriate place to begin follow-up work.

The second area for future study is the comparison of PoCol's performance with that of other consensus classes besides proof-of-work (PoW), for example, with Proof-of-stake (PoS), Practical Byzantine Fault Tolerance (PBFT), Proof-of-Authority (PoA) or Hybrid Protocols. The present work already includes an initial conceptual placement of PoCol against these other classes, but additional empirical benchmarking will provide greater support for where collaborative PoW is most competitive, and where

alternative consensus forms (e.g., non-PoW) continue to be superior [8], [27]. Specifically, future studies should compare throughput and energy, as well as governance assumptions, security thresholds, and operating costs.

A third area is to develop the energy model more robustly, currently, the simulation framework uses estimators integrated within BlockSim, which are useful tools to conduct comparative analyses. However, this method is still so abstracted from hardware-level measurements that a more accurate assessment of the simulated events' power consumption during operation and overall lifecycle carbon footprint would require additional calibration of existing miner devices, validator hardware, and heterogeneous-node classes. By conducting both simulations using, the existing procedure, and applying empirical calibrations from actual mining devices, validator hardware, and heterogeneous node classes, additional external validity and enhanced credibility will allow greater mapping of simulated events to actual power demand and lifecycle carbon impact [150], [153].

The fourth area is in developing adaptive collaboration policies. The current approach to collaboration is rigidly structured, however, one can envision developing adaptive protocols that can respond dynamically to specific network conditions (e.g., network congestion, miner reliability, and carbon-intensity signals) by adjusting partition sizes, modifying collaboration scopes, or changing reward sharing rules. In this regard, establishing a clearer connection will assist in linking consensus designs to new research literature on green blockchain and systems that account for sustainability [76], [65].

Lastly, future studies should also consider ways to facilitate real-world adoption of collaborative mining protocols. Important topics for consideration include incentive compatibility analyses, robustness against adversarial behaviour, interoperability with existing blockchain client software, phased pilot deployment strategies, and governance models for onboarding and managing collaborative miners. In this regard, the long-term viability of PoCol is to be considered not only in terms of whether PoCol can save energy (under simulation conditions), but also whether PoCol will work in practical terms as a secure, transparent, and operatively manageable protocol.

Therefore, future research will continue to integrate theoretical, simulation, and implementation-driven validation of PoCol's collaborative mining protocol.

References

- [1] S. Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System," white paper, 2008. [Online]. Available: <https://bitcoin.org/bitcoin.pdf>
- [2] L. Huang, L. Zhen, J. Wang, and X. Zhang, "Blockchain Implementation for Circular Supply Chain Management: Evaluating Critical Success Factors," *Industrial Marketing Management*, vol. 102, pp. 451–464, 2022, doi: 10.1016/j.indmarman.2022.02.009.
- [3] M. S. Farooq, U. Iftikhar, and A. Khelifi, "A Framework to Make Voting System Transparent Using Blockchain Technology," *IEEE Access*, vol. 10, pp. 59959–59969, 2022, doi: 10.1109/access.2022.3180168.
- [4] F. Ullah and F. Al-Turjman, "A Conceptual Framework for Blockchain Smart Contract Adoption to Manage Real Estate Deals in Smart Cities," *Neural Computing and Applications*, vol. 35, no. 7, pp. 5033–5054, 2023, doi: 10.1007/s00521-021-05800-6.
- [5] S. Vyas, M. Shabaz, P. Pandit, L. R. Parvathy, and I. Ofori, "Integration of Artificial Intelligence and Blockchain Technology in Healthcare and Agriculture," *Journal of Food Quality*, vol. 2022, pp. 1–11, 2022, doi: 10.1155/2022/4228448. [Retracted by the journal in 2024.]
- [6] S. Alam, S. Bhatia, M. Shuaib, M. M. Khubrani, F. Alfayez, A. A. Malibari, and S. Ahmad, "An Overview of Blockchain and IoT Integration for Secure and Reliable Health Records Monitoring," *Sustainability*, vol. 15, no. 7, Art. no. 5660, 2023, doi: 10.3390/su15075660.
- [7] L. Lamport, R. Shostak, and M. Pease, "The Byzantine Generals Problem," *ACM Transactions on Programming Languages and Systems*, vol. 4, no. 3, pp. 382–401, 1982, doi: 10.1145/357172.357176.
- [8] M. Castro and B. Liskov, "Practical Byzantine Fault Tolerance," in *Proc. 3rd Symp. Operating Systems Design and Implementation (OSDI '99)*, New Orleans, LA, USA, 1999, pp. 173–186. [Online]. Available: <https://www.usenix.org/conference/osdi-99/practical-byzantine-fault-tolerance>
- [9] D. Zhang, X. H. Chen, C. K. M. Lau, and B. Xu, "Implications of Cryptocurrency Energy Usage on Climate Change," *Technological Forecasting and Social Change*, vol. 187, Art. no. 122219, 2023, doi: 10.1016/j.techfore.2022.122219.
- [10] Cambridge Centre for Alternative Finance, "Cambridge Blockchain Network Sustainability Index," University of Cambridge Judge Business School. Accessed: Jul. 17, 2026. [Online]. Available: <https://ccaf.io/cbnsi>
- [11] K. McDonald, "Ethereum Emissions: A Bottom-up Estimate," *arXiv preprint arXiv:2112.01238*, 2021, doi: 10.48550/arXiv.2112.01238
- [12] M. Cortes-Goicoechea, L. Franceschini, and L. Bautista-Gomez, "Resource Analysis of Ethereum 2.0 Clients," in *2021 3rd Conference on Blockchain Research & Applications for Innovative Networks and Services (BRAINS)*, 2021, pp. 1–8, doi: 10.1109/BRAINS52497.2021.9569812.
- [13] A. Koutra and V. Tenentes, "Multi-Vt-Based Energy Efficiency Optimization for ASIC Designs of the Double Secure Hash Algorithm Toward a Sustainable Bitcoin Network," *IEEE Transactions on Circuits and Systems II: Express Briefs*, vol. 71, no. 3, pp. 1596–1600, 2024, doi: 10.1109/tcsii.2024.3350035.
- [14] H. L. Pham, T. H. Tran, T. D. Phan, V. T. Duong Le, D. K. Lam, and Y. Nakashima, "Double SHA-256 Hardware Architecture With Compact Message Expander for Bitcoin Mining," *IEEE Access*, vol. 8, pp. 139634–139646, 2020, doi: 10.1109/ACCESS.2020.3012581.
- [15] N. Lasla, L. Al-Sahan, M. Abdallah, and M. Younis, "Green-PoW: An Energy-Efficient Blockchain Proof-of-Work Consensus Algorithm," *Computer Networks*, vol. 214, Art. no. 109118, 2022, doi: 10.1016/j.comnet.2022.109118.
- [16] F. Bravo-Marquez, S. Reeves, and M. Ugarte, "Proof-of-Learning: A Blockchain Consensus Mechanism Based on Machine Learning Competitions," in *2019 IEEE International Conference on Decentralized Applications and Infrastructures (DAPPCON)*, 2019, pp. 119–124, doi: 10.1109/dappcon.2019.00023.
- [17] B. Li, C. Chenli, X. Xu, Y. Shi, and T. Jung, "DLBC: A Deep Learning-Based Consensus in Blockchains for Deep Learning Services," *arXiv preprint arXiv:1904.07349*, 2019, doi: 10.48550/arXiv.1904.07349

- [18] X. Qu, S. Wang, Q. Hu, and X. Cheng, "Proof of Federated Learning: A Novel Energy-Recycling Consensus Algorithm," *IEEE Transactions on Parallel and Distributed Systems*, vol. 32, no. 8, pp. 2074–2085, 2021, doi: 10.1109/TPDS.2021.3056773.
- [19] N. Sapra, I. Shaikh, and A. Dash, "Impact of Proof of Work (PoW)-Based Blockchain Applications on the Environment: A Systematic Review and Research Agenda," *Journal of Risk and Financial Management*, vol. 16, no. 4, Art. no. 218, 2023, doi: 10.3390/jrfm16040218.
- [20] M. Platt, J. Sedlmeir, D. Platt, J. Xu, P. Tasca, N. Vadgama, and J. I. Ibanez, "The Energy Footprint of Blockchain Consensus Mechanisms Beyond Proof-of-Work," in *2021 IEEE 21st International Conference on Software Quality, Reliability and Security Companion (QRS-C)*, 2021, pp. 1135–1144, doi: 10.1109/QRS-C55045.2021.00168.
- [21] M. Saad, Z. Qin, K. Ren, D. Nyang, and D. Mohaisen, "e-PoS: Making Proof-of-Stake Decentralized and Fair," *IEEE Transactions on Parallel and Distributed Systems*, vol. 32, no. 8, pp. 1961–1973, 2021, doi: 10.1109/tpds.2020.3048853.
- [22] P. Sethi, T. Nguyen, M. R. Chowdhury, S. Pirttikangas, and A. P. da Silva, "Fair consensus in blockchain with heterogeneous miners using reinforcement learning aided adaptive proof-of-work," in *2024 IEEE 21st Consumer Communications & Networking Conference (CCNC)*, pp. 937–942. IEEE, 2024. doi: 10.1109/CCNC51664.2024.10454844.
- [23] X. Shi, H. Xiao, W. Liu, K. S. Lackner, V. Buterin, and T. F. Stocker, "Confronting the Carbon-Footprint Challenge of Blockchain," *Environmental Science & Technology*, vol. 57, no. 3, pp. 1403–1410, 2023, doi: 10.1021/acs.est.2c05165.
- [24] A. de Vries, "Bitcoin's Growing Energy Problem," *Joule*, vol. 2, no. 5, pp. 801–805, 2018, doi: 10.1016/j.joule.2018.04.016.
- [25] J. Truby, "Decarbonizing Bitcoin: Law and Policy Choices for Reducing the Energy Consumption of Blockchain Technologies and Digital Currencies," *Energy Research & Social Science*, vol. 44, pp. 399–410, 2018, doi: 10.1016/j.erss.2018.06.009.
- [26] S. King and S. Nadal, "PPCoin: Peer-to-Peer Crypto-Currency with Proof-of-Stake," white paper, 2012. [Online]. Available: <https://www.peercoin.net/read/papers/peercoin-paper.pdf>
- [27] L. Ge, J. Wang, and G. Zhang, "Survey of Consensus Algorithms for Proof of Stake in Blockchain," *Security and Communication Networks*, vol. 2022, pp. 1–13, 2022, doi: 10.1155/2022/2812526.
- [28] BitShares, "Delegated Proof of Stake (DPOS)," BitShares Documentation. Accessed: Jul. 17, 2026. [Online]. Available: <https://docs.bitshares.org/en/master/technology/dpos.html>
- [29] V. Bachani and A. Bhattacharjya, "Preferential Delegated Proof of Stake (PDPoS)—Modified DPoS with Two Layers Towards Scalability and Higher TPS," *Symmetry*, vol. 15, no. 1, Art. no. 4, 2022, doi: 10.3390/sym15010004.
- [30] N. Arali, Narayan D. G., Altaf Husain M., and P. S. Hiremath, "An Efficient and Secure Blockchain Consensus Algorithm Using Game Theory," *International Journal of Computer Network and Information Security*, vol. 16, no. 2, pp. 92–102, 2024, doi: 10.5815/ijcnis.2024.02.08.
- [31] I. Barinov, V. Baranov, and P. Khahulin, "POA Network Whitepaper," POA Network, 2018. Accessed: Jul. 17, 2026. [Online]. Available: <https://github.com/poanetwork/wiki/wiki/POA-Network-Whitepaper>
- [32] M. A. Manolache, S. Manolache, and N. Tapus, "Decision Making Using the Blockchain Proof of Authority Consensus," *Procedia Computer Science*, vol. 199, pp. 580–588, 2022, doi: 10.1016/j.procs.2022.01.071.
- [33] S. Dziembowski, S. Faust, V. Kolmogorov, and K. Pietrzak, "Proofs of Space," in *Advances in Cryptology – CRYPTO 2015 (Lecture Notes in Computer Science)*, Springer, 2015, pp. 585–605, doi: 10.1007/978-3-662-48000-7_29.
- [34] K. Karantias, A. Kiayias, and D. Zindros, "Proof-of-Burn," in *Financial Cryptography and Data Security (Lecture Notes in Computer Science)*, Springer, 2020, pp. 523–540, doi: 10.1007/978-3-030-51280-4_28.
- [35] Hyperledger Sawtooth (Intel), "PoET 1.0 Specification," Sawtooth Documentation. Accessed: Jul. 17, 2026. [Online]. Available: <https://sawtooth.splinter.dev/docs/1.2/architecture/poet.html>
- [36] A. Yakovenko, "Solana: A New Architecture for a High Performance Blockchain," white paper, version 0.8.13. [Online]. Available: <https://solana.com/solana-whitepaper.pdf>

- [37] T. Xue, Y. Yuan, Z. Ahmed, K. Moniz, G. Cao, and C. Wang, "Proof of Contribution: A Modification of Proof of Work to Increase Mining Efficiency," in 2018 IEEE 42nd Annual Computer Software and Applications Conference (COMPSAC), 2018, pp. 636–644, doi: 10.1109/compsac.2018.00096.
- [38] Q. Zhuang, Y. Liu, L. Chen, and Z. Ai, "Proof of Reputation: A Reputation-Based Consensus Protocol for Blockchain Based Systems," in Proc. 2019 International Electronics Communication Conference (IECC), 2019, pp. 131–138, doi: 10.1145/3343147.3343169.
- [39] B. Lashkari and P. Musilek, "A Comprehensive Review of Blockchain Consensus Mechanisms," IEEE Access, vol. 9, pp. 43620–43652, 2021, doi: 10.1109/access.2021.3065880.
- [40] M. Alharby and A. van Moorsel, "BlockSim: A Simulation Framework for Blockchain Systems," ACM SIGMETRICS Performance Evaluation Review, vol. 46, no. 3, pp. 135–138, 2019, doi: 10.1145/3308897.3308956.
- [41] R. Paulavičius, S. Grigaitis, and E. Filatovas, "A Systematic Review and Empirical Analysis of Blockchain Simulators," IEEE Access, vol. 9, pp. 38010–38028, 2021, doi: 10.1109/ACCESS.2021.3063324.
- [42] Y. Hu, M. Liyanage, A. Mansoor, K. Thilakarathna, G. Jourjon, and A. Seneviratne, "Blockchain-Based Smart Contracts—Applications and Challenges," arXiv preprint arXiv:1810.04699, 2019, doi: 10.48550/arXiv.1810.04699.
- [43] N. Smolenski, "Blockchain for Education: A New Credentialing Ecosystem," in OECD Digital Education Outlook 2021: Pushing the Frontiers with Artificial Intelligence, Blockchain and Robots, Paris, France: OECD Publishing, 2021, doi: 10.1787/589b283f-en.
- [44] Y. Ge, W. Zhang, H. Li, and Z. Xie, "Traceability Analysis of Distributed New Energy System Access to Green Power Based on Blockchain Consensus Algorithm," International Journal of Emerging Electric Power Systems, vol. 25, no. 4, pp. 523–532, 2024, doi: 10.1515/ijeeps-2023-0092.
- [45] H. Taherdoost, "Blockchain Innovations, Applications, and Future Prospects," Electronics, vol. 13, no. 2, Art. no. 422, 2024, doi: 10.3390/electronics13020422.
- [46] J. A. Garay, A. Kiayias, and N. Leonardos, "The Bitcoin Backbone Protocol: Analysis and Applications," in Advances in Cryptology—EUROCRYPT 2015, Lecture Notes in Computer Science, vol. 9057, pp. 281–310, 2015, doi: 10.1007/978-3-662-46803-6_10.
- [47] R. Pass, L. Seeman, and A. Shelat, "Analysis of the Blockchain Protocol in Asynchronous Networks," in Advances in Cryptology—EUROCRYPT 2017, Lecture Notes in Computer Science, vol. 10211, pp. 643–673, 2017, doi: 10.1007/978-3-319-56614-6_22.
- [48] F. A. Sunny, P. Hajek, M. Munk, M. Z. Abedin, M. S. Satu, M. I. A. Efat, and M. J. Islam, "A Systematic Review of Blockchain Applications," IEEE Access, vol. 10, pp. 59155–59177, 2022, doi: 10.1109/access.2022.3179690.
- [49] G. Del Monte, D. Pennino, and M. Pizzonia, "Scaling Blockchains Without Giving Up Decentralization and Security: A Solution to the Blockchain Scalability Trilemma," in Proc. 3rd Workshop on Cryptocurrencies and Blockchains for Distributed Systems, 2020, pp. 71–76, doi: 10.1145/3410699.3413800.
- [50] K. Christidis and M. Devetsikiotis, "Blockchains and smart contracts for the internet of things," IEEE access, vol. 4, pp. 2292–2303, 2016, doi: 10.1109/ACCESS.2016.2566339.
- [51] B. Putz, F. Menges, and G. Pernul, "A Secure and Auditable Logging Infrastructure Based on a Permissioned Blockchain," Computers & Security, vol. 87, Art. no. 101602, 2019, doi: 10.1016/j.cose.2019.101602.
- [52] Y. Na, Z. Wen, J. Fang, Y. Tang, and Y. Li, "A Derivative PBFT Blockchain Consensus Algorithm with Dual Primary Nodes Based on Separation of Powers-DPNPBFT," IEEE Access, vol. 10, pp. 76114–76124, 2022, doi: 10.1109/access.2022.3192426.
- [53] OpenEthereum, "Aura – Authority Round," OpenEthereum Documentation. Accessed: Jul. 17, 2026. [Online]. Available: <https://openethereum.github.io/Aura>
- [54] S. Bano, A. Sonnino, M. Al-Bassam, S. Azouvi, P. McCorry, S. Meiklejohn, and G. Danezis, "SoK: Consensus in the Age of Blockchains," in Proc. 1st ACM Conference on Advances in Financial Technologies, 2019, pp. 183–198, doi: 10.1145/3318041.3355458.
- [55] L. Kleinrock, R. Ostrovsky, and V. Zikas, "Proof-of-Reputation Blockchain with Nakamoto Fallback," in Progress in Cryptology – INDOCRYPT 2020 (Lecture Notes in Computer Science), Springer, 2020, pp. 16–38, doi: 10.1007/978-3-030-65277-7_2.

- [56] L. Xu, D. Yu, J. Zhou, and C. Jin, "Blockchain-Based Decentralized Power Dispatching Model for Power Grids Integrated with Renewable Energy and Flexible Load," *Processes*, vol. 11, no. 6, Art. no. 1673, 2023, doi: 10.3390/pr11061673.
- [57] H. Li, H. Xue, K. Guo, R. Zhang, and S. Lu, "Study on the Power Data Assets and Trading System Design of Blockchain," *Journal of Physics: Conference Series*, vol. 2422, no. 1, Art. no. 012006, 2023, doi: 10.1088/1742-6596/2422/1/012006.
- [58] A. A. P. Machado, R. Fiorotti, R. da S. Villaça, and H. R. O. Rocha, "Profit Distribution Through Blockchain Solution from Battery Energy Storage System in a Virtual Power Plant Using Intelligence Techniques," *Journal of Energy Storage*, vol. 98, Art. no. 113150, 2024, doi: 10.1016/j.est.2024.113150.
- [59] X. Lu and Z. Guan, "A Blockchain-Based Trading Matching Scheme in Energy Internet," in *Proc. 2nd ACM International Symposium on Blockchain and Secure Critical Infrastructure*, 2020, pp. 142–150, doi: 10.1145/3384943.3409430.
- [60] V. B. Mišić, J. Mišić, and X. Chang, "Performance of PBFT Consensus under Voting by Groups," *Blockchains*, vol. 2, no. 2, pp. 134–149, 2024, doi: 10.3390/blockchains2020007.
- [61] X. Xu, G. Sun, and H. Yu, "An Efficient Blockchain PBFT Consensus Protocol in Energy Constrained IoT Applications," in *2021 International Conference on UK-China Emerging Technologies (UCET)*, 2021, pp. 152–157, doi: 10.1109/ucet54125.2021.9674967.
- [62] X. Zhang, R. Li, and H. Zhao, "A Parallel Consensus Mechanism Using PBFT Based on DAG-Lattice Structure in the Internet of Vehicles," *IEEE Internet of Things Journal*, vol. 10, no. 6, pp. 5418–5433, 2023, doi: 10.1109/jiot.2022.3222217.
- [63] S. Xu, C. Zhang, L. Wang, S. Zhang, and W. Shao, "A Cross Data Center Access Control Model by Constructing GAS on Blockchain," in *2022 IEEE 28th International Conference on Parallel and Distributed Systems (ICPADS)*, 2023, pp. 312–321, doi: 10.1109/icpads56603.2022.00048.
- [64] S. Tang et al., "Towards a Multi-Chain Future of Proof-of-Space," in *Security and Privacy in Communication Networks (Lecture Notes of the Institute for Computer Sciences, Social Informatics and Telecommunications Engineering)*, Springer, 2019, pp. 23–38, doi: 10.1007/978-3-030-37228-6_2.
- [65] J. Sedlmeir, H. U. Buhl, G. Fridgen, and R. Keller, "The Energy Consumption of Blockchain Technology: Beyond Myth," *Business & Information Systems Engineering*, vol. 62, no. 6, pp. 599–608, 2020, doi: 10.1007/s12599-020-00656-x.
- [66] A. Alofi, M. A. Bokhari, R. Bahsoon, and R. Hendley, "Self-Optimizing the Environmental Sustainability of Blockchain-Based Systems," *IEEE Transactions on Sustainable Computing*, vol. 9, no. 3, pp. 396–408, 2024, doi: 10.1109/tsusc.2023.3325881.
- [67] C. Stoll, L. Klaaßen, and U. Gallersdörfer, "The Carbon Footprint of Bitcoin," *Joule*, vol. 3, no. 7, pp. 1647–1661, 2019, doi: 10.1016/j.joule.2019.05.012.
- [68] J. I. Ibañez and F. Rua, "The Energy Consumption of Proof-of-Stake Systems: Replication and Expansion," *SSRN working paper*, 2023, doi: 10.2139/ssrn.4324137.
- [69] A. Khosravi and F. Säämäki, "Beyond Bitcoin: Evaluating Energy Consumption and Environmental Impact Across Cryptocurrency Projects," *Energies*, vol. 16, no. 18, Art. no. 6610, 2023, doi: 10.3390/en16186610.
- [70] Z. Zhao, Z. Fang, X. Wang, X. Chen, H. Su, H. Xiao, and Y. Zhou, "Proof-of-learning with incentive security," *arXiv preprint arXiv:2404.09005* (2024). doi: 10.48550/arXiv.2404.09005.
- [71] G. Ateniese, I. Bonacina, A. Faonio, and N. Galesi, "Proofs of Space: When Space Is of the Essence," in *Security and Cryptography for Networks (Lecture Notes in Computer Science)*, Springer, 2014, pp. 538–557, doi: 10.1007/978-3-319-10879-7_31.
- [72] B. Hammi, S. Zeadally, and J. Nebhen, "Security Threats, Countermeasures, and Challenges of Digital Supply Chains," *ACM Computing Surveys*, vol. 55, no. 14S, Art. no. 316, 2023, doi: 10.1145/3588999.
- [73] M. N. M. Bhutta et al., "A Survey on Blockchain Technology: Evolution, Architecture and Security," *IEEE Access*, vol. 9, pp. 61048–61073, 2021, doi: 10.1109/ACCESS.2021.3072849.

- [74] S. Islam, M. J. Islam, M. Hossain, S. Noor, K.-S. Kwak, and S. M. R. Islam, "A Survey on Consensus Algorithms in Blockchain-Based Applications: Architecture, Taxonomy, and Operational Issues," *IEEE Access*, vol. 11, pp. 39066–39082, 2023, doi: 10.1109/ACCESS.2023.3267047.
- [75] W. Wang et al., "A Survey on Consensus Mechanisms and Mining Strategy Management in Blockchain Networks," *IEEE Access*, vol. 7, pp. 22328–22370, 2019, doi: 10.1109/ACCESS.2019.2896108.
- [76] Y. Xiao, N. Zhang, W. Lou, and Y. T. Hou, "A Survey of Distributed Consensus Protocols for Blockchain Networks," *IEEE Communications Surveys & Tutorials*, vol. 22, no. 2, pp. 1432–1465, 2020, doi: 10.1109/COMST.2020.2969706.
- [77] A. Elsanousi, E. Pei, and K. Mershad, "Proactive Blockchain Deployment Mechanism in Resource-Constrained Rate-Splitting Multiple Access IoT Networks," *Internet of Things*, vol. 27, Art. no. 101328, 2024, doi: 10.1016/j.iot.2024.101328.
- [78] M. Sánchez-de la Rosa, C. Núñez-Gómez, M. B. Caminero, and C. Carrión, "Exploring the Use of Blockchain in Resource-Constrained Fog Computing Environments," *Software: Practice and Experience*, vol. 53, no. 4, pp. 971–87, 2023, doi: 10.1002/spe.3173.
- [79] D. Hu et al., "Ladder: A Convergence-Based Structured DAG Blockchain for High Throughput and Low Latency," in *Proc. 22nd USENIX Symposium on Networked Systems Design and Implementation (NSDI 25)*, 779–794. Philadelphia, PA: USENIX Association, 2025. <https://dl.acm.org/doi/10.5555/3767955.3767996>.
- [80] H. Lyu, S. Xie, J. Niu, I. Beschastnikh, Y. Zhang, M. Sadoghi, and C. Feng, "Orthrux: Accelerating Multi-BFT Consensus through Concurrent Partial Ordering of Transactions," in *Proc. 2025 IEEE 41st International Conference on Data Engineering (ICDE)*, 2615–2627, 2025. doi: 10.1109/ICDE65448.2025.00197.
- [81] H. Xu, X. Liu, C. Zhang, W. Wang, J. Wang, and K. Li, "Crackle: A Fast Sector-Based BFT Consensus with Sublinear Communication Complexity," in *Proc. IEEE INFOCOM 2024*, 1–10, 2024. doi: 10.1109/INFOCOM52122.2024.10621123.
- [82] M. J. Page et al., "The PRISMA 2020 Statement: An Updated Guideline for Reporting Systematic Reviews," *BMJ*, vol. 372, Art. no. n71, 2021, doi: 10.1136/bmj.n71.
- [83] M. J. Page et al., "PRISMA 2020 Explanation and Elaboration: Updated Guidance and Exemplars for Reporting Systematic Reviews," *BMJ*, vol. 372, Art. no. n160, 2021, doi: 10.1136/bmj.n160.
- [84] B. Kitchenham and S. Charters, *Guidelines for Performing Systematic Literature Reviews in Software Engineering*, EBSE Technical Report EBSE-2007-01, Keele University and Durham University, 2007. [Online]. Available: https://legacyfileshare.elsevier.com/promis_misc/525444systematicreviewsguide.pdf
- [85] IEEE Professional Communication Society, *Author Guidelines for Integrative Literature Reviews*, 2019. [Online]. Available: <https://procomm.ieee.org/wp-content/uploads/2019/05/Author-Guidelines-for-Integrative-Literature-Reviews.pdf>
- [86] K. Z. Azar, H. M. Kamali, H. Homayoun, and A. Sasan, "From Cryptography to Logic Locking: A Survey on the Architecture Evolution of Secure Scan Chains," *IEEE Access*, vol. 9, pp. 73133–73151, 2021, doi: 10.1109/ACCESS.2021.3080257.
- [87] E. Atay and J. L. Y. Terpstra Tong, "The Impact of Blockchain Technology on Human Resource Management," in *Handbook of Research on Social Impacts of E-Payment and Blockchain Technology*, 136–151. Hershey, PA: IGI Global, 2022. doi: 10.4018/978-1-7998-9035-5.ch009.
- [88] M. Rukhiran, S. Boonsong, and P. Netinant, "Sustainable optimizing performance and energy efficiency in proof of work blockchain: A multilinear regression approach," *Sustainability*, vol. 16, no. 4, Art. no. 1519, 2024, doi: 10.3390/su16041519.
- [89] J. Bonneau and N. Heninger, Eds., *Financial Cryptography and Data Security: 24th International Conference (FC 2020), Revised Selected Papers (Lecture Notes in Computer Science)*, Springer, 2020, doi: 10.1007/978-3-030-51280-4.
- [90] P. Szilágyi, "EIP-225: Clique Proof-of-Authority Consensus Protocol," *Ethereum Improvement Proposals*, 2017. [Online]. Available: <https://eips.ethereum.org/EIPS/eip-225>
- [91] Hyperledger Foundation, "Hyperledger Foundation," Accessed: Mar. 28, 2026. [Online]. Available: <https://hyperledger.github.io/>

- [92] S. Deb, R. Raynor, and S. Kannan, “Stakesure: Proof of stake mechanisms with strong cryptoeconomic safety,” arXiv preprint arXiv:2401.05797 (2024). doi: 10.48550/arXiv.2401.05797.
- [93] M. P. Asadauskas, “PoET: An Eco-Friendly Alternative to PoW,” thesis, University of Bern, 2021. [Online]. Available: <https://crypto.unibe.ch/archive/theses/2021.bsc.markus.asadauskas.pdf>
- [94] M. S. Ali, M. Vecchio, M. Pincheira, K. Dolui, F. Antonelli, and M. H. Rehmani, “Applications of Blockchains in the Internet of Things: A Comprehensive Survey,” *IEEE Communications Surveys & Tutorials*, vol. 21, no. 2, pp. 1676–1717, 2019, doi: 10.1109/COMST.2018.2886932.
- [95] J. Pennekamp, F. Alder, R. Matzutt, J. T. Mühlberg, F. Piessens, and K. Wehrle, “Secure End-to-End Sensing in Supply Chains,” in *2020 IEEE Conference on Communications and Network Security (CNS)*, 1–6. 2020. doi: 10.1109/CNS48642.2020.9162337.
- [96] J. Leng, M. Zhou, J. L. Zhao, Y. Huang, and Y. Bian, “Blockchain security: A survey of techniques and research directions,” *IEEE Transactions on Services Computing*, vol. 15, no. 4, pp. 2490–2510, 2022, doi: 10.1109/TSC.2020.3038641.
- [97] S. Jangirala, A. K. Das, and A. V. Vasilakos, “Designing Secure Lightweight Blockchain-Enabled RFID-Based Authentication Protocol for Supply Chains in 5G Mobile Edge Computing Environment,” *IEEE Transactions on Industrial Informatics*, vol. 16, no. 11, pp. 7081–93, 2020, doi: 10.1109/TII.2019.2942389.
- [98] H. Wang, G. Chen, Y. Zhang, and Z. Lin, “Multi-Certificate Attacks against Proof-of-Elapsed-Time and Their Countermeasures,” in *NDSS*. 2022, doi: 10.14722/ndss.2022.24158.
- [99] G. Tereshchenko and I. Kyrychenko, “Analysis and Justification of the Use of Existing Blockchain Solutions for the Protection of Digital Assets,” *Innovative Technologies and Scientific Solutions for Industries*, no. 1(27), pp. 164–178, 2024, doi: 10.30837/ITSSI.2024.27.164.
- [100] A. EL Azzaoui, H. Chen, S. H. Kim, Y. Pan, and J. H. Park, “Blockchain-Based Distributed Information Hiding Framework for Data Privacy Preserving in Medical Supply Chain Systems,” *Sensors*, vol. 22, no. 4, Art. no. 1371, 2022, doi: 10.3390/s22041371.
- [101] A. Bhoware, K. Jajulwar, A. Deshmukh, K. Dabhekar, S. Ghodmare, and A. Gulghane, “Performance Analysis of Network Security System Using Bioinspired-Blockchain Technique for IP Networks,” in *2023 11th International Conference on Emerging Trends in Engineering and Technology*. 2023. doi: 10.1109/ICETET-SIP58143.2023.10151475.
- [102] I. Abellán Álvarez, V. Gramlich, and J. Sedlmeir, “Unsealing the secrets of blockchain consensus: A systematic comparison of the formal security of proof-of-work and proof-of-stake,” in *Proc. 39th ACM/SIGAPP symposium on applied computing*, pp. 278–287. 2024. doi: 10.1145/3605098.3635970.
- [103] M. A. Zarkesh, E. Dastani, B. Safaei, and A. Movaghar, “Edgelinker: Practical blockchain-based framework for healthcare fog applications to enhance security in edge-iot data communications,” in *2024 5th CPSSI International Symposium on Cyber-Physical Systems (Applications and Theory)(CPSAT)*, pp. 1–8. IEEE, 2024. doi: 10.1109/CPSAT64082.2024.10745419.
- [104] R. S. Vairagade and B. S. Hanumantha, “Secure Internet of Things Network Using Light-Weighted Trust and Blockchain-Powered PoW Framework,” *Concurrency and Computation: Practice and Experience* 34, no. 21 (2022). doi: 10.1002/cpe.7057.
- [105] V. Kohli, S. Chakravarty, V. Chamola, K. S. Sangwan, and S. Zeadally, “An Analysis of Energy Consumption and Carbon Footprints of Cryptocurrencies and Possible Solutions,” *Digital Communications and Networks*, vol. 9, no. 1, pp. 79–89, 2023, doi: 10.1016/j.dcan.2022.06.017.
- [106] S. Hadi, W. Syaferi, and A. Wibowo, “Architectural Design of Electricity Power Consumption Misuse Detection Based on Light Gradient Boosting Machine Using Blockchain Technology,” in *Proc. 2023 International Conference on Technology, Engineering, and Computing Applications (ICTECA)*, 2023, doi: 10.1109/ICTECA60133.2023.10490464.
- [107] J. Dutta and D. Puthal, “PoAh 2.0: AI-Empowered Dynamic Authentication Based Adaptive Blockchain Consensus for IoMT-Edge Workflow,” *Future Generation Computer Systems*, vol. 161, pp. 655–72, 2024, doi: 10.1016/j.future.2024.07.048.

- [108] M. R. Islam, M. M. Rashid, M. A. Rahman, M. H. S. Mohamad, and A. H. Bin Embong, "A Comprehensive Analysis of Blockchain-Based Cryptocurrency Mining Impact on Energy Consumption," *International Journal of Advanced Computer Science and Applications*, vol. 13, no. 4, pp. 590–598, 2022, doi: 10.14569/IJACSA.2022.0130469.
- [109] A. C. D. Therry, R. Ardiansyah, M. Y. Pusadan, Y. Y. Joeffie, and A. A. Kasim, "The Implementation and Analysis of The Proof of Work Consensus in Blockchain," *Advance Sustainable Science, Engineering and Technology* 6, no. 1 (2024). doi: 10.26877/asset.v6i1.17878.
- [110] S. Prasanna, "Exploring the Architecture and Consensus Mechanisms of Hyperledger Fabric in Blockchain Technology," *JNAO*, vol. 15, no. 1, 2024. [Online]. Available: <https://jnao-nu.com/Vol.%2015,%20Issue.%2001,%20January-June%20-%202024/13.4.pdf>
- [111] L. Chen, L. Xu, N. Shah, Z. Gao, Y. Lu, and W. Shi, "On Security Analysis of Proof-of-Elapsed-Time (PoET)," in *Stabilization, Safety, and Security of Distributed Systems (Lecture Notes in Computer Science)*, Springer, 2017, pp. 282–297, doi: 10.1007/978-3-319-69084-1_19.
- [112] M. Bowman, D. Das, A. Mandal, and H. Montgomery, "On Elapsed Time Consensus Protocols," in *Progress in Cryptology – INDOCRYPT 2021 (Lecture Notes in Computer Science)*, Springer, 2021, pp. 559–583, doi: 10.1007/978-3-030-92518-5_25.
- [113] V. Shoup, "Proof of History: What Is It Good For," 2022. [Online]. Available: <https://www.shoup.net/papers/poh.pdf>
- [114] L. Alboaie, S. Alboaie, and C. Calancea, "Optimal Byzantine Fault Tolerance Consensus Algorithm for Permissioned Systems," in *Proc. 2021 20th RoEduNet Conference: Networking in Education and Research*, 2021, doi: 10.1109/RoEduNet54112.2021.9638279.
- [115] N. A. Wahab, D. Zhang, N. F. Juniardi, and Y. Wong, "Advances in consortium chain scalability: A review of the practical byzantine fault tolerance consensus algorithm," *International Journal of Advanced Computer Science and Applications (IJACSA)*, vol. 15, no. 7, 2024. doi: 10.14569/IJACSA.2024.0150796.
- [116] M. Castro and B. Liskov, "Practical byzantine fault tolerance and proactive recovery," *ACM Transactions on Computer Systems (TOCS)*, vol. 20, no. 4, pp. 398–461, 2002, doi: 10.1145/571637.571640.
- [117] W. Li, Q. Zhang, S. Deng, B. Zhou, B. Wang, and J. Cao, "Q-Learning Improved Lightweight Consensus Algorithm for Blockchain-Structured Internet of Things," *IEEE Internet of Things Journal*, vol. 11, no. 2, pp. 2855–2869, 2024, doi: 10.1109/JIOT.2023.3294265.
- [118] M. Aldwairi and L. Tawalbeh, "Security Techniques for Intelligent Spam Sensing and Anomaly Detection in Online Social Platforms," *International Journal of Electrical and Computer Engineering*, vol. 10, no. 1, pp. 275–287, 2020, doi: 10.11591/ijece.v10i1.pp275-287.
- [119] S. Y. Fayi and Z. Sheng, "A survey of security, privacy and trust issues in vehicular computation offloading and their solutions using blockchain," *Open Research Europe*, vol. 3, Art. no. 110, 2023, doi: 10.12688/openreseurope.16189.2.
- [120] C. P. Pascual Caceres et al., "Blockchain Architecture Based on Decentralised PoW Algorithm," *International Journal of Advanced Computer Science and Applications*, vol. 14, no. 7, 2023, doi: 10.14569/IJACSA.2023.0140776.
- [121] Q. Wang, R. Li, Q. Wang, S. Chen, and Y. Xiang, "Exploring unfairness on proof of authority: Order manipulation attacks and remedies," in *Proc. 2022 ACM on Asia Conference on Computer and Communications Security*, pp. 123–137. 2022. doi: 10.1145/3488932.3517394.
- [122] R. Sytnyk and V. Hnatushenko, "Study of consensus algorithms in blockchain networks in the design of information systems," *Міжнародна науково-технічна конференція Інформаційні технології в металургії та машинобудуванні (2024)*: 479–484. doi: 10.34185/1991-7848.itmm.2024.01.092.
- [123] M. Anus and A. Bin Ngadi, "Comparison Analysis of Blockchain Consensus Algorithms in Decentralized Public Environment: A Review," *Asia Proceedings of Social Sciences*, 2024, doi: 10.31580/jm7sfp03.
- [124] G. A. F. Rebello, G. F. Camilo, L. C. Guimaraes, L. A. C. de Souza, G. A. Thomaz, and O. C. M. Duarte, "A security and performance analysis of proof-based consensus protocols," *Annals of Telecommunications*, vol. 77, no. 7-8, pp. 517–537, 2022, doi: 10.1007/S12243-021-00896-2.
- [125] S. Pirani, "Burning Up: A Global History of Fossil Fuel Consumption," pp. 700–702, 2018, doi: 10.2307/j.ctv4ncp7q.

- [126] H. Ju, M. Kumar, E. Valavi, and S. Aral, "Explaining Sustained Blockchain Decentralization with Quasi-Experiments: The Resource Flexibility of Consensus Mechanisms," arXiv preprint arXiv:2505.24663 (2025). doi: 10.48550/arXiv.2505.24663.
- [127] Solana Foundation, "Solana's Energy Use Report: November 2021," 2021. [Online]. Available: <https://solana.com/news/solana-energy-usage-report-november-2021>
- [128] B. Fisch, "Tight Proofs of Space and Replication," in *Advances in Cryptology – EUROCRYPT 2019 (Lecture Notes in Computer Science)*, Springer, 2019, pp. 324–348, doi: 10.1007/978-3-030-17656-3_12.
- [129] O. Aluko and A. Kolonin, "Proof-of-Reputation: An Alternative Consensus Mechanism for Blockchain Systems," *International Journal of Network Security & Its Applications*, vol. 13, no. 4, pp. 23–40, 2021, doi: 10.5121/ijnsa.2021.13403.
- [130] J. Liang, X. Liu, X. Wu, and R. Chen, "Improvement and Application of Blockchain Consensus Algorithm for Distributed Power Energy System," in *2023 IEEE 8th International Conference on Big Data Analytics (ICBDA)*, 2023, pp. 77–82, doi: 10.1109/icbda57405.2023.10104765.
- [131] B. Buyukates et al., "Proof-of-Contribution-Based Design for Collaborative Machine Learning on Blockchain," in *2023 IEEE International Conference on Decentralized Applications and Infrastructures (DAPPS)*, 2023, pp. 13–22, doi: 10.1109/dapps57946.2023.00012.
- [132] L. Zhou, Y. Xian, Y. Yang, J. Jiang, P. Liu, and X. Li, "A Microgrid Trading Framework Based on PoC Consensus," arXiv preprint arXiv:2310.07165, 2023, doi: 10.48550/arXiv.2310.07165.
- [133] J. Sedlmeir, H. U. Buhl, G. Fridgen, and R. Keller, "Recent Developments in Blockchain Technology and Their Impact on Energy Consumption," arXiv preprint arXiv:2102.07886, 2021, doi: 10.48550/arXiv.2102.07886
- [134] S. S. Hazari and Q. H. Mahmoud, "Improving Transaction Speed and Scalability of Blockchain Systems via Parallel Proof of Work," *Future Internet*, vol. 12, no. 8, Art. no. 125, 2020, doi: 10.3390/fi12080125.
- [135] P. Szalachowski, D. Reijsbergen, I. Homoliak, and S. Sun, "StrongChain: Transparent and Collaborative Proof-of-Work Consensus," in *Proc. 28th USENIX Security Symposium*, Santa Clara, CA, USA, 2019, pp. 819–836.
- [136] R. Haque, S. M. T. Aziz, T. Hossain, F. H. Bappy, M. N. Yanhaona, and T. Islam, "Collaborative Proof-of-Work: A Secure Dynamic Approach to Fair and Efficient Blockchain Mining," in *Proc. IEEE 15th Annu. Computing and Communication Workshop and Conference (CCWC)*, 2025, doi: 10.1109/CCWC62904.2025.10903711.
- [137] N. Yonezawa, "Proof of Team Sprint: A Collaborative Consensus Algorithm for Reducing Energy Consumption in Blockchain Systems," *IET Blockchain*, vol. 6, no. 1, Art. no. e70034, 2026, doi: 10.1049/blc2.70034.
- [138] S. D. Lerner, "APoW: Auditable Proof-of-Work Against Block Withholding Attacks," arXiv preprint arXiv:2601.02496, 2026, doi: 10.48550/arXiv.2601.02496.
- [139] M. Baboi, "Security of consensus mechanisms in blockchain," *Romanian Cyber Security Journal*, vol. 5, no. 2, pp. 45–53, 2023, doi: 10.54851/v5i2y202305.
- [140] A. Gervais, G. O. Karame, K. Wüst, V. Glykantzis, H. Ritzdorf, and S. Capkun, "On the Security and Performance of Proof of Work Blockchains," in *Proc. 2016 ACM SIGSAC Conference on Computer and Communications Security*, 2016, pp. 3–16, doi: 10.1145/2976749.2978341.
- [141] K. Croman et al., "On Scaling Decentralized Blockchains (a Position Paper)," in *Financial Cryptography and Data Security (Lecture Notes in Computer Science)*, Springer, 2016, pp. 106–125, doi: 10.1007/978-3-662-53357-4_8.
- [142] C. Grunspan and R. Perez-Marco, "Proof of Reputation," arXiv preprint arXiv:2302.06966, 2023, doi: 10.48550/arXiv.2302.06966.

- [143] W. Yao, F. P. Deek, R. Murimi, and G. Wang, "SoK: A Taxonomy for Critical Analysis of Consensus Mechanisms in Consortium Blockchain," *IEEE Access*, vol. 11, pp. 79572–79587, 2023, doi: 10.1109/ACCESS.2023.3298675.
- [144] F. Raheman and T. Bhagat, "Blockchain in Logistics: A Protocol Selection Guide," Accessed: Mar. 28, 2026. [Online]. Available: <https://trace-horizon.eu/wp-content/uploads/2024/07/TRACE-Blockchain-in-Logistics-A-Protocol-Selection-Guide-White-paper.pdf>
- [145] M. S. Solmaz and M. Koray, "Blockchain Technology in Maritime Transportation and Management," in *Handbook of Research on the Applications of International Transportation and Logistics for World Trade*, 483–99. Hershey, PA: IGI Global, 2020. doi: 10.4018/978-1-7998-1397-2.ch026.
- [146] H. Taherdoost, "Blockchain Integration and Its Impact on Renewable Energy," *Computers*, vol. 13, no. 4, Art. no. 107, 2024, doi: 10.3390/computers13040107.
- [147] A. Gautam, R. Mahajan, and S. Zafar, "Revolutionary IoT and Big Data in Healthcare: A Predictive Model Analysis," *ICIDSSD*, vol. 2020, Art. no. 61, 2021, doi: 10.4108/eai.27-2-2020.2303135.
- [148] A. Hassan, M. I. Ali, R. Ahammed, M. M. Khan, N. Alsufyani, and A. Alsufyani, "Secured insurance framework using blockchain and smart contract," *Scientific Programming*, vol. 2021, no. 1, Art. no. 6787406, 2021, doi: 10.1155/2021/6787406.
- [149] S. M. Umran, S. Lu, Z. A. Abduljabbar, J. Zhu, and J. Wu, "Secure data of industrial internet of things in a cement factory based on a Blockchain technology," *Applied Sciences*, vol. 11, no. 14, Art. no. 6376, 2021, doi: 10.3390/app11146376.
- [150] R. S. Rasheed and A. A. AbuSamra, "Extending BlockSim with Energy and Carbon Footprint Modeling for Sustainable Blockchain Evaluation," unpublished manuscript, 2026.
- [151] C. Faria and M. Correia, "BlockSim: Blockchain Simulator," in *2019 IEEE International Conference on Blockchain (Blockchain)*, 2019, pp. 439–446, doi: 10.1109/Blockchain.2019.00067.
- [152] S. Smetanin, A. Ometov, M. Komarov, P. Masek, and Y. Koucheryavy, "Blockchain Evaluation Approaches: State-of-the-Art and Future Perspective," *Sensors*, vol. 20, no. 12, Art. no. 3358, 2020, doi: 10.3390/s20123358.
- [153] R. S. Rasheed, *BlockSim-New (modified BlockSim implementation with energy and carbon accounting)*, computer software, 2026. [Online]. Available: <https://github.com/raedrasheed/BlockSim-New>